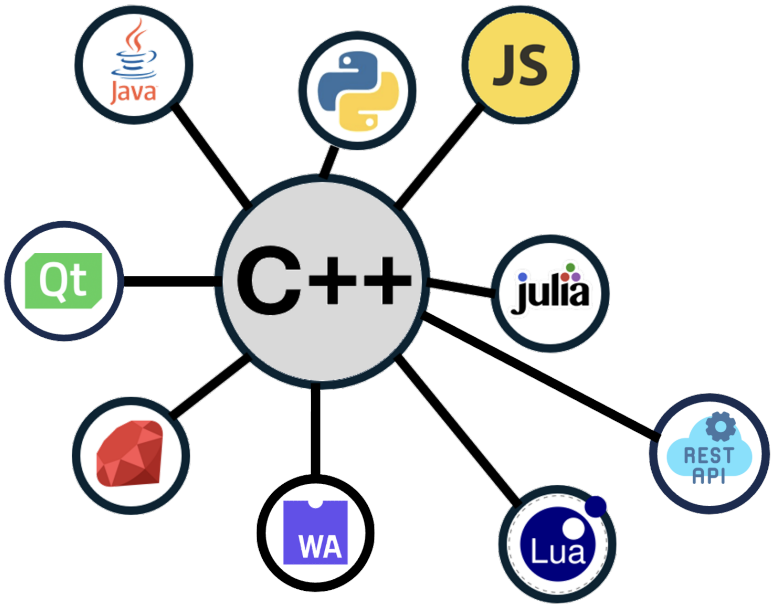
Rosetta

A User's Guide

Multi-language bindings for unmodified C++,
generated once with C++26 static reflection



Frantz Maerten

fmaerten@gmail.com

October 17, 2026

Preface

This book is about a single, narrow claim: that the information needed to expose a C++ library to Python, to JavaScript, to Lua, to a REST endpoint or to a property panel is *already in the header*, and that a compiler can be made to read it out.

Everything else follows from taking that claim seriously. You do not write a binding. You write down *what* you want bound — a list of headers and a list of target languages — and a program compiled against your types answers the rest by reflection. Your class definitions never change.

Who this is for

You have a C++ library. You want it callable from somewhere else, ideally from several somewheres, and you would rather not maintain a hand-written wrapper per destination. You are comfortable with CMake and with your own build.

You do *not* need to know anything about C++26 static reflection to use rosetta. Reflection is what makes the tool possible, and Chapter 2 explains where it runs and why the pipeline has the shape it does — but the surface you work with is a JSON file and a command-line tool.

How to read it

In a hurry
: Chapter 3 is a complete first binding, from an unmodified header to an importable Python module. Read that, then come back.

Evaluating rosetta
: Chapters 1 and 2 say what the system does and what it costs, including the parts that do not work.

Using it in earnest
: Part II is the manifest, field by field, and is the part you will return to. Part IV is the tool that consumes it.

Documenting what you bind
: Part III is the two ways metadata reaches a binding: annotations you write, and the Doxygen comments and default arguments your headers already carry (chapter 11, on by default).

Pushing past bindings
: Part V covers the runtime object model, scripting the metadata itself, and writing a backend of your own.

The appendices are lookup tables: every manifest field, every annotation, every target, and the errors you are most likely to hit.

Conventions

`Monospace` is a literal — a field name, a path, a command. A *manifest* always means the `manifest.json` driving one binding project. *Generation host* is the machine that runs the reflection compiler; *target* is wherever the generated binding is finally built, which is deliberately not the same machine.

Two callouts appear throughout:

> **NOTE**
>
> Something worth knowing that is not on the shortest path.

> **WATCH OUT**
>
> Something that will cost you an afternoon if you meet it unprepared.

Status

Rosetta is a prototype, and this book says so where it matters. It tracks in-flight C++ proposals — P2996 (reflection), P3394 (annotations), P3294 (token injection) — and is developed against the Bloomberg `clang-p2996` fork, which is currently the only front-end that carries all of what it uses. What the tool *generates* has no such constraint: that is the whole point, and Chapter 2 is where the distinction is made precise.

Related documents

The repository carries the reference material this book is built from — `docs/MANIFEST.md`, `docs/ROSETTA_GEN.md`, `docs/ANNOTATIONS.md`, `docs/COVERAGE.md` and the rest. Where a detail here is compressed, those are the long form. The academic write-up of the same system, with its evaluation and its related work, lives under `paper/`; it argues that reflection can serve as an interface description language, where this book simply assumes it and gets on with the job.

Contents

Preface **i**

I Understanding rosetta **1**

1 What rosetta is **2**
1.1 The problem, stated once . . . 2
1.2 The idea . . . 2
1.3 What you write . . . 3
1.4 What comes out without being asked for . . . 3
1.5 What you get out . . . 4
1.6 Reflection-free by construction . . . 4
1.7 The nineteen targets . . . 5
1.8 Annotations are optional . . . 5
1.9 What rosetta is not . . . 5
1.10 Where to go next . . . 6

2 The pipeline, and why it has that shape **7**
2.1 The constraint everything follows from . . . 7
2.2 The five steps . . . 7
2.3 Your library is read three times, for different reasons . . . 8
2.4 Two commands, not five . . . 9
2.5 When to re-run what . . . 9
2.6 The footgun the tool warns about . . . 10

3 Your first binding **11**
3.1 Prerequisites . . . 11
3.2 The whole build, in two commands . . . 11
3.3 The library — leave it alone . . . 12
3.4 The manifest . . . 13
3.5 Generate and compile . . . 13
3.6 What it produced . . . 14
3.7 Use it . . . 14
3.8 Read `coverage.json` once . . . 15
3.9 Enriching, if you want to . . . 15
3.10 Cleaning up . . . 16
3.11 What to read next . . . 16

II The manifest 17

4 Anatomy of a manifest 18
4.1 The minimum . . . 18
4.2 Paths resolve from the manifest . . . 18
4.3 Comments . . . 19
4.4 Variables . . . 19
4.5 Where the toolchain lives . . . 20
4.6 Naming things . . . 21
4.7 Shared defaults: `namespace` and `header_dir` . . . 21
4.8 Multiple include directories . . . 22
4.9 A field map . . . 23

5 Declaring what to bind 24
5.1 Classes . . . 24
5.2 Binding a whole folder . . . 25
5.3 Renaming: `expose` . . . 27
5.4 Free functions . . . 27
5.5 Extension methods . . . 28
5.6 Out-parameters . . . 29
5.7 Module init . . . 30

6 Targets 31
6.1 The two entry forms . . . 31
6.2 The available `lang` values . . . 31
6.3 Per-target linker flags . . . 32
6.4 Where the built artifact lands . . . 33
6.5 Pinning the runtime . . . 33
6.6 Choosing targets . . . 34

7 Making the generated projects build 35
7.1 The two ways to supply definitions . . . 35
7.2 Preprocessor definitions . . . 37
7.3 Build type and optimisation . . . 37
7.4 Headers the library's own build produces . . . 38
7.5 Python wheels . . . 38
7.6 A realistic manifest . . . 40

8 Types across the boundary 42
8.1 Foreign sequence containers . . . 42
8.2 Foreign matrices . . . 43
8.3 Foreign-library interop . . . 44
8.4 Two standard types that need no declaration . . . 45
8.5 Deciding what you need . . . 46

9 Presets 47
9.1 Using one . . . 47
9.2 What applying one does . . . 48
9.3 The shipped preset . . . 48
9.4 Writing your own . . . 48

III Annotations and documentation 50

10 Annotations 51
10.1 Two spellings, one result . . . 51
10.2 The annotation set . . . 52
10.3 Who consumes what . . . 53
10.4 Out-of-line annotations . . . 53
10.5 Internal annotations . . . 55
10.6 Gotchas . . . 55

11 Documentation 56
11.1 Turning it on . . . 56
11.2 An example, end to end . . . 56
11.3 What is read . . . 58
11.4 Which backend does what with it . . . 58
11.5 Default arguments, and why they are different . . . 59
11.6 Precedence . . . 60
11.7 What it will not do . . . 60
11.8 Where to look next . . . 61

IV Driving the tool 62

12 The `rosetta_gen` tool 63
12.1 Building it . . . 63
12.2 The four modes . . . 63
12.3 `-init`: starting a project . . . 64
12.4 `-build`: the whole pipeline . . . 65
12.5 Plain mode: one step at a time . . . 67
12.6 `-clean`: back to sources . . . 68
12.7 Exit status . . . 68

13 Overloads and coverage 69
13.1 Overloads . . . 69
13.2 `coverage.json` . . . 70
13.3 Using it . . . 71
13.4 Instrumenting a backend of your own . . . 71
13.5 A note for backend authors . . . 72

V Beyond bindings 73

14 The dynamic object model 74
14.1 What it emits . . . 74
14.2 Using it . . . 74
14.3 What the tables look like . . . 74
14.4 Why `register_all()` exists . . . 75
14.5 Calling by name . . . 75
14.6 Two consequences worth knowing . . . 76
14.7 What it buys . . . 76
14.8 What it costs . . . 76
14.9 When to reach for it . . . 77

15 Scripting the metadata itself **78**
15.1 The trick, in one sentence . . . 78
15.2 Why a facade rather than binding `dynamic.h` . . . 78
15.3 The one hand-written piece . . . 79
15.4 What a project has to write . . . 79
15.5 What it buys . . . 80
15.6 Running the pipeline twice . . . 80

16 Driving a library from an AI agent **81**
16.1 What an agent needs, and where it comes from . . . 81
16.2 A first server . . . 81
16.3 From your header to what the agent reads . . . 82
16.4 What a conversation looks like . . . 85
16.5 The tools . . . 87
16.6 Constraints the agent cannot bypass . . . 88
16.7 Scripts: one call instead of dozens . . . 89
16.8 Inside a running application . . . 90
16.9 Embedding it in your own application . . . 91
16.10Limits . . . 92

17 Writing a backend **93**
17.1 The mental model: two layers . . . 93
17.2 Layer 1: the runtime visitor . . . 93
17.3 Layer 2: the generation backend . . . 95
17.4 Registering it . . . 96
17.5 Wiring it into a project . . . 96
17.6 API reference . . . 97
17.7 Where to copy from . . . 98

VI Worked examples **99**

18 Worked examples **100**
18.1 The map . . . 100
18.2 The smallest case . . . 101
18.3 Documentation, without writing any . . . 101
18.4 A geometry library . . . 101
18.5 The dynamic model, end to end . . . 102
18.6 Rosetta in the wild . . . 102
18.7 Extending a generated binding by hand . . . 103

VII Reference **104**

A Manifest field reference **105**
A.1 Class entry fields . . . 107
A.2 Function entry fields . . . 107
A.3 Target entry fields . . . 107
A.4 `user_lib` entry fields . . . 108

B Annotation reference **109**
B.1 The kinds . . . 109

B.2 Widget hints . . . 110
B.3 Consumption by backend family . . . 110
B.4 Inline versus side-car . . . 110
B.5 Side-car schema . . . 110
B.6 Merge rules . . . 111
B.7 Internal . . . 111

C Target reference **112**
C.1 Every `lang` value . . . 112
C.2 Feature support . . . 112
C.3 Documentation output . . . 113
C.4 Packaging . . . 114
C.5 Interop and foreign types . . . 114
C.6 Build shapes for `-build` . . . 115
C.7 External requirements . . . 115

D Troubleshooting **116**
D.1 A method is missing from the binding . . . 116
D.2 The generated driver does not compile . . . 116
D.3 A side-car annotation had no effect . . . 117
D.4 The binding is slow . . . 117
D.5 The module will not load . . . 117
D.6 The pipeline produced stale output . . . 117
D.7 `-build` skipped a backend . . . 118
D.8 The wrong Python was used . . . 118
D.9 `-clean` did not remove something . . . 118
D.10 A variable in the manifest did not expand . . . 118
D.11 Reporting something . . . 118

Part I

Understanding rosetta

Chapter 1

What rosetta is

1.1 The problem, stated once

Exposing a C++ library to another language is normally paid for once per destination. A pybind11 module for Python. An N-API layer for Node. A sol2 usertype for Lua. A P/Invoke shim for C#. A moc-annotated hierarchy for a Qt panel. A JSON route table for a REST service.

Look at what those files actually contain. Here is one method, five times:

```
double Grid::at(int i, int j) const;
```

```
py::class_<Grid>(m, "Grid").def("at", &Grid::at);                // pybind11
ut["at"] = &Grid::at;                                            // sol2
exports.Set("at", Napi::Function::New(env, wrap_at));            // N-API
emscripten::class_<Grid>("Grid").function("at", &Grid::at);       // embind
[DllImport("grid")] static extern double Grid_at(IntPtr h, int i, int j);
```

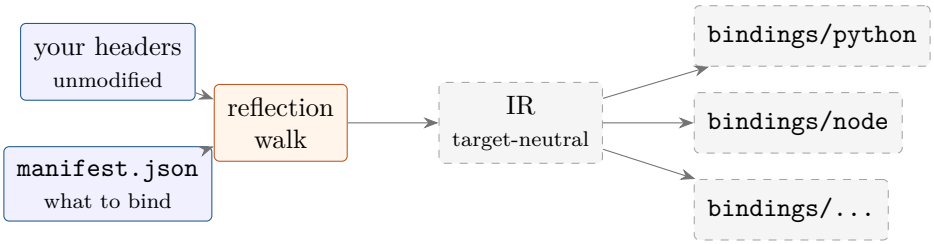
Every one of those restates, in a different notation, something the C++ compiler already knows: that `Grid` has a member called `at`, that it takes two `int`s, that it returns a `double`, that it does not mutate. The information is not missing. It is simply not reachable from anywhere except the compiler.

And the cost is not the initial write — it is the *derivative*. Add a parameter to `at` and five files are now wrong, in five languages, and four of them will not tell you until run time.

1.2 The idea

C++26 static reflection (P2996) makes that information reachable from inside C++ itself. A program can ask, at its own compile time, what members a type has, what their types are, whether they are public, whether they are virtual.

Rosetta does exactly one thing with that: it reads your headers *once*, through reflection, into a target-neutral intermediate representation, and then *projects* that representation into as many bindings as you asked for.



You declare *what* to bind. You never declare *how*.

1.3 What you write

Two things, and one of them you already have.

The header stays exactly as it is:

```
// my_lib/person.h
#include <string>

struct Person {
    std::string name;
    int         age = 0;
    std::string id;
    std::string greet(const std::string &salutation) const;
};
```

The manifest is the only new file:

```
{
  "user_include": "../my_lib",
  "rosetta_include": "/path/to/rosetta/include",
  "targets": [
    { "lang": "python", "name": "person_py" },
    { "lang": "node",   "name": "person_js" }
  ],
  "classes": [
    { "name": "Person", "header": "person.h" }
  ]
}
```

Then one command:

```
rosetta_gen --build manifest.json
```

and `bindings/python` and `bindings/node` exist, compiled, with `Person` in both — its three fields as properties, its method as a method, its constructors as constructors.

1.4 What comes out without being asked for

The manifest above names one class. Reflection supplies the rest of it:

- **Public fields**, as read/write properties with per-backend getters and setters.
- **Public methods**, instance and `static` alike.
- **Constructors** — every overload, not just the default one.

- **Inheritance**. Public base fields and methods are flattened into the derived binding; a derived declaration shadows the base one, and a virtual diamond collapses to a single member. Virtual methods are flagged, so backends that can generate trampolines do.
- **Overload sets**. The whole set reaches the IR; what each target does with it depends on whether its runtime can dispatch on argument types (Chapter 13).
- **Enums**, both plain and scoped, with their enumerators.
- **Nested user types and `std::vector`**, marshalled across the boundary — a `Surface` returning `std::vector<Triangle>` works without a line about `Triangle` beyond listing it.

Free functions are the one thing you must name, because a namespace has no member list to enumerate — you list them in the manifest, still without touching the header (Section 5.4).

1.5 What you get out

One self-contained project per target, under `bindings/`:

```
bindings/
  python/    auto_pybind.cpp      CMakeLists.txt  pyproject.toml  make_wheel.py
  node/      auto_napi.cpp        CMakeLists.txt  package.json
  wasm/      auto_emscripten.cpp  CMakeLists.txt
  typescript/ person.d.ts
  coverage.json
```

Each is an ordinary CMake (or npm, or emcmake) project. Nothing in it refers to rosetta at run time; nothing in it needs reflection to compile. That last point is worth its own section.

1.6 Reflection-free by construction

Every compiled backend *fully expands* what it found. There are no splices left in the output, no `<experimental/meta>`, no trace that reflection was ever involved — just explicit pybind11 / N-API / embind / sol2 / jlcxx calls, or, for the `dynamic` target, aggregate-initialised tables.

The consequence is the one that makes rosetta usable in practice:

> **NOTE**
>
> Reflection runs *once*, on the generation host. The machine that builds the generated binding needs an off-the-shelf toolchain — plain Clang, GCC, MSVC, a stock emsdk, a stock Qt 6. It does not need a C++26 compiler, and it does not need rosetta.

So the workflow for a project whose CI runs on a normal compiler is: generate on one machine that has the fork, commit or ship the generated tree, and build it anywhere.

Two targets are exceptions and are marked as such throughout: `rest` and `json` emit code that still splices reflections, so those two — and only those two — need the C++26 toolchain at their own build time.

1.7 The nineteen targets

Kind	`lang`	Output
Scripting	`python`	pybind11 extension module
	`nanobind`	nanobind extension module
	`node`	N-API native addon
	`wasm`	Emscripten / embind module
	`lua`	sol2 module, `require`-able
	`julia`	CxxWrap.jl / jlcxx shared module
Managed	`csharp`	C-ABI shared library + P/Invoke wrappers
	`java`	C-ABI + handle-backed FFM wrappers
User inter-	`qt`	Qt Widgets property/method inspector
faces	`qml`	QML / QtQuick inspector
	`imgui`	Dear ImGui inspector application
Services	`rest`	cpp-httplib JSON server + browser client
	`openapi`	OpenAPI 3.1 specification
	`json`	nlohmann (de)serialisation
Documents	`typescript`	ambient `.d.ts` declarations
	`markdown`	API reference document
	`html`	styled API reference page
	`paraview`	ParaView Server Manager plugin XML
Metadata	`dynamic`	the IR as runtime data (Chapter 14)

The full table, with what each one needs to build, is Appendix C. The last row is not a language binding at all and is the subject of its own chapter.

1.8 Annotations are optional

Reflection binds the class. *Annotations* enrich it — a docstring, a numeric range enforced at the boundary, a read-only property, a UI hint that turns a field into a slider:

```
[[= rosetta::doc{"Max solver iterations"}, = rosetta::range{0, 200},
   = rosetta::widget::slider ]]
int maxIter{100};
```

They are never required, they never change the C++ behaviour of your type, and you add them only where they pay off. If you would rather not touch the header at all — because it is a third party's, or because you want the generated binding to build on a stock compiler — the same metadata goes in a JSON side-car instead, wired by one manifest field. Chapter 10 covers both forms.

1.9 What rosetta is not

Being clear about this early saves disappointment later.

- **It is not a C++ parser.** It does not read your headers as text. A compiler does, and

rosetta asks the compiler. The one place a heuristic scan exists — `rosetta_gen -init` over a source tree — is explicitly a starting point you then edit.

- **It is not a runtime.** Nothing rosetta ships is loaded by your finished binding. The generated code depends on pybind11, or on N-API, or on sol2 — not on rosetta.
- **It does not bind everything.** Some C++ has no shape in the destination language: a `std::function` parameter, a raw pointer return, an operator with no name to bind to. Those members are *skipped*, never fatal, and every skip is recorded with a reason in `coverage.json` (Chapter 13). A binding that silently loses a method is the failure mode this design exists to avoid.
- **It is not finished.** It is a prototype tracking proposals that are themselves not final. See the status note in the Preface.

1.10 Where to go next

Chapter 2 explains why generating a binding is a *pipeline* rather than a command — which is the one piece of rosetta's shape that is not obvious, and the piece that explains most of the rest. If you would rather see it work first, skip to Chapter 3.

Chapter 2

The pipeline, and why it has that shape

Binding a library with rosetta is a *pipeline* rather than a single command. That is not an implementation choice — it is the only shape the problem admits. This chapter explains why, because almost every surprise in later chapters resolves back to it.

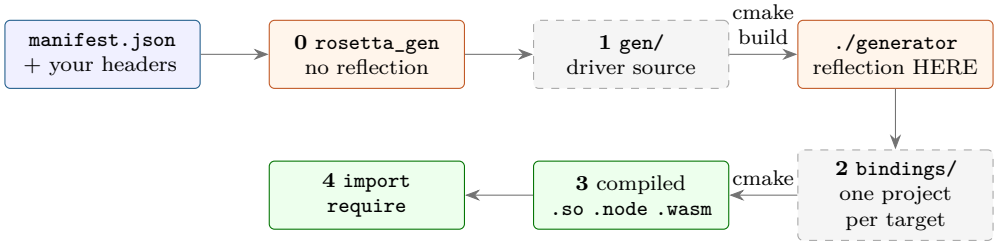
2.1 The constraint everything follows from

C++26 reflection answers questions **at the compile time of a program that includes your headers**. It cannot be queried from a script, from a configuration file, or from a build system. Only from C++, being compiled, with your types in scope.

So to learn the shape of your types, something has to **compile**. To turn those answers into files on disk, that something has to **run**.

Compile, then run. Everything below is that sequence, plus one step before it and one after.

2.2 The five steps



Step 0 — build `rosetta_gen`

`rosetta_gen` is a JSON-to-C++ text templater. It does no reflection at all and builds with an off-the-shelf C++17 compiler. You build it once.

It exists so that what *you* write is a manifest and not a driver program. Given `manifest.json` it emits three files: a `<name>.cpp` containing a `main()` that reflects, a `bindings.h` with your types (and any out-of-line annotations) baked in, and a `CMakeLists.txt` that knows where the reflection toolchain lives.

Your library: untouched. `rosetta_gen` never opens your headers. It only copies their names into the driver it writes.

Step 1 — compile and run the driver

Three commands, because "compile then run" is three things — emit the program, build it, execute it:

```
rosetta_gen manifest.json gen      # emit the driver source
cmake -S gen -B gen/build          # COMPILE it -- reflection happens HERE
cmake --build gen/build
./generator bindings               # RUN it -- writes one project per target
```

During the *compile*, clang-p2996 evaluates the reflection operators in the driver: it walks your classes, enumerates fields and methods, reads annotations, and resolves each member against the target language's type gate. **Every decision about what can and cannot be bound is made at this moment.** During the *run*, the driver does nothing clever — it prints the conclusions it was compiled with.

Neither half can be skipped. A compiler cannot write your binding files; a program cannot see types it was not compiled against.

Your library: read for the **first** time, as source, by the C++26 toolchain.

Step 2 — what lands in `bindings/`

One self-contained project per target, plus `coverage.json` — a machine-readable account of what bound and what did not, available *before* anything is compiled.

The generated code is fully expanded (Section 1.6). No splices, no `<experimental/meta>`, nothing that needs the fork.

Step 3 — compile a backend

Ordinary CMake, or npm, or emcmake, over an ordinary C++ project.

Your library: read a **second** time — the generated binding `#include`s your headers and calls your functions. This is where `user_sources` are compiled and `user_lib` is linked in.

Step 4 — load it

`import`, `require`, `dlopen`. Nothing rosetta-specific is left.

2.3 Your library is read three times, for different reasons

Pass	By	To	Needs C++26
step 0	`rosetta_gen`	**read** your comments	**no**
step 1	the reflection driver	**describe** your types	**yes**
step 3	the generated binding	**call** them	**no**

The middle row is the single most useful thing to hold in your head. The first is the odd one out and worth a sentence: comments are not reflectable — P2996 describes declarations, not the text around them — so the documentation your headers already carry is read *lexically*, by the tool, while it is emitting the driver. That pass compiles nothing and needs no toolchain;

what it finds is matched against what step 1 reflects, and dropped where the two disagree. Chapter 11 is about that pass.

The table has one further consequence worth stating on its own:

WATCH OUT

Annotations written **inline** — `[[= rosetta::doc{...} ]]` — pull `<experimental/meta>` into your header. Pass 2 then needs the C++26 toolchain as well, and the "no reflection on the target" promise is lost. Out-of-line annotations (a `.ann.json` side-car) keep your header plain C++ and pass 2 reflection-free. See Section 10.4.

2.4 Two commands, not five

You rarely run the steps by hand. `rosetta_gen -build` chains all of them:

```
rosetta_gen --build manifest.json
```

It emits `gen/`, builds and runs the driver, and then compiles every backend the manifest declares — each with its own build shape (plain `cmake`, `npm install` + `npm run build`, `emcmake`, a second-stage `dotnet build` or `mvn package`). A backend whose toolchain is missing is *skipped with a note*, not an error, and every run ends with a per-backend summary.

`rosetta_gen -clean manifest.json` takes you back to sources. Chapter 12 is the full reference for both.

The manual sequence remains useful for exactly the reasons you would expect: inspecting `bindings.h`, hacking on the emitted driver, or wiring the stages into a build system of your own.

2.5 When to re-run what

The two-pass structure decides this, and getting it wrong is the most common way to be confused by rosetta.

What changed	What to do
A field or method: added, removed, retyped	Rebuild the backend. The generated binding is expanded, so this needs a regeneration *if the member set changed* — re-run `-build`.
An **inline** annotation	Re-run `-build`: the annotation is read during pass 1.
A **side-car** `.ann.json`	Re-run `rosetta_gen` *and* the generator, not just the binding build — the JSON is baked into `bindings.h` at `rosetta_gen` time.
A class added, or `targets` changed	Re-run the whole pipeline.
A `user_sources` glob now matches new files	Re-run `rosetta_gen`: globs expand at generation time.
The manifest moved	Re-run. Every relative path in it resolves from the manifest's own directory.

NOTE

Re-emitting is idempotent: unchanged files are not rewritten, so the next `cmake -build` recompiles nothing. Running the pipeline again when you are not sure is cheap.

2.6 The footgun the tool warns about

If the emitted sources changed but a `generator` binary from a previous emit still sits next to the output directory, running it without rebuilding **silently re-emits the old bindings**. The tool detects this and prints a rebuild note. Believe it.

The related one: a fresh checkout (or a `-clean`) leaves `out_dir/build` unconfigured, so `cmake -build` alone fails. The *configure* step is printed as a `next:` hint when that happens.

Chapter 3

Your first binding

This chapter goes from an unmodified header to an importable Python module and a loadable Node addon. It is deliberately complete: every command, in order, with what each one produces.

3.1 Prerequisites

- A **clang-p2996** build. Rosetta looks for it at `$ENV{HOME}/devs/c++/clang-p2996/build` by default; anywhere else is fine, you just say where (Section 4.5).
- **CMake** 3.28 or later, and Ninja or Make.
- For the Python target, `pip install pybind11`. For Node, `npm`. For WebAssembly, an activated emsdk.

The reflection flags rosetta's generated projects use, should you need to know them: `-freflection -freflection-latest -fexperimental-library`, plus `-fannotation-attributes` for anything using annotations.

> **NOTE**
>
> Only the *generation host* needs the fork. Everything under `bindings/` builds with a stock compiler — with the two documented exceptions, `rest` and `json`.

3.2 The whole build, in two commands

Everything in this chapter reduces to two lines. Scaffold a binding project next to your library with `rosetta_gen -init` (Section 12.3) or, if you have no `rosetta_gen` yet, `tools/rosetta_init.py` — both write a `manifest.json`, a bootstrap `CMakeLists.txt`, a `.gitignore` and a `README.md` — and then:

```
# 1. one-time bootstrap: fetch rosetta into extern/ and build rosetta_gen
cmake -B build && cmake --build build --parallel

# 2. generate the generator, run it, and compile every target
./extern/rosetta/bin/rosetta_gen --build manifest.json -j
```

That is the whole pipeline. The rest of this chapter takes the two apart.

What the first command does

The bootstrap `CMakeLists.txt` fetches rosetta into `extern/` (an existing checkout is left alone) and builds the `rosetta_gen` tool. The binary always lands in `<rosetta>/bin/rosetta_gen` — not in the build tree — so from your project it is `./extern/rosetta/bin/rosetta_gen`.

This step needs **no reflection**: `rosetta_gen` is a JSON reader and a text templater, and builds with an off-the-shelf C++17 compiler. It is also the only one you run once — nothing below re-runs it.

What the second command does

`-build` chains the three stages of Chapter 2: it emits the generator project into `gen/`, builds it (**this** is the step that needs clang-p2996, and where reflection runs), runs it to write one project per target into `bindings/`, and then compiles each of those.

> **NOTE**
>
> `-j` is passed to *every* CMake build the command drives — the generator's and each backend's. Bare, it means one job per core; `-j8` caps it. (In the bootstrap, `-parallel` likewise leaves the job count to the build tool rather than hard-coding one.) Backends are still built one after another, so this parallelises within a backend, not across them; `-only python,node` is how you cut the matrix while iterating (Section 12.4.2).

Inside the rosetta repository instead

If you are working in a checkout of rosetta itself rather than a scaffolded project, there is no bootstrap to run — build the tool directly:

```
cmake -G Ninja -S tools/rosetta_gen -B tools/rosetta_gen/build
cmake --build tools/rosetta_gen/build --parallel
```

The binary lands in the same place, `<repo>/bin/rosetta_gen`. This is the form the examples in `examples/` use.

3.3 The library — leave it alone

```
// my_lib/person.h
#pragma once
#include <string>

struct Person {
    std::string name;
    int         age = 0;
    std::string id;

    Person() = default;
    Person(std::string n, int a) : name(std::move(n)), age(a) {}

    std::string greet(const std::string &salutation) const;
};
```

Note what is *not* there: no macro, no registration call, no rosetta include, no export annotation. That is the whole point.

3.4 The manifest

Put it wherever you like — every relative path inside resolves from the manifest's own directory. Here, `my_lib/rosetta/manifest.json`:

```
{
  "user_include": "..",
  "rosetta_include": "./extern/rosetta/include",
  "generator_name": "person_gen",
  "module_name": "person",

  "targets": [
    { "lang": "python", "name": "person_py" },
    { "lang": "node",   "name": "person_js" },
    "typescript",
    "markdown"
  ],

  "classes": [
    { "name": "Person", "header": "person.h" }
  ]
}
```

Four required ideas and nothing else:

`user_include`
where your headers live. One directory, or an array of them.

`rosetta_include`
where rosetta's `include/` lives.

`targets`
which backends to emit. A bare string uses `module_name`; the object form sets a per-target module name.

`classes`
what to bind. `header` is required; `name` defaults to the header's stem.

`generator_name` is optional — it names the emitted driver and defaults to the manifest's folder name.

> **NOTE**
>
> Do not want to write this by hand? `rosetta_gen -init` writes a fully-commented starter manifest (plus the bootstrap `CMakeLists.txt`, `.gitignore` and `README.md` beside it), and `rosetta_gen -init manifest.json ./src` pre-fills it from a scan of your source tree. See Section 12.3.

3.5 Generate and compile

The second command from Section 3.2:

```
./extern/rosetta/bin/rosetta_gen --build manifest.json -j
```

It emits the generator project, builds it (this is where reflection runs), runs it, and then compiles every backend the manifest declares. The run ends with a per-backend summary:

```
== summary
   python      OK
   node        OK
   typescript          OK (nothing to compile)
   markdown            OK (nothing to compile)
```

A backend whose toolchain is missing is skipped with a note rather than failing the run:

```
   wasm        SKIPPED (emcc not found -- activate emsdk)
```

3.6 What it produced

Two directories. `gen/` is the generator project — three files, written from the manifest and then compiled and run for you:

`person_gen.cpp`	a `main()` that calls `rosetta::generate<Person>(...)` — the driver
`bindings.h`	your headers included, plus any baked-in out-of-line annotations
`CMakeLists.txt`	knows where clang-p2996 is and which flags it needs

And `bindings/` is the output — one self-contained project per target:

```
bindings/
  coverage.json
  python/      auto_pybind.cpp  CMakeLists.txt  pyproject.toml  make_wheel.py
  node/        auto_napi.cpp    CMakeLists.txt  package.json
  typescript/  person.d.ts
  markdown/    person.md
```

Open `auto_pybind.cpp`. It is ordinary pybind11 — explicit `.def()` calls, one per member, with no reflection anywhere. That file is shippable: it compiles on any machine with pybind11 and a normal C++ compiler.

> **NOTE**
>
> `-build` drives these stages for you, but they can be run one at a time — emit `gen/`, build it, run the generator — which is what you want when inspecting `bindings.h`, hacking on the emitted driver, or wiring the stages into a build system of your own. That is *plain mode*, covered in Chapter 12.

3.7 Use it

Python

```python
import person_py

p = person_py.Person("Alice", 30)
p.name = "Alice B."
print(p.greet("Hello"))
print(p.age)
```

Node

```
const m = require('./build/Release/person_js.node');
const p = new m.Person('Alice', 30);
console.log(p.greet('Hello'));
```

The two you did not have to build

`bindings/typescript/person.d.ts` declares the Node addon's surface, so an editor autocompletes it. `bindings/markdown/person.md` is an API reference page. Neither compiles anything — they are text, produced directly.

3.8 Read `coverage.json` once

Before moving on, open `bindings/coverage.json`. Even for a three-field class it tells you something worth having: exactly which members reached each backend and which did not, with a reason for every drop.

```
{
  "rosetta_coverage": 1,
  "reflection": [],
  "targets": [
    { "target": "python", "bound": 5, "skipped": 0, "classes": [ ... ] }
  ]
}
```

On a real library it will not be empty, and the habit of checking it is the difference between knowing your binding surface and guessing at it. Chapter 13 is about reading it properly.

3.9 Enriching, if you want to

Everything above used reflection alone: names match the C++ identifiers, types map automatically, and you get no docstrings, no validation and no read-only semantics. Adding them is opt-in, member by member:

```
#include <rosetta/annotations.h>

struct Person {
    [[= rosetta::doc{"display name"}]]
    std::string name;

    [[= rosetta::doc{"age in years"}, = rosetta::range{0.0, 150.0}]]
    int age = 0;

    [[= rosetta::doc{"server-assigned id"}, = rosetta::readonly{}]]
    std::string id;
};
```

Now `help(person_py.Person.name)` says something, assigning `200` to `age` raises, and `id` has no setter.

> **WATCH OUT**
>
> Writing annotations inline makes the header C++26-only — every translation unit that includes it, including the generated binding, now needs the fork. If you want to keep the

header plain C++ (and you usually do), use the JSON side-car instead: same metadata, same result, one manifest field. See Section 10.4.

3.10 Cleaning up

```
rosetta_gen --clean manifest.json
```

removes `gen/`, the generator binary, and the per-backend folders under `bindings/` *that this manifest declares*. Anything else you parked there survives, and a folder whose `CMakeLists.txt` lacks the `Generated by rosetta` stamp is never touched. The manifest itself is never removed.

3.11 What to read next

You now have a working binding. Part II is the manifest in full — and the next chapter starts with the rules that apply to all of it.

Part II

The manifest

Chapter 4

Anatomy of a manifest

The manifest is the single hand-written file that drives rosetta. This chapter covers the rules that apply to *all* of it — how paths resolve, how to write comments, how to factor out repetition — before the following chapters go field by field.

It answers three questions for the framework:

1. **Where** are your headers, and where is rosetta's `include/`?
2. **What** classes and free functions should be bound?
3. **Which** language backends should be emitted?

Everything else — fields, methods, constructors, enums, inheritance — is discovered by reflection from the headers themselves.

4.1 The minimum

Three keys are required: `user_include`, `rosetta_include` and `targets`, plus `classes` unless a preset supplies it.

```
{
  "user_include": "./geom",
  "rosetta_include": "../../include",
  "targets": ["python"],
  "classes": [ { "header": "Point.h" } ]
}
```

That is a complete, working manifest.

4.2 Paths resolve from the manifest

Every relative path in the file is resolved against **the directory holding the manifest**, not the current working directory and not the output tree. Absolute paths are taken as written.

> **WATCH OUT**
>
> Move the manifest and you must re-run `rosetta_gen`. Nothing detects a stale resolution for you.

There is one deliberate exception, and it is the one people trip over: a class entry's `header` is resolved against `user_include`, not against the manifest. That is because the header is emitted into an `#include "..."` line in generated code — it must be spelled relative to the *include path*, exactly as the compiler will look for it. `user_sources`, which name files to compile rather than to include, do resolve from the manifest.

4.3 Comments

JSON has no comment syntax, so the manifest uses two conventions, and both work everywhere:

```
{
  "//": "Bindings for the geom library.",
  "//paths": "All relative paths resolve from THIS file's directory.",

  // Line comments are tolerated by the parser too.
  /* As are block comments. */
  "user_include": "./geom"
}
```

Any key beginning with `//` is ignored — `"//1"`, `"//note"`, `"//cpp26"`. Keys must still be unique, which is why the numbered and named forms exist. The `//` and `/* */` line forms are accepted by the parser as an extension; the key form is plain JSON and survives any tool that reads the file.

4.4 Variables

A path that appears in several fields would otherwise be written out in each of them. The `cpp26_*` block is the case that motivated this — one toolchain root, spelled four times with a different suffix:

```
"cpp26_root": "$ENV{HOME}/devs/c++/clang-p2996/build",
"cpp26_cxx":  "$ENV{HOME}/devs/c++/clang-p2996/build/bin/clang++",
"cpp26_cc":   "$ENV{HOME}/devs/c++/clang-p2996/build/bin/clang",
"cpp26_lib":  "$ENV{HOME}/devs/c++/clang-p2996/build/lib",
```

`variables` declares a name for the shared part, which `$NAME` (or `${NAME}`) then stands for:

```
"variables": [
    { "name": "P2996", "value": "$ENV{HOME}/devs/c++/clang-p2996" }
],

"cpp26_root": "$P2996/build",
"cpp26_cxx":  "$P2996/build/bin/clang++",
"cpp26_cc":   "$P2996/build/bin/clang",
"cpp26_lib":  "$P2996/build/lib",
```

The substitution runs over **every string in the manifest** before any field is read, so a variable works anywhere a string does — `user_include`, `out_dir`, a `user_lib` directory, a `compile_definitions` value — not only in the toolchain paths above. The field itself disappears once applied.

Declarations are an **array**, not an object, because they are **ordered**: a value may use the variables declared before it, and only those.

```
"variables": [
    { "name": "TC",  "value": "/opt/toolchains/clang-p2996" },
```

```
    { "name": "BIN", "value": "$TC/build/bin" }
],
"cpp26_cxx": "$BIN/clang++"
```

4.4.1 What is not substituted

A manifest already carries two other dollar notations, and both belong to **CMake**, which expands them at its own configure time. Rosetta hands them over untouched, so the rule is that **only a name you declared is substituted**:

Written	Result
`$ENV{HOME}`	passed through verbatim — CMake's environment lookup, whatever `variables` says
`${CMAKE_SOURCE_DIR}`	passed through verbatim — no `CMAKE_SOURCE_DIR` was declared
`$P2996` with `P2996` declared	substituted

Declaring `ENV` is an error, since `$ENV{...}` could never resolve to it.

> **WATCH OUT**
>
> The price of that leniency is that a **misspelt variable is silent**. `$P2996x` is one name, `P2996x`, which nothing declares — so it stays as written and surfaces later as a path that does not exist. `${P2996}x` is how a variable abuts other text, and is worth preferring for that reason.

A variable name is a C identifier: letters, digits and `_`, not starting with a digit.

4.5 Where the toolchain lives

Four optional fields point at the C++26 / P2996 reflection compiler used to build the generator. Omit them all and rosetta uses these defaults:

Field	Default
`cpp26_root`	`$ENV{HOME}/devs/c++/clang-p2996/build`
`cpp26_cxx`	`${cpp26_root}/bin/clang++`
`cpp26_cc`	`${cpp26_root}/bin/clang`
`cpp26_lib`	`${cpp26_root}/lib`

If your clang-p2996 build lives elsewhere, set `cpp26_root` alone — the other three derive from it. Override an individual one only when the compiler binaries or the `libc++` / `libc++abi` directory sit outside the usual `bin/` and `lib/` layout.

`$ENV{HOME}` is expanded by CMake at configure time, so the path stays portable across machines. Of the targets, these fields affect only `rest` and `json` — every other one builds with an off-the-shelf compiler and ignores them entirely. (The *generator* always needs them.)

The command line wins over the manifest: `rosetta_gen -build -clang-p2996-root PATH`.

`qt_dir` plays the same role for the `qt` and `qml` backends, with `-qt-dir` as its flag.

4.6 Naming things

`generator_name`
Basename of the emitted reflection driver: `"my_person_gen"` produces `gen/my_person_gen.cpp`. The built binary is always called `generator`, so this is mostly cosmetic — its one load-bearing role is being the last-resort default for `module_name`. Defaults to the manifest's directory name.

`module_name`
The default binding module name, used by any bare-string target. A preset may supply one, which is why a preset-based manifest often names neither this nor `generator_name`.

> **NOTE**
>
> The *derived* `generator_name` is folded to an identifier (`my-cool-lib` becomes `my_cool_lib`) so a directory name cannot produce an invalid module name. A value you spell out is used verbatim — if you write something that is not an identifier, you get it.

4.7 Shared defaults: `namespace` and `header_dir`

When every entry repeats the same namespace and the same folder:

```
"classes": [
  {"name": "stressinv::Serie",      "header": "stressinv/Serie.h"},
  {"name": "stressinv::Data",       "header": "stressinv/Data.h"},
  {"name": "stressinv::CostMetric", "header": "stressinv/cost.h"}
]
```

factor them out:

```
"namespace": "stressinv",
"header_dir": "stressinv",
"classes": [
    {"header": "Serie.h"},
    {"header": "Data.h"},
    {"name": "CostMetric", "header": "cost.h"}
]
```

(`Serie` and `Data` also drop their `name`, which defaults from the header stem — and the derived name is namespace-qualified too.)

The rules apply identically to `classes`, `functions` and `extensions`:

- `namespace` qualifies an entry name **only when it carries no `::` of its own**. `"Serie"` becomes `stressinv::Serie`; a name containing `::` passes verbatim. So fully qualified spellings, nested classes (`stressinv::Model::Inner`) and sub-namespaces keep working unchanged, and mixing shortened and full spellings in one manifest is fine.
- A **leading `::`** pins an entry to the global namespace: `"::c_entry"` becomes `c_entry`. This is the escape hatch for the odd global function (an `extern "C"` one, say) in an otherwise namespaced manifest.
- `header_dir` is prepended to every entry header, inserting a `/` if missing: `"Serie.h"` becomes `stressinv/Serie.h`. A header living elsewhere can step out relative to it (`"../other/x.h"`), or you can leave `header_dir` unset and spell every path in full.

4.7.1 Grouped entries

One pair of top-level defaults cannot cover a library whose headers live in several folders, or that uses sub-namespaces. For that, an element of `classes` or `functions` may be a **group**: an object carrying `entries` plus its own local defaults, instead of being an entry itself.

```
"namespace": "arch",
"header_dir": "Arch",
"classes": [
  {"name": "Vector3", "header": "math/math.h"},

  {"header_dir": "solvers", "entries": [
    {"name": "GmresSolver", "header": "Gmres.h"},
    {"header": "ParallelSolver.h"}
  ]},

  {"header_dir": "algos", "entries": [
    {"header": "DataSuperposition.h"},
    {"namespace": "sinv", "header_dir": "stress-inversion", "entries": [
      {"header": "JointData.h"},
      {"header": "types.h", "entries": [
        {"name": "MCMCConfig"},
        {"name": "MCMCResult"}
      ]}
    ]}
  ]}
]
```

- A group's `header_dir` **appends below** the inherited one: `Arch` + `solvers` gives `Arch/solvers/Gmres.h`.
- A group's `namespace` **appends to** the inherited one: `arch` + `sinv` gives `arch::sinv::JointData`. A leading `::` makes it absolute instead of appending.
- A group may set a `header`: the default header for entries that spell none. That is the natural shape for a run of classes declared by one header — `types.h` above yields `arch::sinv::MCMCConfig` and `arch::sinv::MCMCResult`, both from `Arch/algos/stress-inversion/types.h`. Such entries need an explicit `name`, since the stem fallback would give every one the same name.
- Groups **nest**, **mix freely with plain entries** in the same array, and work identically under `functions` — where a shared-header group reads especially well (a dozen shape generators all declared by `shapes.h`).
- A group cannot carry a `name`, and `//`-comment keys are ignored on groups as everywhere else.

Everything is resolved at load time, so backends and generated output are byte-identical to the fully spelled form.

4.8 Multiple include directories

When your headers do not all live under one root, give `user_include` an array:

```
"user_include": ["./geom", "../shared/include", "/opt/thirdparty/include"]
```

Each entry follows the same resolution rules as the single-string form. Every directory is added to the generated bindings' include path, so a class `header` is resolved against *all* of them —

the first directory containing the file wins, exactly like a compiler's `-I` search order. The array must not be empty; the single-string form is just the one-directory shorthand.

4.9 A field map

The rest of Part II is organised by what you are trying to do, rather than alphabetically. Appendix A is the alphabetical version.

You want to…	**See**
say which classes and functions to bind	Chapter 5
choose the languages, and name the modules	Chapter 6
get the generated projects to actually compile and link	Chapter 7
make a foreign container or an Eigen type cross the boundary	Chapter 8
reuse a fixed binding surface across projects	Chapter 9
add docstrings, ranges, UI hints	Chapter 10

Chapter 5

Declaring what to bind

5.1 Classes

`classes` is an array of per-class entries. Each binds one C++ type — a `class`, a `struct`, or an `enum`.

```
"classes": [
  { "name": "Model", "header": "Model.h", "doc": "the model class" },
  { "header": "Point.h" },
  { "name": "space::Vector3", "header": "Vector3.h",
    "annotations": "Vector3.ann.json" }
]
```

Field	Req.	Meaning
`header`	•	Filename emitted into `#include "..."`, resolved against `user_include`. May be a **glob** (Section 5.2).
`name`		C++ type name, reachable after including `header`. May be namespace-qualified. Defaults to the header's stem.
`expose`		The *binding* name — what scripts see. Defaults to the unqualified `name`. See Section 5.3.
`annotations`		Path (relative to the manifest) to an out-of-line annotation side-car. See Section 10.4.
`doc`		A description string, used by the document backends.
`extensions`		Free functions exposed as instance methods. See Section 5.5.
`final`		Treat the class as non-overridable from the host language: no trampoline is generated even when it has public virtual methods.

Inheritance, multiple constructors, enums, nested user types and `std::vector` members are discovered automatically. You do not list base classes — list the most-derived type you want bound and the bases are flattened into it.

5.1.1 `final`: turning off trampolines

By default, a class with public virtual methods gets a *trampoline* subclass in the backends that support host-language overriding (pybind11, N-API), so a Python or JavaScript subclass can override a C++ virtual. Marking the entry `"final": true` suppresses that: the virtuals still bind as ordinary callable methods, they just cannot be overridden from outside.

It has a second, less obvious use. A class with virtuals is wrapped *as its trampoline subclass*, and a Node member-object property (`mesh.vertices`) stores a `T*` — which requires the wrapped type to actually be `T`, not `Js_T`. `final` is what makes such a class eligible.

5.1.2 Members that will not bind

Some members cannot be compiled into a binding at all: a public field whose type is a non-copyable class, a method returning a reference to such a type, a by-value parameter of one. These are **skipped**, not fatal. The class still binds — as an opaque handle plus whatever members do marshal — and every skip is recorded in `coverage.json` with a reason.

Extension methods (Section 5.5) are how you fill the gaps.

5.2 Binding a whole folder

For the common library shape — one class per header, named after its file — an entry whose `header` is a **shell glob** stands for every header it matches: one class entry per file, its `name` derived from the file's stem.

```
"classes": [
  { "header": "shapes/*.h" },
  { "header": "solvers/**/*.h",
    "exclude": ["solvers/detail/*.h", "solvers/*_impl.h"] }
]
```

The pattern syntax is the same one `user_sources` uses: `*`, `?` and `[...]` within a path component, and a component of exactly `**` matching zero or more directories.

Field	Req.	Meaning
`header`	•	The glob, resolved against **every** `user_include` root with the composed `header_dir` in front.
`exclude`		A pattern, or a list of them, whose matches are dropped: headers declaring no bound class, a `detail/` subtree, an `_impl` convention.
`final`		Applies to every matched class — it is a uniform property.

`name`, `expose`, `annotations` and `extensions` are **rejected** on a glob entry: they are per-class and cannot mean anything for a folder.

The rules:

- Only `.h`, `.hh`, `.hpp` and `.hxx` are taken, so `"**/*"` cannot drag in a `.cpp` or a `README`.
- The derived name is the file's **stem**, qualified by the `namespace` in scope exactly like a spelled-out entry with no `name`. A glob works inside a group and takes that group's `header_dir` and `namespace`.
- A header whose stem is not a C++ identifier (`my-utils.h`) is skipped with a warning — there is nothing to derive a name from.
- A pattern matching nothing is a warning, not an error.

5.2.1 The stem is the class name

The loader never opens a bound header — it lists files. So a glob's only claim about a header is the one its *name* makes, and that claim has to hold exactly:

`Point.h` must declare a class named `Point`, spelled identically, in the namespace in scope for the entry.

Nothing checks this at load time. A stem naming no such class surfaces as a *compile error in the generated driver* — `no member named 'vec3' in namespace 'geom'` — which is why the cases below are worth knowing before pointing a glob at a folder.

Your header tree	What the glob does	What to write
`Point.h` declares `Point`	binds `Point`	`{"header": "*.h"}` — nothing else
`Point.h` declares `Point` *and* `Aabb`	binds `Point` only — a file yields one entry	list both explicitly (see the take-over rule)
`types.h` declares `Tolerance`, `Mode` — no `types`	would bind a non-existent `types`	list them; the take-over rule keeps the glob off the file
`vec3.h` declares `Vec3`	would bind a non-existent `vec3`	list it, or `"exclude": ["vec3.h"]`
`Point.h` declares `geom::detail::Point`	binds `geom::Point` — the glob applies the namespace *in scope*	list it, or put the glob in a group with `"namespace": "detail"`

The take-over rule

An entry that names a class of a header **speaks for that whole header**: the glob then leaves the file alone. That is what makes the `types.h` and `vec3.h` rows work — you name the classes, and no bogus stem entry appears beside them.

Globbed entries land *after* everything spelled out and yield to it, both by name and by header. So this composes as you would want:

```
"header_dir": "geom",
"namespace": "geom",
"classes": [
  { "header": "*.h", "exclude": ["fwd.h"] },
  { "header": "Mesh.h", "annotations": "Mesh.ann.json" },
  { "header": "types.h", "name": "Tolerance" }
]
```

Every `geom/*.h` but `geom/fwd.h` binds, with `geom/Mesh.h` taking its side-car and `geom/types.h` binding `geom::Tolerance` rather than a non-existent `geom::types`.

The cost is the second row of the table: if a header declares *both* a class named after the file and another one you list, taking the file over drops the first. `rosetta_gen` says so rather than leaving it to be noticed:

```
rosetta_gen: note: "Point.h" is listed explicitly (class "Aabb"), so the pattern
that also matched it contributes nothing there -- class "Point" is NOT bound.
List it too if the header declares both; name the header in the pattern's
"exclude" if it does not (silences this)
```

The note fires on the `types.h` and `vec3.h` rows too, where losing the stem class is exactly what you wanted — add those headers to `exclude` to say so once and quiet it. A name already taken is *not* reported: a side-car entry beside a glob is the intended composition, not a loss.

NOTE

When stems and class names disagree often, `rosetta_gen -init <src_dir>` *reads* the headers (a heuristic token scan) instead of trusting their names, and writes what it found into a manifest — once, into a file you then edit.

5.3 Renaming: `expose`

Every class binds under **one module-level name**: its `expose` override, or its unqualified C++ identifier. Two entries resolving to the same name is an error — the module attribute and the generated trampoline (`Py_<name>` / `Js_<name>`) would collide.

That happens as soon as two bound namespaces declare the same identifier — a slip-inversion `arch::Data` next to a stress-inversion `arch::sinv::Data`. Rename one side:

```
"classes": [
  {"//": "stress inversion", "namespace": "sinv",
   "header_dir": "stress-inversion", "entries": [
      {"header": "Data.h"},
      {"header": "GpsData.h"}
  ]},
  {"//": "slip inversion", "header_dir": "slip-inversion", "entries": [
      {"header": "Data.h",    "expose": "SlipData"},
      {"header": "GpsData.h", "expose": "SlipGpsData"}
  ]}
]
```

Python then sees `arch.Data` (the `sinv` one) and `arch.SlipData`; C++ is untouched. Cross-references follow the rename automatically — a TypeScript signature touching `arch::Data` says `SlipData` — and the emitted C++ spells every bound class by its *qualified* name, so the shared identifier never becomes ambiguous in the generated translation unit.

`expose` is honoured by every backend: the runtime ones, the C-ABI ones (where it also names the generated `.cs` / `.java` file), the UI inspectors, `rest` and `openapi` (route paths and schema names), and the text outputs. It works on an **enum** entry, on a **free function**, and on an **extension method**. It must be a plain identifier.

5.4 Free functions

`functions` binds free (non-member) functions without editing your headers. A namespace has no member list to enumerate, so unlike class members these must be named one by one.

```
"functions": [
  { "name": "transform", "header": "common.h",
    "doc": "Scale and swizzle a point into (x*2, z*3, y*4)" }
]
```

Field	Req.	Meaning
`name`	•	Function name, optionally namespace-qualified.
`header`	•	Header declaring it.
`doc`		Description — free functions carry no in-source annotations.
`expose`		The binding name. Same rule as a class's.
`signature`		The C++ function type of the one overload to bind.

Free functions share the module namespace with classes, so `arch::solve` and `arch::sinv::solve` — or a function and a class of the same name — collide exactly like two classes do, and are fixed the same way:

```
"functions": [
  { "name": "arch::solve",       "header": "slip.h" },
  { "name": "arch::sinv::solve", "header": "stress.h", "expose": "solve_stress" }
]
```

5.4.1 Binding one overload: `signature`

An overloaded free function has no reflection to name: `^^GEO::mesh_union` is ill-formed the moment the namespace declares the name twice, so the entry could not be written at all. Give it the *signature* of the one you want:

```
"functions": [
  { "name": "GEO::mesh_union",
    "header": "geogram/mesh/mesh_surface_intersection.h",
    "signature": "void(GEO::Mesh&, const GEO::Mesh&, const GEO::Mesh&, bool)" }
]
```

The signature is a C++ function type — return type first, then the parameter list — spliced into the generated driver **verbatim**. So it must resolve there: spell the types the way the header does, qualified. `rosetta_gen` checks only its shape (a return type, balanced parentheses); the types are the compiler's business, and an error names your own spelling back to you.

It is **not a filter** over the overload set. It names one member, and only that one binds. List the entry twice, with two signatures and two `expose` names, to bind two of them:

```
{ "name": "GEO::mesh_union", "header": "...", "expose": "mesh_union_flags",
  "signature": "void(GEO::Mesh&, const GEO::Mesh&, const GEO::Mesh&,
                     GEO::MeshBooleanOperationFlags)" }
```

Backends form the function pointer through a disambiguating `static_cast`, so every backend that emits a pointer binds the selected overload. The one that spells the function's *reflection* instead — `rest`, which emits `bind_rest_function<^^name>` — skips the entry with a note on stderr: there is no reflection for one member of an overload set, and a skipped function is honest where an ambiguous one would not compile.

> **NOTE**
>
> Overload selection is a `functions` feature. An extension method cannot take a `signature` — an extension reaches the backends as a *method*, whose emitters spell its address in many more places than a free function's single address-of.

5.5 Extension methods

Some libraries keep their real API where no binding generator can reach it: `GEO::Mesh`'s geometry lives behind public member objects with raw `double*` accessors, its I/O helpers are overloaded, its UV coordinates sit in an attribute template.

Rather than hand-write a wrapper *class*, list plain free functions — whose **first parameter is `Cls&`** (or `const Cls&`) — as `extensions` of the bound class. They appear to every backend as ordinary instance methods:

```
"classes": [{
  "name": "GEO::Mesh", "header": "geogram/mesh/mesh.h",
  "extensions": [
    { "name": "georo::set_surface", "header": "mesh_ext.h",
      "doc": "Set vertices + triangles from flat arrays." },
    { "name": "georo::vertices",    "header": "mesh_ext.h",
      "doc": "Vertex coordinates as a flat array." }
  ]
}]
```

```
m = geogram.Mesh()
m.set_surface(coords, triangles)   # calls georo::set_surface(m, ...)
print(len(m.vertices()) // 3)
```

The receiver is dropped from the exposed signature; the remaining parameters and the return type marshal exactly like a free function's. The method name is the function's unqualified identifier, unless `expose` says otherwise.

Supported by `python`, `nanobind`, `node` and `wasm`, plus all the text backends. Backends that can only emit member pointers — `qt`, `qml`, `csharp`, `java` — skip them, and say so in `coverage.json`.

This is also the escape hatch for an overload a name-keyed target dropped: wrap it in an extension under a distinct name and it comes back.

5.6 Out-parameters

```
bool get_doubles(const std::string &name, vector<double> &out, index_t &dim) const;
```

Three results, C++-style. No host language has that shape, and every backend skipped such a member — the argument its runtime converts is a temporary, which cannot bind to a non-const reference. Name the outputs and they leave the exposed signature and join the return instead:

```
"out_params": {
  "AttributesManager::get_doubles": [1, 2]
}
```

Keys are `"Class::method"` or `"ns::function"`; values are **0-based parameter indices**. A key matching nothing is reported on stderr rather than ignored.

```
ok, uv, dim = mesh.facet_corners.attributes().get_doubles("tex_coord")
```

Backend	Shape
`python` / `nanobind`	a tuple — `(ok, uv, dim)`
`lua`	Lua's own multiple returns
`node`	an array — `const [ok, uv, dim] = ...`
`wasm`	an array too, built as an `emscripten::val`
`typescript`	a tuple type: `[boolean, number[], number]`

The return value comes first when the function has one; with a `void` return the outputs are all there is.

Never inferred, and that is the design

These two are the same C++:

```
void assign_points(vector<double> &pts, index_t dim, bool steal); // an INPUT
bool get_doubles(const std::string &, vector<double> &out, index_t &dim); // an OUTPUT
```

Guessing wrong silently drops an argument in one direction or a result in the other, so rosetta does not guess. An unmarked mutable reference keeps binding exactly as before — input-only for a container, skipped for a scalar.

Marking a `const` reference does nothing: the callee cannot write through it. A mutable reference to a **bound class** is not an out-parameter either — it already crosses as a handle the callee writes through, which is the more faithful reading and needs no adapter.

5.7 Module init

A library is more than its API. geogram wants `GEO::initialize()`, a run of `CmdLine::import_arg_group()` calls and a C-function-pointer registration to have happened before anything works — none of which is expressible as "bind this name". `module_init` names statements the generated module runs **when it loads**:

```
"module_init": {
  "headers": ["geogram/basic/command_line.h",
              "geogram/basic/command_line_args.h"],
  "statements": [
    "GEO::initialize(GEO::GEOGRAM_INSTALL_NONE)",
    "GEO::CmdLine::import_arg_group(\"standard\")",
    "GEO::CmdLine::import_arg_group(\"algo\")"
  ]
}
```

A bare array is shorthand for `statements` alone. The statements are spliced **verbatim** into the module entry point — `PYBIND11_MODULE`, `NB_MODULE`, `Init`, `EMSCRIPTEN_BINDINGS`, `luaopen_*`, `JLCXX_MODULE` — at the very top, before any binding is registered. So an I/O handler an init call installs is in place before a bound loader can be reached. A trailing `;` is optional and never doubled.

`headers` are emitted as `#include` lines *ahead* of the bound classes' own, since an init statement usually names a namespace the bound headers never mention.

WATCH OUT

Because the statements are spliced verbatim, they must compile at that point: spell types qualified, and remember the module has no arguments to offer. This is *fixed* start-up work, not a configurable entry point. Anything a caller should be able to vary — a verbosity flag, a thread count — still belongs in a bound function.

Emitted by `python`, `nanobind`, `node`, `wasm`, `lua` and `julia` — the backends with a module entry point. The C#, Java, REST, UI and document backends have no single load-time hook and ignore the field.

Chapter 6

Targets

`targets` lists the language backends to emit. One generator produces a **single combined module per target**, exposing every class — not one module per class.

6.1 The two entry forms

A **bare string** uses `module_name` (falling back to `generator_name`) as the module name:

```
"targets": ["python", "node", "markdown"]
```

An **object** sets a per-target module name, and carries the per-target options:

```
"targets": [
  { "lang": "python", "name": "pygeom" },
  { "lang": "node",   "name": "jsgeom" },
  { "lang": "markdown" }
]
```

`name` is optional in the object form too. Mixing the forms in one array is fine.

6.2 The available `lang` values

Every compiled target is **expanded**: reflection runs once on the generation host and the emitted code builds with an off-the-shelf compiler. `rest` and `json` are the two exceptions — both emit a project whose sources include a runtime visitor that still splices reflections, so those two need the C++26 toolchain to build.

`lang`	Output	Builds with
`python`	pybind11 extension module	off-the-shelf compiler
`nanobind`	nanobind extension module	off-the-shelf compiler
`node`	N-API native addon	off-the-shelf compiler
`wasm`	Emscripten / embind module	off-the-shelf emsdk
`qt`	Qt Widgets property/method inspector	off-the-shelf compiler + Qt 6
`qml`	QML / QtQuick inspector	off-the-shelf compiler + Qt 6
`imgui`	Dear ImGui inspector app (GLFW + OpenGL3, auto-fetched)	off-the-shelf compiler
`csharp`	C-ABI shared lib + P/Invoke wrappers	off-the-shelf compiler + .NET

`lang`	**Output**	**Builds with**
`java`	C-ABI + handle-backed FFM wrappers	off-the-shelf compiler + JDK
`julia`	CxxWrap.jl / jlcxx shared module	off-the-shelf compiler + CxxWrap.jl
`lua`	sol2 shared module, `require`-able	off-the-shelf compiler
`rest`	cpp-httplib JSON server + browser client	**C++26 toolchain**
`openapi`	OpenAPI 3.1 specification	text output
`json`	nlohmann (de)serialisation + `demo.cpp`	**C++26 toolchain**
`typescript`	ambient `.d.ts` declarations	text output
`markdown`	API reference document	text output
`html`	styled API reference page	text output
`paraview`	ParaView Server Manager plugin XML	text output
`dynamic`	the IR as runtime data (Chapter 14)	off-the-shelf compiler
`mcp`	MCP server for AI agents, over the same runtime data (Chapter 16)	off-the-shelf compiler

Notes worth having:

- **Qt / QML** need Qt 6, but no `moc` on the generated code.
- **MCP** needs nlohmann/json (fetched if missing). Lua 5.3+, when found, adds the `run_script` tool; `-DROSETTA_MCP_PYTHON=ON` adds `run_python` (not sandboxed). See Chapter 16.
- **Lua** needs Lua 5.1–5.4 or LuaJIT — sol2 does not support Lua 5.5 yet. sol2 itself is fetched at configure time.
- The text-only outputs (`markdown`, `html`, `typescript`, `openapi`, `paraview`) compile nothing, so the C++26-vs-off-the-shelf distinction does not apply to them. `json` is *not* one of them: besides the serialisation header it emits a buildable `demo.cpp` at `CMAKE_CXX_STANDARD 26`.
- Each name also accepts a deprecated `-expanded` spelling (`lua-expanded`, `qt-expanded`, ...) resolving to the same backend, so manifests written before the suffix was dropped keep working.

> **NOTE**
>
> Until 2026-08, seven languages shipped a second, *thin* backend whose generated code re-ran the reflection walk at the target's compile time. Those are gone: the short name now means what `-expanded` used to. No target spells `-expanded` any more, and every old spelling still resolves.

6.3 Per-target linker flags

`link_options` applies extra linker flags to *this target only*, emitted as `target_link_options(... PRIVATE ...)` in the generated project.

```
"targets": [
  { "lang": "python" },
  { "lang": "wasm", "link_options": ["-lnodefs.js"] }
```

```
]
```

It is per-target — unlike `compile_definitions`, which is global — because link flags are toolchain-specific. geogram's `GEO::initialize()` mounts the host filesystem with NODEFS under Node, which needs emscripten's nodefs library on the **wasm** link line and would break a native link.

The flags are emitted *after* the template's own `target_link_options`, so on the wasm targets — where emcc honours the *last* occurrence of a repeated `-s` setting — they can also override a template default, e.g. `"-sALLOW_MEMORY_GROWTH=0"` for a fixed-size heap.

6.4 Where the built artifact lands

A generated project drops its built module next to its own sources. Handy for a smoke test; useless when the `.so` belongs inside your Python package or the `.js` next to a web app's assets.

```
"targets": [
  { "lang": "nanobind", "name": "implicit3d", "out_dir": "./dist" },
  { "lang": "wasm",     "name": "implicit3d", "out_dir": "../www/assets" }
],
"out_dir": "./dist"
```

The top-level `out_dir` is the default for every target that names none; a target's own entry wins. Paths resolve against the manifest's directory and are created if missing.

> **WATCH OUT**
>
> This is **not** where the generated project goes — that is `rosetta_gen`'s own output tree, controlled by `-bindings-dir`. It is where the *loadable artifact* lands: `libfoo.so` / `foo.pyd`, `foo.node`, `foo.so` for Lua, and for wasm *both* halves, the `.js` loader and its `.wasm`.

The copy is `copy_if_different`, so an unchanged build touches nothing downstream, and the existing next-to-the-sources copy still happens.

Supported by every backend that builds a loadable module: `python`, `nanobind`, `node`, `wasm`, `lua`, `julia`. The document backends have no artifact to place, and the application-shaped ones (`qt`, `qml`, `imgui`, `rest`, `csharp`, `java`) are not covered.

6.5 Pinning the runtime

By default a generated project takes whatever the `PATH` gives it: the emitted CMake probes `python3` / `node`, with a 3.8 floor and N-API 8 written in. Pin them per target instead:

```
"targets": [
  { "lang": "nanobind", "name": "geom",
    "python": "3.11", "requires_python": ">=3.10" },
  { "lang": "node", "name": "geom",
    "napi_version": 9, "node_engine": ">=18" }
]
```

Field	Effect	Default
`python`	The interpreter the binding is built for. A bare version (`"3.11"`) becomes `python3.11`, resolved on `PATH`; anything else is used as written (`"/opt/py311/bin/python3"`).	probe `python3`
`requires_python`	Minimum version. Feeds **both** `find_package(Python ...)` and `pyproject.toml`'s `requires-python`, so the build floor and the wheel metadata cannot drift apart.	`>=3.8`
`napi_version`	`NAPI_VERSION=` for the addon. This is the real Node floor — N-API 8 means Node 12.22+, N-API 9 means Node 18.17+.	`8`
`node_engine`	`engines.node` in `package.json` — documentation for npm rather than a compile setting, which is why it is separate from `napi_version`.	absent

These are per-target, not top-level, so a manifest carrying both a `python` and a `nanobind` target can point them at different interpreters.

Two things worth knowing:

- `-cmake-arg -DPython_EXECUTABLE=...` **does not work**, which is why these fields exist. The generated CMake sets that variable with `CACHE ... FORCE` after probing, overriding anything passed on the command line. Before these fields, a venv or a `pyenv` shim was the only way to choose an interpreter.
- `python` **also drives the wheel.** A wheel is tagged for whichever interpreter runs `make_wheel.py`, so `rosetta_gen -build -wheel` packages with the pinned interpreter rather than the probed one — otherwise a 3.11 build would emit a `cp312` wheel.

6.6 Choosing targets

A few rules of thumb, since nineteen is a lot:

Python
`python` (pybind11) is the safe default — the most permissive type gate, full overload sets, mature. `nanobind` is leaner and faster and produces `abi3` wheels on 3.12+, at the cost of a slightly narrower feature set. Emitting both costs nothing but build time.

JavaScript
`node` for a native addon, `wasm` for the browser. Add `typescript` in either case: it costs nothing to generate and describes the Node addon's surface to an editor.

Documentation
`markdown` or `html` are free — no compilation, no toolchain, and they stay in step with the code by construction. There is little reason not to emit one.

A user interface
`qt`, `qml` and `imgui` each generate a working inspector over your annotated types. If you want a UI that is a *query* rather than generated code, see Chapter 14 instead.

During development
Generate everything, but compile one or two: `-only python,node` (Section 12.4.2).

Chapter 7

Making the generated projects build

Chapter 5 said what to bind. This chapter is about the other half: getting the emitted projects to actually compile and link, which for a real library means telling rosetta where the *definitions* are, what preprocessor state the headers expect, and which optimisation to build at.

7.1 The two ways to supply definitions

Your bound headers usually only *declare* the API. The bodies live either in a pre-built library or in source files. Rosetta has one field for each, and they are independent — use either, or both.

`user_lib`	Link the bindings against a pre-built `.so` / `.dylib` / `.a`.
`user_sources`	Compile `.cpp` (or `.c`) files *straight into* each binding target.

7.1.1 Linking a pre-built library

```
"user_lib": {
  "name": "space",
  "dir":  "../space/bin",
  "link": "shared"
}
```

Field	Req.	Meaning
`name`	•	Library name — the `space` in `libspace.dylib`.
`dir`	•	Directory holding it, relative to the manifest; used for `-L` and rpath.
`link`		`"shared"` (default), `"static"`, or `"dynamic"` (an alias of shared). The *preferred* form, with fallback to whichever is actually built.

A bound library rarely stands alone. Give `user_lib` an **array**, one object per library:

```
"user_include": ["../mylib/include", "/opt/foo/include"],
"user_lib": [
  { "name": "mylib", "dir": "../mylib/bin", "link": "shared" },
  { "name": "foo",   "dir": "/opt/foo/lib" },
  { "name": "bar",   "dir": "/opt/foo/lib", "link": "static" }
]
```

- **Order is the link order** — list a library before the ones it depends on, which is what static archives require.
- Each entry keeps its own `link`, so a shared library and a static dependency mix freely.
- Every distinct `dir` lands in the binding's `BUILD_RPATH` and `INSTALL_RPATH`, so the module loads without `LD_LIBRARY_PATH` / `DYLD_LIBRARY_PATH`.
- The **generator driver** links the same list — it instantiates the bound types during the reflection walk, so it needs the dependencies' definitions too, not just yours.
- Headers of the dependencies go in `user_include`; their configuration macros in `compile_definitions`.

WATCH OUT

On `wasm`, every entry is taken as the static archive `lib<name>.a` built with the **same** emsdk. A native `.dylib` / `.so` cannot enter a wasm module, so a dependency you cannot rebuild for emscripten rules that target out.

Flags that are not a `-L`/`-l` pair — a framework, a `-start-group`, a package's own link line — go through a target's `link_options` (Chapter 6); the two compose.

7.1.2 Compiling sources in

```
"user_sources": [
  "../src/widget.cpp",
  "../src/shape.cpp"
]
```

Always a list of paths, each relative to the manifest (or absolute); a single string is accepted as a one-element list. Every compiled backend adds them via `target_sources(...)`, so they build with the same include path and flags as the generated binding.

Entries may be **shell globs**, expanded at generation time:

```
"user_sources": ["extern/arch/src/**/*.cxx"]
```

`*`, `?` and `[...]` work within a path component, and a component that is exactly `**` matches **zero or more directories** — so one pattern covers a whole source tree. Like the other wildcards, `**` does not enter hidden directories. Matches are sorted for reproducible output and the final list is de-duplicated, so mixing a literal path with a glob that also covers it is safe. A pattern matching nothing warns and is skipped.

WATCH OUT

Globs expand when `rosetta_gen` runs. **Re-run it after adding or removing matching files** — a new `.cpp` does not appear in the build by itself.

Entries may also be **C sources**. When any `.c` file is listed, the generated `CMakeLists.txt` calls `enable_language(C)` automatically, which is what makes a library's vendored dependencies work:

```
"user_sources": [
  "./geogram/src/lib/geogram/mesh/*.cpp",
  "./geogram/src/lib/geogram/third_party/rply/*.c",
```

```
  "./geogram/src/lib/geogram/third_party/zlib/*.c"
]
```

The text-only backends compile nothing and ignore the field.

7.2 Preprocessor definitions

Most often a third-party library's configuration macros. Each entry is `"NAME"` or `"NAME=VALUE"`:

```
"compile_definitions": [
  "GEOGRAM_USE_BUILTIN_DEPS",
  "GEOGRAM_WITH_HLBFGS"
]
```

They are emitted as `target_compile_definitions(... PRIVATE ...)` in **two** places, so the whole pipeline sees a consistent configuration:

- the **generator driver** — it includes the bound headers for the reflection walk, which must see the same preprocessor state the bindings will be built with;
- **every compiled binding target**, where they reach the bound headers and the `user_sources` alike.

That first bullet is the one that matters. A macro that changes a class's member set — and plenty do — would otherwise produce a binding for a different class than the one you compile.

A single string is accepted as a one-element list.

7.3 Build type and optimisation

```
"build_type": "Release",
"optimization": "-O2"
```

Both optional and independent.

`build_type` is one of `Debug`, `Release`, `RelWithDebInfo`, `MinSizeRel` (case-insensitive). It is emitted as a *default* inside `if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)`, so `-DCMAKE_BUILD_TYPE=...` at configure time still wins and multi-config generators (Visual Studio, Xcode) are left to pick a configuration at build time.

> **NOTE**
>
> **Omitted means `Release`.** It used to mean "emit nothing", which left CMake with no build type and hence no optimisation flags at all — so the documented two-line build produced an unoptimised module, around eight times slower on a trivial pybind11 call. Pass `-DCMAKE_BUILD_TYPE=Debug`, or set `"build_type": "Debug"`, when you want an unoptimised build.

`optimization` is an explicit level: `-O0` through `-O3`, `-Os`, `-Oz`, `-Og` or `-Ofast` (the leading `-` may be omitted). It is emitted as `add_compile_options(...)` and `add_link_options(...)`, which land *after* the build type's own per-config flags — so this `-O` overrides the level the build type implies. `"build_type": "Release", "optimization": "-O2"` builds `-DNDEBUG` but at `-O2` instead of Release's `-O3`. The link option matters for the wasm targets, where emscripten optimises at link time too.

Both apply to the bindings and to any `user_sources` compiled into them, in every compiled backend.

7.4 Headers the library's own build produces

A header configured by the bound library's build system is simply absent when rosetta compiles that library's sources without running its build. geogram's `<geogram/version.h>` is the case in point — configured by geogram's CMake from `version.h.in`.

```
"generated_headers": [
  {
    "path": "geogram/version.h",
    "template": "./extern/geogram/src/lib/geogram/basic/version.h.in",
    "substitutions": {
      "VORPALINE_VERSION_MAJOR": "1",
      "VORPALINE_VERSION_MINOR": "10",
      "VORPALINE_VERSION_PATCH": "1",
      "VORPALINE_VERSION": "1.10.1-rosetta",
      "VORPALINE_BUILD_NUMBER": "",
      "VORPALINE_BUILD_DATE": "",
      "VORPALINE_SVN_REVISION": ""
    }
  }
]
```

Field	Req.	Meaning
`path`	•	The *relative include path* — exactly what the sources `#include`.
`template`	one of	A file with `@KEY@` placeholders (CMake's `configure_file` form), resolved against the manifest's directory.
`content`	one of	Literal lines, as an array of strings, for a header with no template.
`substitutions`		`@KEY@` to value, applied to `template`.

The file is written to `<bindings>/include/<path>` and that directory is placed **first** on every generated target's include path — ahead of the library's own sources, so a stale checked-in copy of the same header cannot win the lookup.

Substitution happens when the manifest is read, so a missing key is reported against your manifest (`no substitution for @VORPALINE_VERSION@`) rather than compiling a literal `@KEY@` into the binding. Only the `@KEY@` form is recognised, since `${KEY}` would collide with the shell and CMake expansions such templates carry.

> **NOTE**
>
> A `-D` define is a different tool for a nearby job. geogram's `GEOGRAM_VERSION` is passed by its CMake on the command line, not written into the header, so it belongs in `compile_definitions`.

7.5 Python wheels

The `python` and `nanobind` backends emit, alongside `CMakeLists.txt`, everything needed to build a redistributable wheel:

`pyproject.toml`	scikit-build-core build config. It drives the generated `CMakeLists.txt` itself, so the wheel and a plain `cmake -build` compile the same target the same way — there is no second build description to keep in sync.
`make_wheel.py`	One-command builder: builds the wheel, then bundles external shared libraries and fixes the platform tag. Python rather than a shell script, so Linux, macOS and Windows are covered by one file.

```
cd bindings/nanobind
python make_wheel.py                    # -> dist/*.whl, dist/repaired/*.whl
python3.11 make_wheel.py                # a specific interpreter
python make_wheel.py --outdir /tmp/whl  # somewhere other than ./dist
python make_wheel.py --no-repair        # skip the bundling / retagging step
```

The wheel is built for whichever interpreter runs the script — there is no `-python` flag, just invoke the one you want (or pin it with the target's `python` field, Section 6.5).

`version` is the distribution version stamped into the packaging artifacts, PEP 440 style (`"1.2.0"`, `"0.3.0rc1"`). Default `0.1.0`.

Two fields make packaging automatic for a project that always ships wheels. They go **on the target**, beside the interpreter it is built for:

```
"targets": [
  { "lang": "python",   "name": "pygeom", "python": "3.11" },
  { "lang": "nanobind", "name": "nbgeom", "wheel": true, "wheel_dir": "./dist" }
]
```

`wheel`	Build a wheel for *this* target on every `-build`, without passing `-wheel`.
`wheel_dir`	Where this target's wheels go instead of its own `dist/`. Implies `wheel`, and resolves against the manifest's directory.

Only the two backends that emit a `make_wheel.py` accept them: `wheel` on a `markdown` or `node` target is an error, not a no-op.

Per-target rather than top-level, and for the same reason `out_dir` is (Section 6.4): the two Python backends genuinely differ here. `nanobind` produces one `abi3` wheel covering every CPython 3.12+ where `python` produces one per version, and `python` (Section 6.5) already pins per target which interpreter each is tagged for. Sharing one `wheel_dir` between them is safe — each `make_wheel.py` repairs only the wheels it just produced.

The **flags still win, in the direction of doing more**. `-wheel` packages every `python` / `nanobind` target whatever the manifest says, so a target spelling `"wheel": false` cannot turn off a run that asked for it; `-wheel-dir DIR` overrides every target's own directory, which is what makes "collect them all in one place" a single flag. With neither set anywhere, `-build` only compiles the extension module.

WATCH OUT

`wheel` and `wheel_dir` were **top-level** keys. They are per-target now, and a top-level one is a load error naming the fix rather than a key that silently does nothing — a manifest that used to ship wheels must not quietly stop.

NOTE

On 3.12+, nanobind wheels are tagged `abi3`, covering every later CPython with a single artifact.

7.5.1 Bundling files in the wheel

A binding rarely ships alone. There is usually a pure-Python layer that belongs with it — helpers more naturally written in Python than bound from C++, a bridge process, a data file the module reads — and without a way to put it *in* the wheel it becomes a second package users have to know about. Three more per-target fields, same backends and same rules as `wheel`, say what the wheel holds beyond the module:

```
"targets": [
  { "lang": "nanobind", "name": "exocore", "wheel": true,
    "wheel_files": [
      "../python/exocore_py",
      {"path": "../js/native.js", "dest": "exocore_py/web"}
    ],
    "wheel_dependencies": ["numpy>=1.20"],
    "wheel_scripts": {"exocore-web-bridge": "exocore_py.web_bridge:main"} }
]
```

`wheel_files`	Files and directories copied into the wheel next to the module. A bare string is a source with the default destination `"."` (the wheel root); the object form adds `dest`, a directory *inside* the wheel. Sources resolve against the manifest and must exist at load time. A directory is copied as a tree (minus `__pycache__`, `*.pyc`, `*.egg-info`), so a package directory at the root is importable from the same wheel as the module.
`wheel_dependencies`	PEP 508 requirement strings written to `pyproject.toml`'s `[project] dependencies`: the bundled code's own imports, declared where pip reads them.
`wheel_scripts`	Console entry points, command name → `"module:function"`, written to `[project.scripts]`.

The mechanism is the one the module itself already uses: scikit-build-core packages **what CMake installs**, so each `wheel_files` entry becomes an `install(DIRECTORY ...)` / `install(FILES ...)` rule in the generated `CMakeLists.txt`, guarded by `if(SKBUILD)`. A plain `cmake -build` copies nothing, and nothing is snapshotted at generation time either — the wheel takes the files as they are when `make_wheel.py` runs, so editing the Python layer needs no regeneration.

A `dest` must stay inside the wheel — an absolute path or a `..` component is an error — and a listed source that does not exist fails the load rather than producing a wheel that is quietly missing a package. Like `wheel`, the three fields are per-target and `python` / `nanobind` only.

7.6 A realistic manifest

Putting the last four chapters together — this is roughly the shape a real library's manifest takes:

```
{
  "//": "Bindings for the geom library.",
```

```
  "variables": [
    { "name": "SRC", "value": "./extern/geom" }
  ],

  "user_include": ["$SRC/include", "./extern/eigen-3.4.0"],
  "rosetta_include": "./extern/rosetta/include",
  "generator_name": "geom_gen",
  "module_name": "geom",

  "user_sources": ["$SRC/src/**/*.cpp"],
  "compile_definitions": ["GEOM_NO_ASSERT"],
  "build_type": "Release",
  "version": "1.2.0",

  "interop": ["eigen"],
  "sequences": [{ "type": "Eigen::VectorXd" }],

  "namespace": "geom",
  "header_dir": "geom",

  "targets": [
    { "lang": "python",   "name": "pygeom", "requires_python": ">=3.10" },
    { "lang": "nanobind", "name": "nbgeom", "out_dir": "./dist",
      "wheel": true, "wheel_dir": "./dist/wheels" },
    { "lang": "node",     "name": "jsgeom" },
    "typescript",
    "markdown"
  ],

  "classes": [
    { "header": "*.h", "exclude": ["fwd.h", "detail/*.h"] },
    { "header": "Mesh.h", "annotations": "Mesh.ann.json" }
  ],

  "functions": [
    { "name": "remesh", "header": "algorithms.h", "doc": "Isotropic remeshing." }
  ]
}
```

Chapter 8

Types across the boundary

Reflection tells rosetta that a method takes a `GEO::vector<double>`. It does not tell it what a Python caller should hand over. That mapping — the *type gate* — is per backend, and this chapter is about widening it for the types your library actually uses.

The default gate handles the obvious cases without a word in the manifest: arithmetic types, `bool`, `std::string`, enums, `std::vector<T>` of any of those, and any *bound class*. Everything below is for the rest.

> **NOTE**
>
> The IR's rule for anything it cannot claim is uniform: mark the type `kind: "unknown"` and let each backend skip the member. That is why a type rosetta does not understand costs you one skipped method with a stated reason, not a compile error.

8.1 Foreign sequence containers

Many libraries carry their bulk data in their *own* vector type — geogram's `GEO::vector<T>`, an aligned or pooled vector — and the marshalling layers only know `std::vector`. List the container template (qualified, **one type parameter**) and it crosses like a `std::vector` of its element:

```
"sequences": ["GEO::vector"]
```

`rosetta_gen` emits

```
template <typename T>
struct rosetta::is_sequence<GEO::vector<T>> : std::true_type {};
```

into the generated `bindings.h`. The container must be default-constructible with `value_type`, `size()`, `resize(n)` and `begin()`/`end()`; elements must be arithmetic, `bool`, `std::string` or a bound enum.

The opted-in backends — `python`, `nanobind`, `node`, `wasm`, `lua`, plus `typescript` declarations — marshal it **by copy through a `std::vector<element>` boundary** inside an emitted adapter. Scripts pass and receive plain arrays, lists or tables. Every other backend keeps skipping the type.

Three consequences worth knowing:

- **Mutable `Seq&` parameters bind, input-only.** The adapter's local container is a real lvalue, so geogram's `assign_points(vector<double>&, dim, steal)` works — `steal` steals from the adapter's copy, which is fine. In-place mutations are discarded, exactly like pybind's `std::vector&` casters.
- **Overload sets need the sequence overload to be first.** The adapter calls the method *by name* with concrete arguments instead of spelling the ambiguous `&T::name` member pointer. A backend that keys methods by name binds the first-declared overload, so a set whose first declaration is the raw-pointer one stays skipped there.
- **Virtual methods naming the container cannot be overridden script-side** — the exact spelling cannot round-trip — but they still bind as callable methods.

Sequence-typed public *fields* bind as copying properties in `python`, `nanobind`, `wasm` and `lua`; `node` skips them.

8.1.1 Registering a concrete type

The string form registers a *template*, and the adapter spells the container it builds as `Namespace::Template<element>`. That composition is wrong the moment the specialisation carries more than its element: `Eigen::VectorXd` is really `Eigen::Matrix<double, -1, 1>`, and the composed `Eigen::Matrix<double>` does not compile.

Register the concrete type instead, spelled exactly as the adapter should write it:

```
"sequences": [{ "type": "Eigen::VectorXd" }]
```

which emits a full specialisation — plus the spelling, as `rosetta::is_sequence` and `rosetta::sequence_cpp_name` — rather than the partial one.

`{ "template": "GEO::vector" }` is the long form of a plain string entry. An object with *both* keys, or neither, is an error — the two cannot be told apart by looking at the text, and guessing wrong emits code that does not compile.

This is also the only way to register a container that is not a template at all.

8.2 Foreign matrices

`sequences` one dimension up. A 2-D type — `Eigen::MatrixXd`, a library's own dense grid — is a sequence in no useful sense, so it needs its own registration:

```
"matrices": [{ "type": "Eigen::MatrixXd" }]
```

Entries take the **same two forms** as `sequences`: a plain string for a one-type-parameter template (`"mylib::Grid"`, spelled `mylib::Grid<double>` by the adapter), or `{ "type": "..." }` for a concrete type.

The type must be default-constructible with `value_type`, `rows()`, `cols()`, `resize(r, c)` and `operator()(i, j)`; the element must be **arithmetic** — a grid of strings has no natural script shape.

The opted-in backends marshal it **by copy through a `std::vector<std::vector<element>>` boundary** — an array of row arrays. `typescript` says `number[][]`.

- **Row-major, whatever the matrix stores.** `operator()(i, j)` is the only access the registration promises, so the boundary is built row by row regardless of the type's own storage order.

- **A ragged incoming array is squared off** to the first row's length: rows are the outer size, columns the first row's; short rows keep whatever `resize()` left and extra columns are dropped.
- **A mutable `Mat&` parameter binds input-only**, like a sequence one.

8.3 Foreign-library interop

A method taking or returning `Eigen::VectorXd` is, to the reflection walk, a method taking an ordinary class. Every backend used to bind it happily — and the call then **threw at run time**, because no caster for that class was ever registered.

`interop` names foreign libraries whose types the target's binding framework can already marshal on its own:

```
"interop": ["eigen"]
```

Recognition is by **enclosing namespace**, not by a list of type names, so one line covers everything the library owns: `VectorXd`, `MatrixXd`, `Vector3d`, `Map`, `Ref`, `Block`, `Array`, the sparse types — including spellings a hand-maintained list would have missed.

Backend	Behaviour
`python`	Emits `#include <pybind11/eigen.h>`; the types bind **natively as numpy arrays**, both directions, matrices included. No adapter, no copy where the layout allows a view.
`nanobind`	Same, through `<nanobind/eigen/dense.h>`. (Sparse is not emitted — `<nanobind/eigen/sparse.h>` costs compile time a rare signature does not justify.)
every other backend	**Skips** the member. Deliberate, and an improvement: a skipped method is honest, a bound one that always throws is not.

Two practical notes: Eigen must be on the include path of the generated binding (it already is if `user_include` lists it, which it must anyway for the bound headers to compile), and the caster header is only emitted when a bound signature actually names such a type — so declaring `interop` on a library that never exposes one costs nothing.

8.3.1 Reaching the caster-less backends

`node`, `wasm` and `lua` have no caster to lean on, so on their own they skip every Eigen-typed member. Give them the flat array by **also** registering the concrete type as a sequence:

```
"interop":   ["eigen"],
"sequences": [{ "type": "Eigen::VectorXd" }]
```

The IR then carries both marks, and each backend picks:

`python` / `nanobind`	The **caster wins** — still numpy, still no copy. A dual-marked type costs them nothing.
`node` / `wasm` / `lua`	Bind through the **sequence adapter**: in, out and back through a `std::vector<double>` boundary. Scripts see a plain array.
`typescript`	Declares `number[]`, matching what those runtimes hand out.

1-D only. `MatrixXd` has no useful reading as a sequence — register it under `matrices` instead, which splits the backends exactly the same way:

```
"interop":   ["eigen"],
"sequences": [{ "type": "Eigen::VectorXd" }],
"matrices":  [{ "type": "Eigen::MatrixXd" }]
```

covers both shapes, in both directions, on every expanded backend.

8.4 Two standard types that need no declaration

Nothing to write in the manifest for these. They are here because a factory API tends to name both at once, and until recently either one made a signature quietly unbindable on the caster-less backends:

```
std::shared_ptr<Mesh> compile_file(const std::filesystem::path &input);
```

`std::filesystem::path` marshals as a string

It is described as `kind: "string"` with an `is_path` flag.

`python` / `nanobind`	**Native**, through `<pybind11/stl/filesystem.h>` / `<nanobind/stl/filesystem.h>` — callers pass a `str` *or* any `os.PathLike`, and a returned path comes back as `pathlib.Path`.
`node` / `wasm` / `lua`	Through the same copy adapter the foreign containers use: the boundary declares `std::string`, the emitted code builds a path from it and calls `.string()` on the way back.
`typescript`	Declares `string`.

`std::shared_ptr<T>` crosses in the return direction

Which is where a factory needs it. The pointee `T` must itself be a bound class.

`python`	`py::class_<T, std::shared_ptr<T>>` — the holder is declared automatically for every pointee the module hands out.
`nanobind`	Native, through `<nanobind/stl/shared_ptr.h>`.
`wasm`	The class registration gains `.smart_ptr<std::shared_ptr<T>>("T_sp")`, and only for classes that actually travel that way.
`lua`	Native — sol2's own `unique_usertype_traits` handles it.
`node`	The JS object **adopts** the pointer: a third ownership mode in the wrapper, next to "owns" and "aliases a member of a pinned parent". The C++ object lives as long as the JS handle does.
`typescript`	Declares the *pointee* (`Doc`), not `shared_ptr`.

Two limits:

- **Returns only.** A `shared_ptr<T>` *parameter* stays skipped: converting an incoming script object would mean manufacturing a control block over memory the binding does not own — a double-free waiting to happen. Take `const T&`.
- **`node`: untrampolined pointees only.** A class with virtual methods is wrapped as its trampoline subclass, and the adopting wrapper stores a `T*`; reading one as the other

is undefined, so such a return stays skipped there. The other backends have no such restriction.

Both work with a non-copyable `T` — which is the point, since a class handed out by a factory usually is one.

8.5 Deciding what you need

Your signature names…	**Write**
your own `Vec<T>` template	`"sequences": ["my::Vec"]`
`Eigen::VectorXd`, Python only	`"interop": ["eigen"]`
`Eigen::VectorXd`, everywhere	`"interop"` + `"sequences": [{"type": "Eigen::VectorXd"}]`
`Eigen::MatrixXd`, everywhere	`"interop"` + `"matrices": [{"type": "Eigen::MatrixXd"}]`
a 2-D grid of your own	`"matrices": ["my::Grid"]`
`std::filesystem::path`	nothing
`std::shared_ptr<Bound>` returned	nothing
a `std::function` parameter	nothing will help — the member is skipped, with a reason. Add an `extensions` wrapper if you need it.

When in doubt, generate and read `coverage.json`: it names every type that failed a gate, and which one.

Chapter 9

Presets

Some binding surfaces are **fixed**: they do not depend on your project at all. Binding rosetta's own reflection API is the case that motivated this — the twelve `rosetta::script` handle classes and the eight registry free functions are the same in every project that wants them, and copying twenty entries into each manifest makes the three lines that *are* project-specific hard to find.

9.1 Using one

`preset` names a JSON fragment shipped under

`<rosetta_include>/rosetta/presets/<name>.json`

```
{
    "preset": "scriptable",
    "rosetta_include": "../../include",
    "user_include": ["../lib", "../lib/bindings/dynamic"],
    "user_sources": ["../lib/bindings/dynamic/auto_dynamic.cpp"],
    "module_init": { "headers": ["auto_dynamic.h"],
                     "statements": ["bank::register_all()"] },
    "targets": ["python", "lua"]
}
```

That is a complete manifest. Six keys, every one of them about *this* project: where its files are, which targets to emit, what to run at load. `classes`, `functions` and `module_name` all come from the preset — and a preset that supplies `module_name` is what lets the targets stay bare strings.

Give an array to apply several: `"preset": ["scriptable", "..."]`, applied in order. An unknown name is an error naming the path that was searched, so a typo surfaces immediately.

9.2 What applying one does

Merge	Deep, with the **manifest winning** every conflict it can express. A key you did not write is taken from the preset; arrays present in both are concatenated *preset first*; objects merge recursively; a scalar you spelled out is never overridden.
`classes` / `functions`	Parsed with a **pristine context**: your `namespace` and `header_dir` defaults describe *your* tree and are not prepended to a preset's headers. They are appended after your own entries.
Include path	A preset binds headers that live under `rosetta_include`, so it puts that directory on the **target's** include path too. Every other binding is reflection-free by construction and has no reason to see rosetta's headers, which is why this is not the default.

The array rule is what makes `module_init` compose: the preset contributes its caster include and *your* statements survive, rather than one silently replacing the other.

`classes` becomes optional when a preset supplies it — which is why the manifest above has none.

9.3 The shipped preset

Name	What it binds
`scriptable`	`<rosetta/script.h>` — the bindable facade over the runtime object model — plus the per-language `Value` casters. Lets a host language walk the metadata of any library that emits `dynamic` tables, and call into it by name.

Chapter 15 is what that preset is *for*, and is worth reading before using it — it is the most unusual thing in rosetta and the hardest to guess at from the manifest alone.

9.4 Writing your own

A preset is only a JSON fragment. Drop one into `<rosetta_include>/rosetta/presets/` and it is available by name to every manifest that points at that include directory.

It is worth doing when a binding surface is genuinely fixed across projects:

- a company-wide utility library every binding also exposes;
- a facade type plus its helpers, where the twenty entries never vary;
- a set of `sequences` / `matrices` / `interop` registrations shared by every project using the same numerical stack.

It is *not* worth doing to factor out the parts of a manifest that differ per project — `user_include`, `user_sources`, `targets` — because the merge rule (manifest wins) means those would be overridden anyway, and a preset nobody can use unchanged is just an indirection.

The fields most worth putting in one:

```
{
  "module_name": "mylib",
  "classes":   [ ... fixed entries ... ],
  "functions": [ ... fixed entries ... ],
  "module_init": { "headers": ["mylib/init.h"],
```

```
                    "statements": ["mylib::initialize()"] },
  "sequences": ["mylib::Vector"]
}
```

NOTE

Remember the pristine-context rule when writing one: a preset's `classes` entries must spell their headers and names in full, because the consuming manifest's `namespace` and `header_dir` will *not* be applied to them.

Part III

Annotations and documentation

Chapter 10

Annotations

Annotations are rosetta's **opt-in enrichment layer**: small declarative tags attached to your fields and methods that backends turn into docstrings, validation and user interface. They are never required — reflection alone binds the whole class — and they never change the C++ behaviour of your types.

They buy three things:

1. **Documentation** — `doc` becomes docstrings, reference pages, tooltips.
2. **Validation** — `range` rejects bad assignments at the binding boundary.
3. **User interface** — `label`, `button`, `combobox` and the `widget::*` hints drive the generated Qt, QML and Dear ImGui inspectors.

10.1 Two spellings, one result

Every annotation can be written **inline** (as a P2996/P3394 attribute in the header) or **out of line** (as a JSON side-car, leaving the header plain C++). The two forms produce identical bindings and can be mixed on the same class.

```
#include <rosetta/annotations.h>

class Algo {
public:
    [[= rosetta::doc{"Max solver iterations"}, = rosetta::range{0, 200},
       = rosetta::widget::slider, = rosetta::label{"Max iter"} ]]
    int maxIter{100};

    [[= rosetta::doc{"Run the solver"}, = rosetta::button{"Run"} ]]
    double run();
};
```

. . . or the same thing with a clean header and a side-car:

```
{
  "maxIter": { "doc": "Max solver iterations", "range": [0, 200],
               "widget": "slider", "label": "Max iter" },
  "run":     { "doc": "Run the solver", "button": "Run" }
}
```

Annotations are read *once*, on the generation host, and merged (inline first, then JSON) into the same annotation list. Backends cannot tell the two sources apart.

10.2 The annotation set

Kind	Applies to	Spellings	Meaning
`doc`	fields, methods, classes*	`= rosetta::doc{"..."}` `"doc": "..."`	Description text. Docstrings, reference pages, OpenAPI descriptions, hover tooltips.
`range`	numeric fields	`= rosetta::range{lo, hi}` `"range": [lo, hi]`	Value bounds. Compiled backends emit a **validating setter**; UI inspectors render a clamping slider; doc backends print the bound. Scientific notation is fine (`[1e-10, 1e-6]`).
`readonly`	fields	`= rosetta::readonly` `"readonly": true`	Getter-only property; writes are rejected per backend; UI editors grey out.
`combobox`	string fields	`= rosetta::combobox{{"a","b"}}` `"combobox": ["a","b"]`	Allowed choices (maximum 16). UI renders a drop-down.
`label`	fields, methods	`= rosetta::label{"..."}` `"label": "..."`	Display-name override in UI backends. The C++ identifier stays the lookup/binding key — only the visible text changes.
`button`	methods	`= rosetta::button{"Run"}` `"button": "Run"`	Caption of the call button a UI backend renders for the method.
`widget::*`	fields	`= rosetta::widget::slider` `"widget": "slider"`	Editor hint — see below.

* Class-level and free-function `doc` come from the **manifest** (a `doc` key on the entry), since free functions carry no in-source annotations.

10.2.1 Widget hints

Hints pick the editor when more than one would fit the field's type. They are consumed by the three UI inspector backends and ignored everywhere else.

Hint	Field type	Renders as
`spin`	number	spin box / typed input
`slider`	number + `range`	slider with the range as bounds (clamps by construction)
`textfield`	number	validated free-text input — the right editor for values a slider cannot hit precisely (an eps of `1e-7`)
`checkbox`	number (0/1), bool	checkbox
`color`	string `"#rrggbb"`	colour picker (ImGui `ColorEdit3`, Qt swatch + `QColorDialog`, QML `ColorDialog`); edits write back as hex
`multiline`	string	multi-line text area
`radio`	string + `combobox`	the choices as a radio-button group instead of a drop-down
`file`	string path	text entry + browse button opening the platform file dialog

10.3 Who consumes what

Scripting backends
(`python`, `nanobind`, `node`, `wasm`, `lua`, `julia`, plus `csharp` / `java`): `doc` becomes docstrings where the framework has them; `range` becomes validating setters; `readonly` becomes getter-only properties. `label`, `button` and `widget::*` are ignored — they are UI concepts.

UI inspectors
(`qt`, `qml`, `imgui`): everything. `doc` as tooltips, `range` as slider bounds or commit validation, `readonly` as disabled editors, `combobox` as drop-downs, plus `label`, `button` and `widget::*`.

Document backends
(`markdown`, `html`, `openapi`, `typescript`): `doc` and the constraints (`range`, `readonly`, `combobox`) are rendered into the reference text or schema.

An annotation a backend has no use for is simply ignored — never an error.

10.4 Out-of-line annotations

The side-car keeps your header plain C++. That matters for three reasons, and the second is the one that usually decides it:

	Inline	**Side-car**
Header stays plain C++	**no**	**yes**
Generated binding builds on an off-the-shelf compiler	**no**	**yes**
Works on third-party headers you cannot edit	**no**	**yes**
Expressiveness	full set	full set

The middle row is the load-bearing one. Inline annotations pull `<experimental/meta>` into your header, and the generated binding `#includes` that header — so the "reflection-free by construction" promise (Section 1.6) is lost for every target.

10.4.1 Wiring it up

One manifest field per class:

```
"classes": [
  { "name": "Algo", "header": "Algo.h", "annotations": "Algo.ann.json" }
]
```

The path is resolved relative to the manifest. The side-car's keys are member identifiers — fields *or* methods — and each value object may carry any of the annotation kinds:

```
{
  "title": { "doc": "The widget title", "label": "Title" },
  "count": { "doc": "Visible items", "range": [0, 100], "widget": "slider" },
  "id":    { "readonly": true },
  "mode":  { "combobox": ["fast", "slow"] },
  "run":   { "doc": "Run the thing", "button": "Run" }
}
```

The bound header never mentions rosetta:

```
#pragma once
#include <string>
struct Widget {
    std::string title;
    int         count = 0;
    int         id    = 0;
    std::string mode;
};
```

That is the whole surface. Run `rosetta_gen`, build, and every backend sees the annotations.

10.4.2 What it does under the hood

`rosetta_gen` bakes the JSON bytes into `bindings.h`, right after the class header is included so `T` is complete:

```
template <> inline constexpr auto rosetta::detail::ann_storage<Widget> =
    std::to_array<char>({ char(0x7b), /* baked JSON bytes */ '\0' });
template <> inline constexpr std::string_view rosetta::ann_json_source<Widget> =
    std::string_view{ rosetta::detail::ann_storage<Widget>.data(),
                      rosetta::detail::ann_storage<Widget>.size() - 1 };
static_assert(rosetta::detail::ann_keys_error<Widget>().empty(),
              rosetta::detail::ann_keys_error<Widget>());
```

A small `consteval` parser reads it at `walk()` time and concatenates the entries with the member's inline annotations. Nothing downstream changes.

Loading the JSON at *run time* could not work: annotations are non-type template parameters consumed during constant evaluation, so the data must exist then. Baking keeps everything in the one constant-evaluated pass — and keeps the `static_assert` below possible.

10.4.3 The staleness guard

That `static_assert` checks every JSON key names a real member of the type. A renamed or misspelt field **fails the build**, naming the offending key, instead of silently dropping its metadata.

> **WATCH OUT**
>
> Side-car edits need a **regeneration**, not just a rebuild — the JSON is baked when `rosetta_gen` runs. Re-run `rosetta_gen` and the generator, not only the binding build.

10.4.4 Attaching the JSON yourself

The manifest's `annotations` field is sugar for specialising a customisation point. You can do it directly, from anywhere — another file, a translation unit, after the class:

```
#include "widget.h"            // the untouched class
#include <rosetta/annotate.h>  // for the customization point

template <>
constexpr std::string_view rosetta::ann_json_source<Widget> = R"({
  "title": { "doc": "The widget title" },
  "count": { "doc": "Visible items", "range": [0, 100] },
  "id":    { "readonly": true },
  "mode":  { "combobox": ["fast", "slow"] }
```

```
})";
```

Same schema, same merge, same effect on every backend. The one rule is that the specialisation must be visible before `rosetta::walk<Widget>()` runs in a translation unit — which is exactly why the manifest route bakes it into `bindings.h`.

10.4.5 Merge precedence

Inline annotations come first in the merged pack, JSON after. So a reader that keeps the last match (`ann::get_or<A>`) lets a JSON entry *override* an inline one of the same kind, while cumulative readers such as `doc` see both.

A class with no side-car uses the empty primary template and is simply un-annotated — no error, no special-casing.

10.5 Internal annotations

`rosetta::virtual_spec` is **synthesised by the walk**, never written by users: it marks a method's reflection as virtual or overriding so backends can emit overridable bindings (pybind11 and N-API trampolines). You only meet it when writing a custom backend.

10.6 Gotchas

- **Inline annotations make the header C++26-only.** Every translation unit that includes it must build with the fork and `-fannotation-attributes`. Use the side-car when you want off-the-shelf builds.
- **Side-car edits need a regeneration**, not just a rebuild.
- `widget::slider` without a `range` falls back to the default editor — a slider needs bounds.
- `widget::radio` does nothing without a `combobox`. It restyles the choices; it does not define them.
- `combobox` is capped at 16 choices. Beyond that a drop-down is the wrong widget anyway.
- `label` changes only the visible text. Scripting backends still bind the C++ identifier.

> **NOTE**
>
> The side-car parser is deliberately small: it takes string values literally (no escape unescaping) and parses numbers as plain decimals. Enough for the annotation schema, not a general JSON library.

Chapter 11

Documentation

The previous chapter was about metadata you *add*. This one is about metadata you already have.

Your library is probably documented. Its headers carry `///` lines and `/** @param ...*/` blocks, its declarations name their parameters, and its defaults are written down where they belong — in the C++. Until you bind it, none of that reaches the people calling your library from Python or TypeScript: an annotation-only pipeline can say nothing your headers did not repeat, in rosetta's own vocabulary, a second time.

rosetta reads the headers it is already opening. The result is that a generated binding arrives documented, with keyword arguments and working defaults, and you write nothing.

11.1 Turning it on

There is nothing to turn on. Harvesting is the default; the only setting is how to turn it **off**.

```
{ "doc_comments": false }     // default: true
```

Set it to `false` to generate exactly what earlier versions of rosetta did — documentation from annotations alone, positional parameter names, no defaults. There is no per-class or per-target switch: the harvest is a property of the *manifest*, and the backends decide what to do with what it finds.

11.2 An example, end to end

A header that never mentions rosetta:

```
/// The direction a polygon's vertices are wound in.
enum class Winding {
    CCW, ///< Counter-clockwise: a positive signed area.
    CW,  ///< Clockwise: a negative signed area.
};

/**
 * @brief A closed polygon, stored as parallel coordinate arrays.
 */
class Polygon {
public:
    /// The x coordinate of each vertex, in order.
    std::vector<double> xs;
```

```
    /**
     * @brief Resample the outline at evenly spaced points.
     *
     * @param samples how many points to produce
     * @param closed  whether to repeat the first point at the end
     * @return the coordinates, x and y interleaved
     */
    std::vector<double> resample(int samples = 64, bool closed = true) const;
};
```

becomes, in Python:

```
>>> help(polygon.Polygon.resample)
resample(self, samples: int = 64, closed: bool = True) -> list[float]

    Resample the outline at evenly spaced points.

    Args:
        samples: how many points to produce
        closed: whether to repeat the first point at the end

    Returns:
        the coordinates, x and y interleaved

>>> p.resample(samples=8, closed=False)   # keyword arguments
>>> p.resample()                          # the C++ defaults apply
```

and, in the `.d.ts`:

```
/**
 * Resample the outline at evenly spaced points.
 * @param samples how many points to produce
 * @param closed whether to repeat the first point at the end
 * @returns the coordinates, x and y interleaved
 */
resample(samples?: number, closed?: boolean): number[];
```

11.3 What is read

In the header	In the IR
`///`, `//!`, `/** */`, `/*! */`, and trailing `///<`	the member's documentation (`GenField::doc`, `GenMethod::doc`)
`@brief`, `@details`, untagged prose	the same
`@param <name> ...`, including the `[in]` / `[out]` / `[in,out]` markers and text running over several lines	`GenParam::doc` — per argument, matched **by name**
`@return`, `@returns`, `@result`	`GenMethod::returns`
`@note`, `@warning`, `@deprecated`, `@throws`, `@pre`, `@post`, `@see`, `@since`	labelled paragraphs appended to the documentation
a comment above a `class` / `struct`	`GenClass::brief`
parameter names in the declaration	`GenParam::name`
default arguments (`int samples = 64`)	`GenParam::has_default` and `GenParam::default_text`

`@tparam`, `@file`, `@ingroup`, `@defgroup` and the other bookkeeping commands are dropped — they describe the source, not the API.

11.4 Which backend does what with it

Every backend receives the same IR. What each one can *do* with it differs, because the host languages differ.

Target	What it emits
`python` `nanobind`	The docstring, with `Args:` and `Returns:` sections folded in — Python has one place for all of it. Parameter names become `py::arg("...")` / `nb::arg("...")`, so calls take keyword arguments; recovered defaults become real Python defaults. The class comment is the type's docstring.
`typescript`	A TSDoc block: the prose, one `@param` per documented argument, and `@returns`. A defaulted parameter is declared **optional** (`samples?: number`), so an editor offers the short call.
`markdown` `html`	The signature carries names and defaults; the `@param` text becomes a list under each method, `@return` its own line. The class comment opens the section.
`openapi`	The operation `description`; each positional argument gets a `title` (its name), a `description` (its `@param` text) and a `default`. The wire shape does not change.
`csharp`	A `/// <summary>` XML doc block, and real parameter names in the wrapper signature.
`java`	A Javadoc block, and real parameter names in the wrapper signature.
`qt` `qml` `imgui`	A field's documentation becomes its tooltip, and parameter names label the argument boxes of a method's call row — an inspector built from a header that was never written for one.
`node` `wasm` `lua` `julia`	Nothing. These runtimes have no docstring slot and their emitted wrappers spell positional locals, so the names and text sit unused in the IR. For `node` and `wasm`, `typescript` is where they land — which is the argument for adding it.
`dynamic`	Names and text sit in the emitted `MetaClass` tables, so a UI built by querying the registry can show them without ever including your headers.

NOTE

Add `typescript` to a manifest that already has `node` or `wasm`. It costs one line, compiles nothing, and it is the only place those two targets' documentation can go.

11.5 Default arguments, and why they are different

Parameter *names* come from reflection: `std::meta::identifier_of` reads them from the declaration. So does the *fact* of a default — `std::meta::has_default_argument`. But P2996 offers no way to ask what the default *expression was*, so its spelling is read from the header text instead.

The two readings are of the same declaration, and they are **cross-checked**: a spelling is used only where reflection independently confirms there is a default to spell. Where they disagree, the text loses.

That leaves the question of which expressions rosetta is willing to repeat into generated C++. The bar is high, because a wrong answer here does not produce a poor docstring — it produces a binding that does not compile.

Accepted	Refused
literals: `64`, `1e-6`, `true`, `'x'`, `"polygon"`	anything with a call: `Point(0, 0)`, `sizeof(int)`
`{}`	an expression: `a + b`, `n * 2`
qualified-ids: `Winding::CCW`, `Limits::Max`	a bare identifier: `kDefaultEpsilon` — it would not resolve where the binding spells it

A parameter whose default is refused is still *known* to be optional, and the backends that only need that say so: `eps?: number` in the `.d.ts`, `required: false` in OpenAPI, `eps?` in the Markdown signature. Only Python, which needs the value itself, falls back to a required argument.

> **WATCH OUT**
>
> The Python family adds a rule of its own. pybind and nanobind convert an `arg` default into a Python object at **module import**, before any call — so a default whose type is not registered yet does not degrade gracefully, it raises `arg(): could not convert default argument` and the whole module fails to import. Defaults are therefore emitted only for numbers, booleans, strings, and **enums bound in the same module**. Bind the enum (list it in `classes`) and `Winding::CCW` becomes a working Python default; leave it out and the default is silently withheld rather than breaking the import.

11.6 Precedence

Harvested text only ever fills a blank. In order:

1. an inline `[[= rosetta::doc{"..."}]]` annotation — the harvester skips a member that carries one outright;
2. a JSON side-car entry (chapter 10), or a `doc` on a manifest `functions` entry;
3. the header's own comment.

So adopting this cannot change a binding you have already annotated, and the two mechanisms compose: annotate the handful of members that need a `range`, a `readonly` or a rewritten description, and let the headers document everything else.

11.7 What it will not do

The harvester is a **scanner, not a compiler**. It tracks namespaces, class bodies, access specifiers, comments, string and character literals and preprocessor lines, and it reads declarations well enough to get a name, its parameters and its defaults — but it does not expand macros and does not follow `#include`.

Which is why nothing it finds is used on sight. Every entry is matched against the **reflected** signature before it is applied, by arity and by parameter names, and dropped when the two disagree. The consequences are worth stating plainly:

- **Ambiguous overloads document neither.** Two entries of the same arity whose names cannot tell them apart are both discarded — a plausible sentence on the wrong overload is worse than no sentence.

- **A stale comment loses.** A `@param radius` naming an argument the signature no longer has is dropped, not placed by position.
- **Private members are never read.** A documented private field is invisible to every backend, as it should be.
- **Constructors, destructors and operators carry no documentation** — the IR has no slot for it. Constructor parameter *names* do reach `py::init` and the TypeScript `constructor(...)`.
- **Inherited members need their base bound.** A member flattened in from a base declared in a header no `classes` entry names is not covered; list that base as a class to reach it.
- **Template classes are not matched.** The IR names a specialisation (`SpatialFieldND<2>`); the header declares a template.

> **NOTE**
>
> Harvesting reads only the headers the manifest names — the ones listed by `classes`, their `extensions`, and `functions` — resolved against `user_include` exactly as the compiler will resolve them. A header that resolves nowhere contributes nothing and is not an error: it may be a `generated_headers` entry that does not exist yet.

11.8 Where to look next

`examples/doxygen` is this chapter as a runnable project: a documented header, a three-line manifest, and a `test.py` that asserts against the compiled module — including all three of the cases above where rosetta deliberately does less than you might expect. The manifest field is in appendix A; the reference document is `docs/MANIFEST.md`.

Part IV

Driving the tool

Chapter 12

The `rosetta_gen` tool

`rosetta_gen` is the single binary that drives the whole pipeline. It reads a manifest and can take you from "a folder of C++ headers" to "compiled bindings for every declared language" in one command — or one step at a time, when you want to inspect what each stage produces.

The tool itself uses **no reflection**, only JSON parsing and text templating, so it builds with an off-the-shelf C++17 compiler. Only the *generator* it emits — and the `rest` and `json` targets — needs the clang-p2996 toolchain.

12.1 Building it

One-time, from the repository root (CMake $\geq$ 3.28; it fetches the single-header nlohmann/json automatically):

```
cmake -G Ninja -S tools/rosetta_gen -B tools/rosetta_gen/build
cmake --build tools/rosetta_gen/build
```

The binary always lands in **`<repo>/bin/rosetta_gen`**, not in the build tree. If you scaffolded your project with `rosetta_gen -init` or `tools/rosetta_init.py`, its bootstrap `CMakeLists.txt` does this for you and the binary appears at `extern/rosetta/bin/rosetta_gen`.

12.2 The four modes

```
rosetta_gen <manifest.json> [out_dir]        # plain: emit the generator project only
rosetta_gen --build <manifest.json> [opts]   # the whole pipeline in one command
rosetta_gen --clean <manifest.json> [opts]   # remove everything generated
rosetta_gen --init  [manifest.json] [src]    # write a starter manifest + project skeleton
```

`rosetta_gen -help` prints the full built-in help; `-build -help` and `-clean -help` list each mode's options.

```
--init --> manifest.json --> --build ------------> bindings/python  node  wasm ...
 (start)    (you edit it)      |                          ^
                               | = the three manual steps:|
                               |   1. emit gen/           |
                               |   2. cmake gen/, run it -+
                               |   3. cmake / npm / emcmake each backend
--clean <-- removes gen/ (or generated/), the generator binary and bindings/
```

12.3 `-init`: starting a project

Writes a starter manifest (default `./manifest.json`). It **never overwrites** an existing file — remove it first to regenerate.

Beside the manifest it also writes the project skeleton:

CMakeLists.txt bootstrap only: fetches rosetta into `extern/` and builds `rosetta_gen` (`extern/rosetta/bin/rosetta_gen`);

.gitignore the generated trees (`build/`, `extern/`, `gen/`, `bindings/`, ...);

README.md the two build commands of Section 3.2.

Each is written only if it is not already there — an existing file is kept and reported — so `-init` can be run inside a project that has its own. The CMake project and README title take the scanned folder's name (`mylib` for the commented example).

A blank, commented manifest

```
bin/rosetta_gen --init
```

Emits a fully-commented example exercising every commonly-used field: a `variables` declaration feeding the `cpp26_*` overrides, a multi-entry `user_include`, `user_sources`, `compile_definitions`, `build_type` / `optimization`, a representative spread of targets, one class and one function. The idea is that you delete what you do not need rather than hunting the documentation for what exists. `doc_comments` is in there too, spelled at its default (`true`) so you can see the switch exists — it is the one field you set in order to get *less*.

Pre-filled from a source scan

```
bin/rosetta_gen --init manifest.json ./src
```

An argument naming an existing **directory** is the source tree to scan; the other, if any, is the manifest path. The scan walks for headers (`.h/.hh/.hpp/.hxx`) and sources (`.cpp/.cxx/.cc`) and pre-fills:

- `classes` and `functions`, from the declarations found, with shared `namespace` / `header_dir` defaults factored out when every entry agrees;
- `user_sources` — every source file found, **except** any defining `main()` (it would clash with the binding modules' entry points; reported as a note);
- `rosetta_include`, guessed by walking up from the `rosetta_gen` binary; a `FIXME` comment otherwise.

WATCH OUT

The scan is a comment- and string-aware *token scan*, not a real C++ parse. Template classes, overloaded free functions and anonymous namespaces are skipped, each with a printed note. An overloaded free function is skipped because the scan has no signature to write, not because it cannot be bound — add it by hand with a `signature` (Section 5.4.1). **Review the result before building.**

VCS directories, build trees and vendored code (`build/`, `extern/`, `third_party/`, `node_modules/`, ...) are not descended into.

> **NOTE**
>
> No `rosetta_gen` binary yet? `tools/rosetta_init.py` writes the same skeleton (and a manifest skeleton) with nothing but Python.

12.4 `-build`: the whole pipeline

```
bin/rosetta_gen --build manifest.json
```

runs, in order:

1. **emit** the generator project into `<manifest dir>/gen/`. This is also where the bound headers are read for their doc comments and default arguments (chapter 11), so the driver carries them as data — no toolchain involved, and `doc_comments false` skips it;
2. **build** it with CMake — this step needs the clang-p2996 toolchain, and reflection runs here — and **run** the resulting generator, writing one self-contained project per target into `<manifest dir>/bindings/<lang>/`;
3. **compile every backend** the manifest declares, each with its own build shape.

Backend	Build
`python`, `nanobind`, `rest`, `json`, `lua`, `imgui`, ...	plain `cmake` configure + build
`node`	`npm install` + `npm run build`
`wasm`	`emcmake cmake` + `cmake --build`
`julia`	`cmake` (locates JlCxx by running `julia`)
`csharp`	`cmake`, then `dotnet build` as a second stage
`java`	`cmake`, then `mvn package` as a second stage
`qt`, `qml`	`cmake` with `-DQT_DIR`
`markdown`, `html`, `typescript`, `openapi`, `paraview`	nothing to compile

A backend whose toolchain is **missing** — no `npm`, no `emcc`, no `julia` — is *skipped with a note*, not an error. Likewise a missing `dotnet` or `mvn` downgrades `csharp` / `java` to "OK (native only)". A backend whose build **fails** is reported, and `-build` exits non-zero *after trying the rest*.

Every run ends with a per-backend summary:

```
== summary
   python      OK
   node        OK
   wasm        SKIPPED (emcc not found -- activate emsdk)
   markdown            OK (nothing to compile)
```

12.4.1 Options

```
--gen-dir DIR            generator project dir   (default: <manifest dir>/gen)
--bindings-dir DIR       generated bindings dir  (default: <manifest dir>/bindings)
```

```
--clang-p2996-root PATH  -DCLANG_P2996_ROOT for every CMake configure
--qt-dir PATH            -DQT_DIR for the qt / qml builds
--only a,b / --skip a,b  restrict the backend builds to / exclude these
--cmake-arg ARG          extra argument for every CMake configure (repeatable)
--jobs N, -jN            parallel build jobs (bare -j: one per core)
--fresh                  wipe the gen and bindings dirs first
--wheel                  also build a Python wheel for the python /
                         nanobind targets
--wheel-dir DIR          collect every wheel in DIR (implies --wheel)
```

12.4.2 Iterating on one or two backends

While developing you rarely want the full matrix. `-only` and `-skip` filter **which backends get compiled** — generation still emits them all, and the filters name the `lang` values:

```
# just python and node, 8-way parallel
bin/rosetta_gen --build manifest.json --only python,node -j8

# everything except the slow wasm link
bin/rosetta_gen --build manifest.json --skip wasm
```

12.4.3 A custom reflection toolchain

`rest` is the one target whose generated code still splices reflections, so it re-runs reflection when *it* compiles. Its CMake configure needs to know where the clang-p2996 build lives if it is not at the manifest's default:

```
bin/rosetta_gen --build manifest.json \
    --clang-p2996-root ~/devs/clang-p2996/build --fresh
```

`-fresh` wipes `gen/` and `bindings/` first — the right move after toolchain or manifest-layout changes, when stale CMake caches would otherwise bite.

12.4.4 Qt and QML

```
bin/rosetta_gen --build manifest.json --only qt,qml --qt-dir ~/Qt/6.8.3/macos
```

Equivalently, set `qt_dir` in the manifest and drop the flag.

12.4.5 Passing flags to every configure

`-cmake-arg` is repeatable and applies to the generator's configure *and* every backend's, including the emcmake configure for wasm:

```
bin/rosetta_gen --build manifest.json \
    --cmake-arg -DCMAKE_BUILD_TYPE=Debug \
    --cmake-arg -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

12.4.6 Python wheels

By default `-build` compiles the extension module and stops — it produces an importable `.so` / `.pyd`, not a distributable artifact. `-wheel` adds the packaging step for the two backends that emit a `make_wheel.py`:

```
bin/rosetta_gen --build manifest.json --only python,nanobind --wheel
```

Wheels land in `<bindings>/<lang>/dist`, with the repaired (redistributable) copies in `dist/repaired`. It is opt-in because packaging is a release action, not a development one: it pip-installs `build`, `scikit-build-core` and a repair tool into the environment, and takes noticeably longer than the compile `-build` is otherwise optimised for.

`-wheel-dir` collects every backend's wheels in one place instead — the shape you want when the next step is uploading them:

```
bin/rosetta_gen --build manifest.json --wheel-dir ./dist
```

```
dist/
  pygeom-0.2.0-cp312-cp312-macosx_15_0_arm64.whl
  nbgeom-0.2.0-cp312-abi3-macosx_15_0_arm64.whl
  repaired/          <- the redistributable copies, same names
```

Sharing one output directory is safe: each `make_wheel.py` repairs only the wheels *it* just produced.

Both flags have manifest counterparts — `wheel` and `wheel_dir` on the `python` / `nanobind` entry of `targets` (Section 7.5) — so a project that always packages says so once. They are per-target, which is what lets a manifest ship the `nanobind` wheel and not the `python` one. The flags still win, and only ever toward packaging *more*: a target spelling `"wheel": false` cannot disarm a `-wheel` you typed, and `-wheel-dir` overrides every target's own directory.

> **NOTE**
>
> The wheel build is *independent* of the compile above it — scikit-build-core re-runs the same `CMakeLists.txt` in its own tree, so it neither reuses nor disturbs `<lang>/build/`. The interpreter is probed on `PATH`; if none is found, the backend is recorded `OK (no wheel - no python interpreter found)` rather than failing. A `-wheel` that cannot fire — no such target, or none surviving `-only`/`-skip` — says so instead of quietly doing nothing.

12.5 Plain mode: one step at a time

```
bin/rosetta_gen manifest.json [out_dir]
```

emits **only** the generator project (default `out_dir`: `<manifest dir>/generated/`) and stops. Useful for inspecting `bindings.h`, hacking on the emitted driver, or wiring the steps into your own build system:

```
# 1. manifest -> generator project
bin/rosetta_gen manifest.json gen

# 2. build the generator (clang-p2996) and run it
cmake -S gen -B gen/build && cmake --build gen/build
./generator bindings          # the binary is dropped NEXT TO the project dir

# 3. build any backend by hand
cmake -S bindings/python -B bindings/python/build
cmake --build bindings/python/build
```

Two footguns the tool warns about:

- a fresh checkout (or a `-clean`) leaves `out_dir/build` unconfigured, so `cmake -build` alone fails — the *configure* step is printed as a `next:` hint;
- if the emitted sources changed but a generator binary from a previous emit still sits next to `out_dir`, running it without rebuilding **silently re-emits the old bindings**. The tool prints a rebuild note when it detects this.

Re-emitting is idempotent: unchanged files are not rewritten, so the next `cmake -build` recompiles nothing.

12.6 `-clean`: back to sources

```
bin/rosetta_gen --clean manifest.json [--gen-dir DIR] [--bindings-dir DIR]
```

Removes everything the pipeline generated for this manifest:

- the generator project directory — the explicit `-gen-dir`, or **both** defaults (`gen/` from `-build` and `generated/` from plain mode);
- the generator binary dropped beside it, including per-config subdirectories of multi-config generators;
- the per-backend folders under `bindings/` **that the manifest declares**. Anything else you parked there survives, and `bindings/` itself is only removed once empty.

Never the manifest — and never a folder whose `CMakeLists.txt` lacks the `Generated by rosetta` stamp the emitters write, so a hand-written folder sharing a name survives:

```
not removing gen (no rosetta_gen marker in its CMakeLists)
```

12.7 Exit status

Mode	0	non-zero
plain	project emitted	manifest missing / invalid, emit error
`-build`	every *attempted* backend built (skips do not count)	generator build or run failed, or at least one backend build failed
`-clean`	clean ran (nothing to remove is fine)	manifest missing / invalid
`-init`	manifest written	target file already exists, bad arguments

Chapter 13

Overloads and coverage

Two features that belong together, because the second is what makes the first readable: rosetta binds **overload sets**, and every generation writes a machine-readable `coverage.json` saying what bound, what did not, and why.

13.1 Overloads

The reflection walk deduplicates member functions by **(name, signature)**, so of

```
struct Grid {
    double at(int i) const;
    double at(int i, int j) const;
};
```

the whole set arrives in the IR. What happens next depends on the target language, because "two functions with one name" is not something every runtime can express.

Target	Policy	Why
`python`	**all**	repeated `.def("at", ...)` builds one overload set; pybind dispatches on argument types
`nanobind`	**all**	same model as pybind11
`julia`	**all**	each becomes a Julia method; multiple dispatch picks
`qt`	**all**	builds an independent widget row per method — nothing is keyed by name
`wasm`	first only	a second `.function("at", ...)` throws `BindingError` at module init
`node`	first only	property descriptors are keyed by name; JS has no type dispatch
`lua`	first only	`c["at"] = ...` is an assignment — a second one overwrites
`csharp` / `java`	first only	the op table is a name-keyed map, and marshalling goes through JSON
`typescript`	first only	the `.d.ts` describes the N-API module, so it must not promise what `node` did not bind
`rest`	first only	a route is a URL path; two overloads would register the same path
`qml`	first only	`registerInvoker` is keyed by name, and QML calls with an untyped `QVariantList`

"First" always means **first-declared**, which is stable across regenerations.

Every dropped overload appears in `coverage.json` with the reason `overload_not_expressible`, so you can see exactly what a name-keyed target left behind — and reach it with an extension method under a distinct name if you need it.

> **NOTE**
>
> This is also the one thing the runtime object model buys back. Through the `dynamic` projection, those same five name-keyed targets get their overloads *back*, because resolution happens at call time rather than at generation time. See Chapter 15.

13.1.1 C++ name hiding is honoured

```
struct Base    { int f(int) const; int f(int, int) const; };
struct Derived : Base { int f(double) const; };
```

`Derived` binds **only** `f(double)`. That is what a C++ caller sees without a `using Base::f`, so binding the hidden base overloads would hand scripts a call the C++ API does not offer. Both hidden declarations are recorded as `hidden_by_derived` drops rather than vanishing.

A diamond-shared base member is still emitted exactly once. Two *different* bases declaring the same name both bind — that call is ambiguous in C++ without qualification, but the binding has no ambiguity to resolve (each entry is spliced from its own reflection), so binding both beats binding neither.

13.2 `coverage.json`

`generate()` writes it to `<out_dir>/coverage.json` on every run, always. An empty skip list is itself the useful answer, and a file that is always present is one you can diff.

```
{
  "rosetta_coverage": 1,
  "reflection": [                        // never reached ANY backend
    { "class": "Shape",
      "dropped": [
        {"member": "operator<", "signature": "bool(const Shape &) const",
         "reason": "no_identifier"},
        {"member": "debug", "signature": "", "reason": "function_template"}
      ] }
  ],
  "targets": [
    { "target": "wasm",
      "bound": 3, "skipped": 1,
      "classes": [
        { "class": "Shape",
          "bound":   [ {"member": "area", "signature": "double () const"} ],
          "skipped": [ {"member": "scale", "signature": "void (double, double)",
                        "reason": "overload_not_expressible",
                        "detail": "the target binds methods by name and ..."} ] }
      ] }
  ]
}
```

13.2.1 The two stages

`reflection` lists members the walk never handed to any backend. These are backend-independent:

Reason	Meaning
`no_identifier`	an operator or conversion function — no name to bind to
`function_template`	a member template: no fixed parameter pack to splice
`hidden_by_derived`	a base overload hidden by a derived declaration of the name

`targets` is what each backend's own gates decided:

Reason	Meaning
`unmarshalable_signature`	no conversion for a type in the signature; `detail` names it
`overload_not_expressible`	the target keys methods by name and this one lost the tie
`sequence_not_adaptable`	a registered sequence with no `std::vector` boundary adapter here
`static_callback_receiver`	embind cannot give a static member the receiver a callback adapter needs
`extension_has_no_member_pointer`	this backend dispatches through a member pointer

Both the **bound** and the **skipped** side are recorded. Counting only skips cannot tell "everything bound" apart from "this class never reached this backend" — and that is exactly the distinction that matters when a whole target silently produces nothing.

Free functions get their own `"functions": {"bound": [...], "skipped": [...]}` array beside `"classes"`. On a library whose algorithms are free functions over one data structure, that is most of the API.

13.3 Using it

The point is the *diff*. Check `coverage.json` into your repository next to the manifest, and a method that stops binding becomes a reviewable line in a pull request instead of an `AttributeError` a week later.

To gate CI on it:

```
jq -e '[.targets[].skipped] | add == 0' bindings/coverage.json
```

More usefully: allow known skips and fail on new ones by diffing against a committed baseline. A binding surface that only ever grows deliberately is worth the twenty lines of CI it costs.

> **WATCH OUT**
>
> One asymmetry to know before comparing numbers across targets: `typescript` and `dynamic` record bound *fields*, while the other language backends record only methods. The top-level `bound` scalar is therefore not comparable across those two groups — count methods by signature instead.

13.4 Instrumenting a backend of your own

Skips are recorded at the existing gate sites, one line each:

```
for (const auto &m : k.methods) {
    if (!coverage::emit_overload(coverage::overloads::first_only,
                                 "my-backend", k, m)) {
        continue;                                   // records the dropped overload
    }
    if (!my_method_ok(m, c)) {
        coverage::note_skip("my-backend", k, m, "unmarshalable_signature",
                            my_skip_reason(m, c));  // a slug plus a sentence
        continue;
    }
    segs.push_back(my_method(k, m));
    coverage::note_bound("my-backend", k, m);
}
```

Free functions use `note_bound_function` / `note_skip_function`, fields `note_bound_field` / `note_skip_field`.

> **NOTE**
>
> Keep the reason function next to the predicate it explains. A reason that drifts out of step with the decision is worse than no reason at all.

13.5 A note for backend authors

`&T::name` names an **overload set**, not a function, so any emitter that spells a member pointer for an overloaded name must cast it to the exact signature. Use the shared helper:

```
const std::string mp = px_member_pointer(k, m);   // &T::f, or a static_cast
```

The gate is `GenMethod::is_overloaded` — does the *C++ class* overload the name? — and **not** `overload_count`, which is how many entries reached the IR. A set whose siblings were all gated out still needs the cast.

Part V

Beyond bindings

Chapter 14

The dynamic object model

Every backend so far spends the reflection result on *framework calls*: `py::class_<Mesh>().def("at", ...)`, an N-API property descriptor, a sol2 usertype. The information is consumed at generation time, and what survives is code for one specific runtime.

The `dynamic` backend spends it on **data instead**. It writes the IR back out as static tables, so the answer to "what does this class have" is still available at run time — to a caller that was never compiled against your headers.

14.1 What it emits

```
bindings/dynamic/auto_dynamic.h     declares <lib>::register_all()  -- idempotent
bindings/dynamic/auto_dynamic.cpp   the tables
bindings/dynamic/inspect.cpp        a registry walker, for free
bindings/dynamic/CMakeLists.txt     a static library, <lib>_dynamic
```

`auto_dynamic.cpp` is the only file that includes your header. It is ordinary C++20: the metadata is data, so nothing here needs reflection to compile.

14.2 Using it

```
#include <rosetta/dynamic.h>     // the runtime -- ordinary C++20
#include "auto_dynamic.h"        // the generated tables (scene.h is NOT included)

scene::register_all();           // publish the tables into the registry, once

for (const rosetta::dyn::MetaClass *k : rosetta::dyn::registry().classes())
    for (std::size_t i = 0; i < k->n_fields; ++i)
        std::cout << k->qualified << "::" << k->fields[i].name << '\n';
        // ... plus .type, .doc, .readonly, .range, .choices
        //     and the .get / .set thunks
```

Note what the consumer does *not* do: it never includes `scene.h`, and it names no bound type. That is the property everything else in this chapter follows from.

14.3 What the tables look like

One `TypeDesc` per distinct type, shared by address; one `MetaField` / `MetaMethod` / `MetaCtor` array per class; one `MetaClass` tying them together. Straight from generated output:

```
TypeDesc td_0{.kind = Kind::number, .spelling = "double", .integral = false};
TypeDesc td_6{.kind = Kind::vector, .spelling = "std::vector<double>",
              .element = &td_0};

const MetaField k_scene__Vec3_fields[] = {
    {.name = "x", .type = &td_0, .doc = "X component",
     .annotations = k_scene__Vec3_x_anns, .n_annotations = 2,
     .get = +[](const ObjectRef &self, const ArgList &) {
         return make_any(&td_0, static_cast<scene::Vec3 *>(self.ptr)->x);
     },
     .set = +[](const ObjectRef &self, const ArgList &a) {
         static_cast<scene::Vec3 *>(self.ptr)->x = value_cast<double>(a[0]);
         return Any::none();
     }},
```

Three properties of that shape are deliberate:

- **Aggregate initialisers of trivially-constructible data** — `const char *`, pointer-and-count spans, function pointers. No `std::string`, no `std::vector`, no dynamic initialisation. A 500-class library's metadata lands in `.rodata`, not in a static constructor that runs at load.
- **One captureless lambda per member**, decaying to a plain function pointer: `using Thunk = Any (*)(const ObjectRef &self, const ArgList &args)`. That is the *only* callable shape in the model — a field getter, a field setter, an instance method, a static method and a free function all reduce to it. `self` is the receiver, default-constructed when there is not one.
- **`TypeDesc` is structural, not an identity.** `std::type_index` answers "are these the same type"; a UI and a marshaller need "what is *inside* it", so `element` recurses and `vector<vector<Foo>>` is describable.

14.4 Why `register_all()` exists

Two classes can name each other's types, so `TypeDesc::cls` cannot be filled during static initialisation without an ordering you have no way to guarantee. The generated function adds every class, enum and function to the registry and then calls `link()` to resolve those back-pointers:

```
void register_all(rosetta::dyn::Registry &r) {
    r.add_class(&k_scene__Vec3);
    r.add_class(&k_scene__Mesh);
    r.link();
}
```

It is idempotent, and it is the one line a consumer must run before querying anything.

14.5 Calling by name

`Any` is the value type crossing the boundary, and its **canonical representation** is what makes a single generic wrapper possible: whatever the C++ said, the box holds exactly one of `bool`, `long long`, `double`, `std::string`, `std::vector<Any>` or an `ObjectRef`.

So a caller reads a number with `as_number()` without caring whether the signature said `int`, `size_t` or `uint32_t` — and changing one for the other in your library is not a break for the wrapper.

`resolve()` scores every candidate of a name against the actual arguments on a deliberately small ladder — `exact` / `promote` / `convert` / `none` — rather than reproducing C++'s conversion lattice. The question from a dynamically typed host is "did the user mean this overload", not "which standard conversion sequence is shorter".

An equal best score is reported as *ambiguous* rather than silently picked, and a failure names every candidate and why it lost:

```
Mesh::at(std::string) matched no overload of 2:
  at(int) -- argument 1 ("nope") is not a int
  at(int, int) -- takes 2 argument(s), 1 given
```

14.6 Two consequences worth knowing

What cannot be marshalled is described, not deleted

A member whose type no gate claimed keeps its entry, with a null thunk and a stated reason:

```
.skip_reason = "callback: a std::function parameter needs a foreign-callable adapter"
```

So a user interface can grey it out instead of the member silently vanishing. This is the same information `coverage.json` carries, available at run time rather than at review time.

Ownership is explicit

`ObjectRef::owner` is non-null when the reference owns the object. A getter returning `T&` can hand back a *borrowed* handle carrying the parent's owner — which pins the parent and closes the dangling-sub-object hole that a raw-pointer return has.

14.7 What it buys

Relative to the language backends:

- **Overloads come back.** The metadata keeps the whole overload set and scores it against the actual arguments, so the name-keyed targets' first-only limit (Chapter 13) does not apply — and a failed call can name every candidate it rejected, with the reason.
- **UI by query, not by codegen.** A property inspector is a walk over `registry()`: `label` to caption, `doc` to tooltip, `range` to slider bounds, `readonly` to a disabled row, `combobox` and enumerators to a drop-down, `button` to an action row. Annotations are enforced once, in the core, instead of per backend.
- **Lifetime is explicit**, as above.

14.8 What it costs

Every access is a linear name lookup plus a scoring pass.

That is the right price for menus, property panels, commands and scripting glue. It is the wrong price for bulk data — pulling a mesh across boxes every coordinate into an `Any`.

NOTE

Cache the resolved `MetaMethod`: the pointer is stable for the process. And keep an expanded binding for hot paths — the two projections are not exclusive, and a library that emits both gets the property sheet from one and the throughput from the other.

14.9 When to reach for it

You want...	**Use**
Python/JS/Lua callers of your API	a language backend
a property panel over your types	`dynamic` + a walker, or the `qt` / `qml` / `imgui` backends
a command console, a menu system, an undo stack keyed by member name	`dynamic`
one scripting wrapper that works for *any* library you have bound	`dynamic` + the `scriptable` preset (Chapter 15)
maximum throughput on a hot call	a language backend

The worked example is `examples/dynamic`: one set of generated metadata driving a terminal interpreter *and* a Qt viewer — 3-D view, property panel, console — neither of which names a single bound type.

Chapter 15

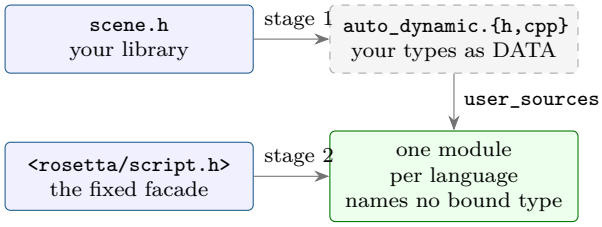
Scripting the metadata itself

The `dynamic` backend makes your library's metadata available *to C++*. Every consumer of it in the repository is C++: the terminal interpreter, the Qt viewer, the ImGui runtime, the QML reflected object — hand-written walkers, one per toolkit.

The last step is to bind *that API* through rosetta's own backends, so the walker can be written in Python, Lua or JavaScript instead. This is the Graphite/GOM move: expose the object model itself to the scripting layer, and the user interface becomes a script rather than a backend.

15.1 The trick, in one sentence

Run the pipeline twice, and make the first run's *output* the second run's *input library*.



By stage 2 your types are **data**, not C++. So stage 2's manifest never names one, and the resulting module works for *any* library that has been through stage 1.

15.2 Why a facade rather than binding `dynamic.h`

`rosetta::dyn` cannot be fed to the backends as it stands. Three shapes block it:

In `dynamic.h`	Why it blocks	In `script.h`
`MetaField *fields; size_t n_fields`	pointer-and-count, so vector marshalling never fires	`std::vector<FieldInfo> fields()`
`const MetaClass *, const TypeDesc *`	raw pointer to an unbound type — the gate skips it	value-semantics handles
`using Thunk = Any (*)(...)`	function pointers marshal nowhere	ordinary `get` / `set` / `call`

`<rosetta/script.h>` is that reshaping and nothing more. Each handle wraps **one `const` pointer** into the tables, so binding it copies no metadata — the `.rodata` guarantee survives.

NOTE

Nothing in the facade is named `Class`, `Type`, `Object` or `Method`, because those collide in the target languages (`java.lang.Class`, JavaScript's `Object`, Lua's `type`). Hence `ClassInfo`, `TypeInfo`, `Instance`, `MethodInfo`.

Errors come back as `Outcome` (an ok flag plus text), never as an exception. Unwinding through a foreign VM's frames is how you get a crash instead of a `TypeError`.

15.3 The one hand-written piece

`Value` is type erasure — one C++ type standing for "whatever the host language just handed us". Reflection can *describe* it but cannot know that a Python `float`, a Lua number and a JavaScript `Number` should all become the same box. That mapping is per-language knowledge, so it is written per language, in `runtime/script/{python,lua,node,wasm}.h`.

Only half of each caster is per-language, though:

- **Reading** a host value is irreducible — only Python knows what `PyFloat_Check` means.
- **Writing** one is the same shape everywhere, so it lives once in `rosetta::script::visit()`, and each caster supplies a sink of nine small methods (`on_none`, `on_bool`, `on_int`, `on_real`, `on_string`, `on_enum`, `on_list`, `on_object`, `on_unknown`).

No two casters can drift apart on whether an enum crosses as its integer, or on what an unclaimed type renders as.

The casters reach the generated code through `module_init`'s `headers`, which is the only hook that emits an `#include` *after* the framework header and *before* the bound classes — exactly the window a type caster needs. Because that list is global rather than per-target, each file guards its whole body on a macro only its own target defines.

WATCH OUT

One subtlety the type gate forces: `Value` must stay in the bound `classes` list. Bindability is decided at generation time and cannot know a caster will exist, so dropping `Value` would silently skip every member that mentions it — `Outcome::value`, `Instance::set`, `Instance::call`, all of it. The cost is one vestigial `Value` type per module that nothing ever produces.

15.4 What a project has to write

Nothing, beyond where its own files are. The binding surface is fixed, so it ships as manifest data in `include/rosetta/presets/scriptable.json` and a manifest names it:

```
{
    "preset": "scriptable",
    "rosetta_include": "../../include",
    "user_include": ["../lib", "../lib/bindings/dynamic"],
    "user_sources": ["../lib/bindings/dynamic/auto_dynamic.cpp"],
    "module_init": { "headers": ["auto_dynamic.h"],
                     "statements": ["bank::register_all()"] },
    "targets": ["python", "lua", "node"]
}
```

Six keys, every one about *this* project. Read them against the diagram: `user_sources` is stage 1's output, and `module_init` is the `register_all()` that publishes it.

15.5 What it buys

- **One wrapper per language instead of one per class.** Ship the module once; any library that has been through the `dynamic` backend becomes scriptable with no further code generation.
- **UI by query, not by codegen.** A property sheet is a walk over `fields()`: `doc` to tooltip, `range` to slider bounds, `combobox` and enumerators to a drop-down, `readonly` to a disabled row. The dispatch that a C++ property panel spends a file on is about ten lines of Python — editable without recompiling anything.
- **Overloads come back** for the name-keyed targets. Lua, Node, Wasm, C# and Java bind the first overload of a name and drop the siblings at generation time; through this module they do not, because `resolve()` scores the whole set at call time and can explain a failure.
- **Annotations are enforced once**, in the core, rather than re-implemented per backend — a `range` violation comes back as the same message in every language.

That third point is worth dwelling on, because it is the sharpest evidence that the IR is genuinely target-neutral. The same reflected overload set survives into both projections; the expressiveness difference between them is attributable entirely to *when* names are resolved, not to information lost on the way.

The set of C++ members a target can express is not a property of the target language. It is a property of when the target resolves names.

15.6 Running the pipeline twice

`examples/plain-binding` runs it **once**: one library, one manifest, four language bindings. That is the base case, and it is what most of this book describes.

`examples/scriptable-model` runs stages 1–2 once per *manifest*, and it has two:

1. **The user library to metadata.** The `dynamic` backend turns `scene.h` into `auto_dynamic.{h,cpp}` — the same information the other backends spend on framework calls, emitted as plain data instead.
2. **`<rosetta/script.h>` to bindings.** A second pipeline that binds rosetta's own reflection API, taking stage 1's output as its `user_sources`.

Practically: two manifests, two `rosetta_gen -build` invocations, in that order. Stage 2 must be re-run when stage 1's *output location* changes — but not when your library's *classes* change, which is the whole point.

Chapter 16

Driving a library from an AI agent

The `dynamic` backend (Chapter 14) lets a program ask what a library contains and call it by name. An AI agent is that kind of program. The `mcp` target serves your library over the **Model Context Protocol**, the open protocol AI assistants use to call external tools. With it, an agent such as Claude can explore your classes, create objects, set their fields and call their methods, and read each result before deciding what to do next:

> *"Load the bunny, relax it until its worst triangle is above 0.1, and tell me how many passes that took."*

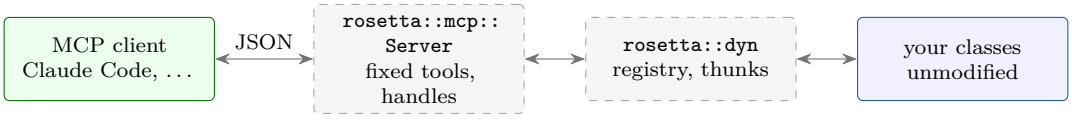
Your headers do not change. The agent reads the documentation rosetta already carries — `doc` annotations, `///` comments, parameter names (Chapter 11) — and stays inside the constraints rosetta already enforces: `range`, `readonly`, overload resolution.

16.1 What an agent needs, and where it comes from

An MCP server lists *tools*, each with a name, a description and a JSON Schema for its arguments; the client calls them and gets JSON back. Every piece of that already exists in rosetta:

The agent needs	Rosetta already has
what exists	the registry: classes, enums, free functions
what each member means	`doc`, harvested comments, parameter names
argument types	the `TypeDesc` of every field and parameter
limits	`range` and `readonly`, enforced by `Object::set`
calling by name	`resolve()` and the per-member thunks

So the `mcp` backend adds no new marshalling layer. It emits the same metadata tables as `dynamic`, plus a ten-line `main()`; everything protocol-side lives in one reflection-free header, `<rosetta/runtime/mcp.h>`, which knows nothing about your library.



16.2 A first server

Add the target next to the others:

```
"targets": [
  { "lang": "dynamic", "name": "scene" },
  { "lang": "mcp",     "name": "scene_mcp" }
]
```

Generate as usual (Chapter 12); the tree lands in `bindings/mcp/`. Like `dynamic`, it builds with an off-the-shelf C++20 compiler — the metadata is data, not splices. The only extra dependency is nlohmann/json, used if installed and fetched otherwise.

```
cmake -S bindings/mcp -B bindings/mcp/build
cmake --build bindings/mcp/build
claude mcp add scene -- "$PWD/bindings/mcp/build/scene_mcp"
```

The last line registers the server with Claude Code; any MCP client takes the same thing in its JSON configuration. The client launches the executable and talks to it over stdin/stdout. Start Claude Code, check with `/mcp` that the server is connected, and ask.

NOTE

`examples/dynamic` has the target already: `./run.sh mcp` builds the server and prints the `claude mcp add` line.

16.3 From your header to what the agent reads

What an agent can do with your library depends on what it can *read* about it. With the `mcp` target, everything you already wrote for a human — doc comments, annotations, parameter names — is what the agent reads. A small, complete example shows the path end to end.

The library

A pendulum, documented the two ways rosetta reads: Doxygen comments, and annotations. `length` and `angle` carry *inline* annotations; `damping` and `time` are plain C++ and get theirs from a *side-car* file (Chapter 10):

```
#include <rosetta/annotations.h>
#include <cmath>

namespace lab {

    /// A simple pendulum, integrated with a fixed time step.
    class Pendulum {
    public:
        [[= rosetta::doc{"Length of the rod, in metres"}, = rosetta::range{0.1, 10.0}]]
        double length = 1.0;

        [[= rosetta::doc{"Current angle from the vertical, in degrees"},
          = rosetta::range{-90.0, 90.0}]]
        double angle = 30.0;

        double damping = 0.05; // annotated in Pendulum.ann.json
        double time    = 0.0;  // annotated in Pendulum.ann.json

        /**
         * @brief Advance the simulation.
         * @param seconds how long to simulate
```

```
         * @param dt      the time step, in seconds
         * @return the angle at the end, in degrees
         */
        double simulate(double seconds, double dt);

        /// Small-angle period, 2 pi sqrt(L/g), in seconds.
        double period() const;
    };
}
```

```
// Pendulum.ann.json -- the side-car: same vocabulary, outside the header
{
  "damping": { "doc": "Friction coefficient; 0 swings forever", "range": [0, 1] },
  "time":    { "doc": "Simulated time so far, in seconds", "readonly": true }
}
```

```
"targets": [ { "lang": "mcp", "name": "lab_mcp" } ],
"classes": [ { "name": "lab::Pendulum", "header": "pendulum.h",
               "annotations": "Pendulum.ann.json" } ]
```

What the agent reads

The agent calls `describe_class` on `Pendulum` and gets, verbatim (trimmed to two fields and one method):

```
{
  "name": "Pendulum",
  "doc": "A simple pendulum, integrated with a fixed time step.",
  "fields": [
    { "name": "length", "type": "double", "kind": "number",
      "doc": "Length of the rod, in metres", "range": [0.1, 10.0] },
    { "name": "time", "type": "double", "kind": "number",
      "doc": "Simulated time so far, in seconds", "readonly": true }
  ],
  "methods": [
    { "name": "simulate",
      "signature": "simulate(double seconds, double dt) -> double",
      "doc": "Advance the simulation.\n\nArgs:\n    seconds: how long to simulate\n    dt:
    the time step, in seconds\n\nReturns:\n    the angle at the end, in degrees",
      "params": [ { "name": "seconds", "type": "double", "kind": "number" },
                  { "name": "dt",      "type": "double", "kind": "number" } ] }
  ]
}
```

Every line of it traces back to something in the header or the side-car:

You wrote	The agent reads	So it can
`///` above the class	the class `doc` in `list_classes` and `describe_class`	pick the right class for the request
`doc{...}`, inline or side-car	a field's `doc`	know what `damping` means, and its unit
`range{lo, hi}`, inline or side-car	`"range": [lo, hi]`	choose valid values — and is *stopped* if it does not
`readonly`, inline or side-car	`"readonly": true`	treat the field as an output, never set it
`@brief`, `@param`, `@return`	the method `doc`, with `Args:` and `Returns:` sections	call it with the right arguments, and interpret the result
the parameter names	`params[].name`, the signature	pass arguments by name: `{"seconds": 1.4, "dt": 0.001}`
the C++ types	`type` and `kind`	send a number, a string, a handle, an enumerator name
`combobox{...}`	`"choices": [...]`	offer one of them (listed, not enforced)

And what happens when it gets something wrong

The same annotations are *enforced* at call time, by the core — not by the agent's good will. From a session against this server:

```
set_fields {"object": "p", "values": {"length": 25, "time": 3}}
  -> error: updated []; rejected:
       length = 25 is outside [0.1, 10]
       Pendulum::time is read-only

set_fields {"object": "p", "values": {"length": 2.0, "damping": 0}}
  -> {"updated": ["damping", "length"]}

call_method {"target": "p", "method": "period"}
  -> {"result": 2.837006706885772}

call_method {"target": "p", "method": "simulate",
             "args": {"seconds": 1.418, "dt": 0.001}}
  -> {"result": -29.960569829913847}
```

The `range` that the agent read in `describe_class` is the one that rejects `length = 25`; the side-car's `readonly` is the one that protects `time`. Having read the documentation of `period()`, the agent can sanity-check the model: simulated for half a period with no damping, the pendulum ends at $-30°$, the mirror of its starting angle.

NOTE

Inline or side-car? They produce the same metadata and can be mixed, as above. The difference is who has to compile your header. The `mcp` server — like `dynamic` — includes it, and inline annotations make it C++26 source, so a header annotated inline needs the C++26 toolchain to build the server too. With side-car annotations the header stays plain C++ and the server builds with an off-the-shelf compiler. That is why `examples/dynamic`

keeps every annotation in `*.ann.json` files — and it is the natural choice for a third-party library you do not want to touch.

WATCH OUT

Default arguments do not reach the dynamic tables: the agent must pass every parameter (`simulate(1)` is rejected as "takes 2 argument(s), 1 given"). Document the value you would have defaulted, so the agent knows what to send.

16.4 What a conversation looks like

The point of all this is not that an agent can call `set_fields`. It is that you can ask a *question about your domain* and the agent works out which calls answer it. The five conversations below were run against the pendulum server of Section 16.3, with Claude Opus 5.5 through Claude Code, connected to that server and nothing else (no shell, no files). Replies are abridged, not rewritten; the agent's scripts are excerpted, with some comments added. Every session started with the same two calls — `list_classes`, then `describe_class` — which is how the agent learned everything it knew about the library.

1. "What can this library do?"

> *What can this library do? Give me a one-paragraph tour.*
>
> This library has one class, `lab::Pendulum`, which simulates a simple pendulum using a fixed time step. You create one with no arguments and set three fields: `length` (0.1–10 m), `angle` (the starting angle from vertical, −90° to 90°) and `damping` (a friction coefficient from 0 to 1, where 0 means it swings forever). A read-only `time` field counts the simulated seconds so far. `simulate(seconds, dt)` runs the simulation forward and returns the final angle in degrees; `period()` returns the small-angle period, $2\pi\sqrt{L/g}$. [...]

Every fact in that paragraph is a `doc`, a `range`, a `readonly`, a parameter name or a `@return` from the header and the side-car. Nothing was guessed.

2. Asking for something the library refuses

> *Make a pendulum with a 15 metre rod and release it from 120 degrees.*
>
> I couldn't build that pendulum: both values are outside what the library allows. `Pendulum.length` only accepts 0.1–10 m [...] `Pendulum.angle` only accepts −90° to 90° [...] I haven't created any object. Which would you like? (1) a 10 m rod released from 90°; (2) keep the 15 m rod's swing shape at a length the library accepts — swing timing scales with $\sqrt{L}$, so multiply the times by $\sqrt{1.5} \approx 1.22$; (3) different values within the limits.

The agent never called `set_fields`: it read the two `range` annotations in `describe_class` and declined *before* touching the C++ side — then offered a physically sound workaround. Had it tried anyway, the core would have rejected the values (Section 16.3).

3. An inverse problem

> *I want a pendulum that ticks once per second, i.e. a period of exactly 2 seconds. What rod length do I need? Set it on a pendulum and verify it by simulation.*

The agent answered with *one* `run_script` call. It recovered g from `period()` itself rather than assuming it, set the length, then timed the swing from its zero crossings:

```
local p = new("Pendulum")
p.length = 1
local g = 4*math.pi^2 / p:period()^2   -- the library's own g
p.length = g / math.pi^2               -- T = 2 s
p.damping, p.angle = 0, 2              -- small swing, no friction
-- step with dt = 1e-4, record the upward zero crossings,
-- average the period over the crossings found in 20 s
...
-> {"L": 0.9939608115313356, "period_formula": 2,
    "T_sim": 2.0001523111193213, "n_periods": 9}
```

> You need a rod length of $L = gT^2/4\pi^2 = g/\pi^2 \approx 0.99396\,\text{m}$. I set `length` to that value: `period()` returns 2.000000 s, and a simulation timed over 9 full swings gives 2.00015 s. The extra 0.00015 s is a real effect of swinging at a non-zero angle, not a simulation error: at 2° the expected period is $T_0(1 + \theta^2/16) \approx 2.00015\,\text{s}$.

4. A numerical search

> *Starting from 30 degrees, what damping makes the swing amplitude drop to half after 20 seconds of simulation? Find it numerically.*

Again one script: a function measuring the amplitude over the last period before $t = 20\,\text{s}$, then 40 steps of bisection on `damping`, run twice with different time steps to check convergence (3.6 s inside the server):

```
local function amp(c, dt) ... end       -- largest |angle| in the last period
local lo, hi = 0, 1
for i = 1, 40 do
  local mid = (lo + hi) / 2
  if amp(mid, 0.001) > 15 then lo = mid else hi = mid end
end
-> c(dt=1e-3) = 0.075475,  c(dt=2e-4) = 0.075476
```

> The damping that halves the swing amplitude after 20 s is $c \approx 0.0755$. With no damping the amplitude stays at 30.00°, so the integrator isn't adding or losing energy on its own, and the result doesn't depend on the time step. This is a bit higher than $2\ln 2/20 \approx 0.0693$, the textbook value for linear damping. **The library doesn't document its damping law**, so this difference may come from how it applies friction or from the swing being large.

That last sentence is worth noticing: the agent is exactly as precise as your documentation. The header says what `damping` is for, not how it enters the equation of motion, and the agent says so instead of pretending. One more line in the `doc` — "$\ddot\theta = -(g/L)\sin\theta - c\,\dot\theta$" — and it could have reasoned about the difference too.

5. Testing the model's own assumptions

> *Is the small-angle period formula still accurate if I release the pendulum from 80 degrees? Measure the real period by simulation and compare.*

The agent timed the period at 5° and 80°, each at two time steps, in one script, and compared it with `period()`:

Release angle	Simulated period	`period()`, $2\pi\sqrt{L/g}$	Error
5°	2.0070 s	2.0061 s	0.05 %
80°	2.2819 s	2.0061 s	**13.75 %**

> No, not at 80°: the small-angle formula is about 14 % too short. The real period is 1.1375 times the formula's value, which matches the exact large-amplitude result $T = T_0\,(2/\pi)\,K(\sin(\theta_0/2))$. The formula assumes $\sin\theta \approx \theta$; at large angles the restoring force is weaker, so the pendulum swings more slowly. Rule of thumb: the error is about $\theta_0^2/16$, which stays under 1 % up to about 23°.

The library provided the two primitives, `simulate` and `period`; the agent combined them into an experiment, checked convergence, and connected the numbers to the physics — including where the library's own `period()` stops being valid.

What made these possible

- **The documentation was the interface.** Units, ranges, what each parameter means, what a method returns: the agent used all of it, and named the one thing that was missing.
- **Scripts turned questions into one call.** Sessions 3–5 each took four turns in total, the work being a single `run_script`; the same experiments as individual tool calls would have needed hundreds of round trips.
- **The C++ did the computing.** The agent wrote the experiment and read the result; every number came from your `simulate()`.

To reproduce them, point Claude Code at the server and ask:

```
claude mcp add lab -- "$PWD/lab_mcp"
claude
```

16.5 The tools

The agent always sees the same small set, whatever the library:

Tool	Does
`list_classes`	classes (with their static factories), enums, free functions — the agent starts here
`describe_class`	constructors; fields with type, doc, range, choices, enum values, read-only; methods with signature and doc. Inherited members included
`create_object`	best-matching constructor; returns a handle and the new object's fields
`call_method`	a method on a handle, or a static method when `target` is a class name; `store_as` names a returned object
`inspect_object`	every readable field at once
`set_fields`	several assignments in one call, each validated
`list_objects`, `release_object`	the handle table
`call_function`	free functions (listed only when some are bound)
`run_script`, `run_python`	a script run next to the objects — Section 16.7

Why not one tool per method

Generating `Mesh_subdivide`, `Relaxer_run`, … would look more direct, and scales badly: agents pick tools less reliably as the list grows, every schema costs context on every turn, and a 500-class library would produce thousands. With a fixed set, discovery is `describe_class`, paid only for the classes the agent actually touches — and the tool list never changes when the library grows.

Handles

Tool calls are stateless; the server process is not. Live objects sit in a handle table, under `<Class>#<n>` or a name the agent chooses. A handle goes back in as `{"$ref": "bunny"}`, or as the bare string `"bunny"` wherever the parameter's type is known to be a class:

```
call_method   {"target": "Mesh", "method": "bunny", "store_as": "bunny"}
  -> {"result": {"$ref": "bunny", "class": "scene::Mesh"}}
call_method   {"target": "Relaxer", "method": "minQuality", "args": ["bunny"]}
  -> {"result": 0.014254101544304952}
```

A `T&` return comes back as a handle that *pins* its parent, exactly as in Chapter 14: release the mesh, and a handle to its `origin` stays valid. Each handle reports whether it is `owned`, `pinned` or `borrowed`.

Arguments

JSON has no enums, no classes and no int/double distinction, so arguments are converted against the parameter type wherever it can be known. Arguments may be positional (`"args": [1, 2]`) or by parameter name (`"args": {"m": "bunny"}`) — the names are the ones reflected from your declarations. Enum values are passed and returned by enumerator name.

Bulk data is summarised

A vector longer than 64 elements does not come back whole. An agent needs to know a mesh has 35 947 vertices and roughly where they are, not to read 107 841 doubles into its context:

```
call_method {"target": "bunny", "method": "positions"}
  -> {"result": {"$summary": true, "length": 107841,
                 "head": [-0.134, 0.114, 0.037, ...],
                 "min": -0.5002, "max": 0.5002, "mean": -0.0308}}
```

A script, on the other hand, receives vectors whole (Section 16.7).

16.6 Constraints the agent cannot bypass

Everything the core enforces stays enforced, and a rejection comes back as a *tool error* with the core's own message — which is what lets the agent correct itself on the next call. From a real session on the `scene` library:

```
set_fields {"object": "bunny",
            "values": {"opacity": 7, "shading": "Wireframe", "id": "x"}}
  -> error: updated ["shading"]; rejected:
       Mesh::id is read-only
       opacity = 7 is outside [0, 1]

call_method {"target": "bunny", "method": "at", "args": ["nope"]}
  -> error: Mesh::at(std::string) matched no overload of 2:
       at(int) -- argument 1 ("nope") is not a int
       at(int, int) -- takes 2 argument(s), 1 given

call_method {"target": "bunny", "method": "onProgress", "args": [1]}
  -> error: ... onProgress(std::function<void(double)>) -- not callable:
       callback: a std::function parameter needs a foreign-callable adapter
```

The valid assignment in the first call (`shading`) still lands; only the two refused ones are reported.

WATCH OUT

There is no authorisation model beyond your annotations: anything bound can be called. Bind only what an agent should be able to reach, and use `readonly` and `range` for what it may read but not freely change.

16.7 Scripts: one call instead of dozens

Every tool call is a round trip through the model. "Relax until the worst triangle passes 0.1" as individual calls — run, measure, decide, repeat — takes dozens of them, each one re-reading the conversation. As a script it is one:

```
-- run_script: Lua, next to the live objects
local m = obj("bunny")
local r = new("Relaxer")
r.iterations, r.step = 1, 0.7
local passes = 0
while classes.Relaxer.minQuality(m) <= 0.1 do
  r:run(m)
  passes = passes + r.lastPasses
end
return {passes = passes, worst = classes.Relaxer.minQuality(m)}
```

On the Stanford bunny this returns `{passes = 1, worst = 0.1534}`: the worst triangle went from 0.0143 to 0.1534 in a single tool call.

Both scripting tools see the same small API, which their tool descriptions spell out for the agent:

	Lua (`run_script`)	**Python** (`run_python`)
fetch a handle	`obj("bunny")`	`obj("bunny")`
construct	`new("Relaxer")`, `classes.Relaxer()`	same
field	`o.step = 0.7`	same
method	`o:run(m)`	`o.run(m)`, keyword arguments too
static method	`classes.Relaxer.minQuality(m)`	same
publish a handle	`keep("name", o)`	same
result	the `return` value	the last expression, as in a notebook

They go through the same core as the JSON tools — a range violation is a Lua error (catchable with `pcall`) or a Python `RuntimeError` — and share the handle table, so an object made in a script is a handle for the next tool call. Unlike the JSON tools, scripts get vectors *whole*: running next to the data is the point. Only what a script returns is summarised, and `print()` is captured.

```
# run_python: the environment comes with it
import numpy as np
a = np.array(obj("bunny").positions()).reshape(-1, 3)
{"bbox_min": a.min(0), "bbox_max": a.max(0)}
```

Lua is sandboxed; Python is not

	`run_script` (Lua)	`run_python`
built when	Lua 5.3+ is found (default)	`-DROSETTA_MCP_PYTHON=ON`
reaches	the bound library only: no `io`, `os`, `require`, `load`	everything the server process can
limits	wall clock (default 30 s), Lua heap (512 MB)	wall clock; raises `TimeoutError`
brings	nothing else	numpy and the whole environment

WATCH OUT

CPython cannot be meaningfully sandboxed from inside. With `run_python` enabled, an agent's script runs with the server's own permissions — it can read your files and start processes. That is why it is off by default, and why its tool description says so. Enable it only for an agent you would let run Python on that machine anyway.

Neither language can interrupt a long call into C++: that is your library's own code, and it finishes.

16.8 Inside a running application

The stdio server is a process the client launches, so it owns its own objects: nothing it does shows up in an application that is already open. The other way round is often what you want — the agent working on the objects the application is *showing*.

`<rosetta/runtime/mcp_qt_http.h>` does that for a Qt application. It serves the same `Server` over HTTP on `127.0.0.1`, on the GUI thread, so a tool call runs between two frames and needs no locking against the code that draws. The Qt viewer of `examples/dynamic` starts it at launch:

```
./run.sh viewer
#   console: MCP server for this scene: http://127.0.0.1:8770/mcp
claude mcp add --transport http scene-viewer http://127.0.0.1:8770/mcp
```

Ask Claude Code to "subdivide the sphere and make it glossy", and the sphere goes from 768 to 3 072 triangles in the 3-D view while you watch; the viewer's console logs each outside call (`[mcp] call_method ...`).

NOTE

The host binds to the loopback interface only and refuses requests whose `Origin` is not localhost, so a web page cannot drive it. There is no authentication: any local process can connect, as any local process could run the stdio binary. `ROSETTA_MCP_PORT` changes the port; `0` turns the host off.

The viewer's own Claude tab

The viewer also has a **Claude** tab next to its console, with two backends:

- **Claude Code (your login)**, the default when `claude` is installed. Like the VS Code extension, the tab runs the Claude Code CLI for each message — with `ANTHROPIC_API_KEY`

removed, so your Claude Code subscription is used — connected to the viewer's own MCP server and to nothing else: no shell, no file access.

- **Anthropic API**, which calls the Messages API directly and needs an `ANTHROPIC_API_KEY` with credits.

Either way the transcript shows Claude's replies, each tool call (scripts as code) and its result, and the scene updates after every call. Figure 16.1 shows it.

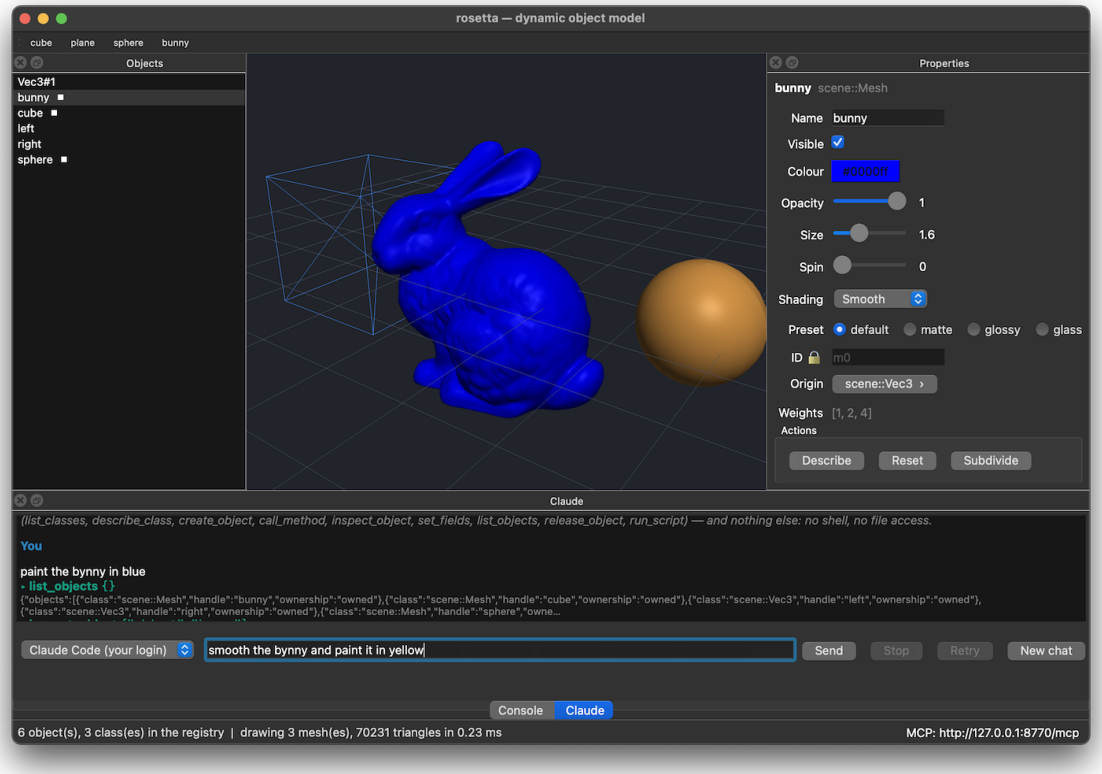


Figure 16.1: The Claude tab of the `examples/dynamic` viewer, on the Claude Code backend. Asked to "paint the bunny in blue", Claude listed the objects (`list_objects`) and set the bunny's `colour`; the 3-D view and the property panel show the result, and the next request is being typed. The status bar shows the viewer's own MCP endpoint.

16.9 Embedding it in your own application

All of the above is a few lines over the runtime headers. This is the shape of the viewer's own wiring:

```
#include <rosetta/runtime/mcp.h>
#include <rosetta/runtime/mcp_lua.h>      // run_script
#include <rosetta/runtime/mcp_qt_http.h>  // QtHttpHost

scene::register_all();                    // your generated metadata

rosetta::mcp::Server server({.name = "my-app"});
rosetta::mcp::enable_lua(server);         // optional

// Publish objects the application already owns, under the names it uses.
const auto *meshClass = rosetta::dyn::registry().find_class("scene::Mesh");
```

```
server.put("bunny", rosetta::dyn::Object::borrow(*meshClass, &myMesh));

rosetta::mcp::QtHttpHost host(server);
host.set_handler([&](const rosetta::mcp::json &msg) {
    auto reply = server.handle(msg);      // one JSON-RPC message
    if (msg.value("method", "") == "tools/call")
        view->update();                   // the agent changed something
    return reply;
});
host.listen(8770);
```

The other entry points, for a non-Qt host or a test:

- `server.serve_stdio()` — the generated `main()`; diverts `std::cout` to stderr so a chatty library cannot corrupt the protocol.
- `server.handle(json)` — one message in, one reply out, no I/O: the whole protocol is testable without a pipe.
- `server.add_tool(definition, fn)` — your own tools next to the generic ones. `run_script` and `run_python` are added exactly this way.

16.10 Limits

- **The agent orchestrates; it does not compute.** Loops belong in a script, heavy numerical work in your C++ methods.
- **No callbacks.** `std::function` parameters are described but not callable, as in every dynamic consumer.
- **No persistence.** Handles live as long as the server process — the client session for stdio, the application for the Qt host.
- **Positional enum names across disagreeing overloads** stay strings; pass the integer value or use named arguments.

The design note `docs/MCP.md` has the reasoning behind each decision and the list of next steps.

Chapter 17

Writing a backend

Rosetta ships nineteen backends, but the generator is **open for extension**: you can teach it a new target without editing `rosetta::generate` or any core file. This chapter adds a Lua backend as a worked example — Lua because it is small enough to show whole, and because the real one exists to compare against.

17.1 The mental model: two layers

A backend is two independent pieces that run at two different times, and keeping them straight is most of the work.

Layer	File you write	Runs when	Reflection?
Runtime visitor	`rosetta/visitors/lua.h`	when the *generated* binding is compiled	yes
Generation backend	a `rosetta::Backend` subclass	when the driver runs	no

The split exists because `rosetta::generate<Ts...>` **erases the type pack up front** into plain strings:

```
struct GenClass { std::string name; std::string header; std::string doc; };
```

So a `Backend` never sees a C++ type — only `{name, header, doc}` per class. All the real reflection happens later, in your *visitor*, when the generated `auto_lua.cpp` is compiled.

```
manifest.json -(rosetta_gen)-> gen/ -(build)-> ./generator -(run)-> bindings/lua/
                                                    |                  auto_lua.cpp
                                                    |                  CMakeLists.txt
                                          LuaBackend::emit()    (compiling auto_lua.cpp
                                                                 runs the walk)
```

17.2 Layer 1: the runtime visitor

This is where binding actually happens. It implements the **walk visitor concept**: three required member templates plus one optional, each receiving a `std::meta::info` of the member and its annotation pack.

```
// include/rosetta/visitors/lua.h
#pragma once
```

```cpp
#include <experimental/meta> // C++ P2995
#include <rosetta/walk.h>    // Rosetta
#include <sol/sol.hpp>       // Lua
#include <string>            // C++ std

namespace rosetta {

    template <typename T> struct LuaVisitor {
        sol::usertype<T> &ut;

        // A data member. `Fld` is the reflected field; `Anns...` is its
        // annotation pack (rosetta::doc / readonly / range / ...).
        template <std::meta::info Fld, auto... Anns> void field(const char *name) {
            using F = [:std::meta::type_of(Fld):];
            if constexpr (ann::has<readonly>(Anns...)) {
                ut[name] = sol::readonly_property(
                    [](const T &s) -> F { return s.[:Fld:]; });
            } else {
                ut[name] = sol::property(
                    [](const T &s) -> F { return s.[:Fld:]; },
                    [](T &s, F v) { s.[:Fld:] = v; });
            }
        }

        template <std::meta::info Fn, auto... Anns> void method_instance(const char *name) {
            ut[name] = &[:Fn:];
        }

        template <std::meta::info Fn, auto... Anns> void method_static(const char *name) {
            ut[name] = &[:Fn:];
        }

        // OPTIONAL: constructors. Omit the method entirely and the walk skips
        // it -- it is detected with `requires`.
        // template <std::meta::info Ctor, auto... Anns> void constructor() { ... }
    };

    // Entry point the generated code calls, one per class.
    template <typename T> void bind_lua(sol::state &lua, const char *name) {
        sol::usertype<T> ut = lua.new_usertype<T>(name);
        LuaVisitor<T> v{ut};
        walk<T>(v);
    }

} // namespace rosetta
```

The annotation helpers you will use inside a visitor:

- `rosetta::ann::has<readonly>(Anns...)` — is annotation `A` present?
- `rosetta::ann::get_or<doc>(doc{""}, Anns...)` — fetch annotation `A`, or a fallback.

And the reflection splices: `[:std::meta::type_of(Fld):]` for a field's type, `obj.[:Fld:]` to access it, `&[:Fn:]` for a member-function pointer.

NOTE

If your target cannot marshal some parameter or return type, mirror the `napi_supported` / `rest_supported` pattern — a `consteval` predicate plus an `if constexpr` guard in `field` and

`method_*` — so unbindable members are *skipped* instead of failing the build. And record them: see Section 13's instrumentation recipe, which is one line per gate.

17.3 Layer 2: the generation backend

This scaffolds the per-target project. It implements:

```
struct Backend {
    virtual ~Backend() = default;
    virtual void emit(const GenContext &) const = 0;
};
```

with everything it needs handed over in `GenContext`:

`out_dir`	root of the generated tree (`bindings/`)
`lib`	this target's module/library name, from the manifest entry
`classes`	`std::vector<GenClass>` — `{name, header, doc}` per class
`user_include`	absolute directory holding the class headers
`rosetta_include`	absolute path to rosetta's `include/`

A minimal backend writes one `.cpp` that includes the visitor and calls `bind_lua<T>` per class, plus a `CMakeLists.txt`:

```
#include <rosetta/generate.h>
#include <filesystem>
#include <fstream>

struct LuaBackend : rosetta::Backend {
    static void write(const std::filesystem::path &p, const std::string &s) {
        std::filesystem::create_directories(p.parent_path());
        std::ofstream(p) << s;
    }

    void emit(const rosetta::GenContext &c) const override {
        std::string includes, binds;
        for (const auto &k : c.classes) {
            includes += "#include \"" + k.header + "\"\n";
            using rosetta::exposed_of;
            using rosetta::qualified_of;
            binds += "    rosetta::bind_lua<" + qualified_of(k) +
                     ">(lua, \"" + exposed_of(k) + "\");\n";
        }

        const auto dir = c.out_dir / "lua";
        write(dir / "auto_lua.cpp",
              "// Generated by rosetta -- do not edit by hand.\n" + includes +
              "#include <rosetta/visitors/lua.h>\n"
              "#include <sol/sol.hpp>\n\n"
              "extern \"C\" int luaopen_" + c.lib + "(lua_State *L) {\n"
              "    sol::state_view lua(L);\n" + binds +
              "    return 0;\n}\n");

        write(dir / "CMakeLists.txt", /* ... mirroring bindings/python's ... */);
    }
};
```

WATCH OUT

Two names, never interchangeable. `qualified_of(k)` gives the **C++** spelling — `arch::sinv::Data` — which is what anything the generated translation unit compiles must use. `exposed_of(k)` gives the **host-language** name: the manifest's `expose` override, else the identifier. Getting these the wrong way round is how two same-named bound types become ambiguous in the generated source, and how a rename silently fails to take effect.

17.4 Registering it

`rosetta::generate` dispatches each target through a runtime registry keyed by the manifest's `lang` string:

```
std::map<std::string, std::shared_ptr<Backend>> &backend_registry();
void register_backend(std::string lang, std::shared_ptr<Backend> backend);
```

Register at static-init time with the `BackendRegistrar` helper, in one self-contained plugin translation unit:

```
// lua_plugin.cpp -- your plugin, owned entirely by you
#include "LuaBackend.h"

static const rosetta::BackendRegistrar
    lua_reg{"lua", std::make_shared<LuaBackend>()};
```

Because the registrar runs before `main`, the `"lua"` backend is present by the time the driver calls `generate` — **no edit to `generate` required**. An unregistered target is warned about and skipped.

NOTE

Your backend does *not* belong in `rosetta::backend`. That namespace is where rosetta's own implementations live, not a registration point. Put yours in your own namespace; what you inherit from is `rosetta::Backend`, and what you call is the public API below.

17.5 Wiring it into a project

Point the manifest at your plugin with `plugins`, and add a target. `rosetta_gen` compiles each listed source into the generated driver's `add_executable`, resolving paths relative to the manifest.

```
{
  "user_include": "./geom",
  "rosetta_include": "../../include",
  "plugins": ["lua_plugin.cpp"],
  "targets": [
    { "lang": "python", "name": "pygeom" },
    { "lang": "lua",    "name": "mygeom" }
  ],
  "classes": [
    { "name": "Model", "header": "Model.h" },
    { "header": "Point.h" }
  ]
```

```
}
```

Then the usual flow:

```
../../bin/rosetta_gen manifest.json gen   # gen/ + your plugin in its CMakeLists
cmake -S gen -B gen/build && cmake --build gen/build
./generator bindings                      # emits bindings/python AND bindings/lua
```

17.6 API reference

Generation layer (<rosetta/generate.h>)

```
struct GenClass { std::string name; std::string header; std::string doc; };

struct GenContext {
    std::filesystem::path out_dir;
    std::string           lib;
    std::vector<GenClass> classes;
    std::string           user_include, rosetta_include;
};

struct Backend {
    virtual void emit(const GenContext &) const = 0;
};

void register_backend(std::string lang, std::shared_ptr<Backend>);

struct BackendRegistrar {
    BackendRegistrar(std::string lang, std::shared_ptr<Backend>);
};

std::map<std::string, std::shared_ptr<Backend>> &backend_registry();
```

IR accessors — how a backend spells a bound type

- `qualified_of(const GenClass&)` / `(const GenEnum&)` — the **C++** spelling, for anything the generated TU compiles.
- `exposed_of(const GenClass&)` / `(const GenEnum&)` — the **host-language** name.
- `exposed_object_of(const GenType&, const GenContext&)` — the exposed name of the type an IR entry *references*.
- `names_type(const GenType&, const K&)` — does this IR type name that bound class or enum?

> **NOTE**
>
> These six lived in `rosetta::gen_detail` before 2026-08. That spelling still compiles — the old names are `using`-declarations onto these — but it was a "detail" namespace documented as an extension point, so they now sit in `rosetta::` with the rest of the backend API.

Runtime layer (<rosetta/walk.h>) — the visitor concept

- `v.template field<Fld, Anns...>(const char *name);`
- `v.template method_instance<Fn, Anns...>(const char *name);`

- `v.template method_static<Fn, Anns...>(const char *name);`
- `v.template constructor<Ctor, Anns...>();` — **optional**, called only if defined.
- `rosetta::ann::has<A>(Anns...)` / `rosetta::ann::get_or<A>(fallback, Anns...)`

17.7 Where to copy from

The built-in backends are a declaration header in `include/rosetta/backends/` with its implementation alongside in `include/rosetta/backends/inline/`. They live in `rosetta::backend` — `backend::Python`, `backend::Node` — so the directory, the file and the namespace each say "backend" once and the class says only which one. They are the reference for the CMake flags and the clang-p2996 link line.

The reflection-free header-only runtimes those backends emit `#include` lines for — the N-API marshaller, the C#/Java C-ABI shims, the Qt and ImGui widget kits — live under `include/rosetta/runtime/`. They are compiled by the *generated* project, never by the generator, which is why they are a directory of their own.

The existing visitors in `include/rosetta/visitors/` are the templates to copy for layer 1.

Part VI

Worked examples

Chapter 18

Worked examples

The repository carries eighteen runnable examples. This chapter is a guide to which one answers which question, followed by three walked through in enough detail to copy from.

18.1 The map

Path	What it shows
`examples/manifest`	Manifest-driven generation for `Person`, with no class modification. The smallest complete case.
`examples/annotate-manifest`	Out-of-line annotations from an external JSON file, wired by the manifest's `annotations` field.
`examples/doxygen`	The documentation a library already has: plain Doxygen comments and default arguments in an untouched header become Python docstrings and keyword arguments, TSDoc, OpenAPI descriptions and a Markdown reference. Ch. 11
`examples/geom-lib`	Manifest-driven bindings for a small geometry library — nested types, vectors.
`examples/geom-expanded`	Ten backends (`python`, `nanobind`, `node`, `wasm`, `qt`, `qml`, `csharp`, `java`, `lua`, `julia`) built with off-the-shelf toolchains, plus out-of-line annotations. The proof of Section 1.6.
`examples/dynamic`	The `dynamic` backend end to end: one set of generated metadata driving a terminal interpreter *and* a Qt viewer (3-D view, property panel, console), neither naming a bound type. Also the `mcp` target: a stdio server, an MCP host inside the viewer, and a Claude tab (Ch. 16).
`examples/scriptable-model`	The two-stage pipeline of Chapter 15.
`examples/plain-binding`	One library, one manifest, four language bindings — the base case, run once.
`examples/trampoline-python`	Overriding C++ virtuals from Python: generated pybind11 trampolines from `virtual_spec`.
`examples/trampoline-node`	The same from JavaScript, with N-API trampolines.
`examples/dynamic-lib`	Linking the bindings against a pre-built library — the `user_lib` case.

Path	What it shows
`examples/moc`	The moc-less signals/slots/properties demo built on `mini_moc.h`.
`examples/docgen`	A reflection-driven Markdown / HTML reference generator.
`examples/paraview`	ParaView plugin property-panel XML from an annotated `vtkThreshold` spec — exercises every annotation feature.
`examples/qt`, `examples/qml`, `examples/imgui`	The three inspector backends, each building a form from a reflected struct. `imgui` uses an `Algo.ann.json` side-car and builds with an off-the-shelf C++20 compiler, dependencies auto-fetched.
`examples/bindings/*`	Hand-written reference backends — pybind11, N-API, jlcxx, REST, embind. What the generator emits, written by hand, for comparison.

18.2 The smallest case

`examples/manifest` is the one to read first. An unmodified `person.h`, a manifest of ten lines, and the pipeline. It is Chapter 3 as a runnable directory.

18.3 Documentation, without writing any

`examples/doxygen` is chapter 11 as a runnable project: a documented `polygon.h` that never mentions rosetta, a manifest that says nothing about documentation, and four targets showing where the comments land.

Read its `test.py` rather than its README. It asserts, against the compiled module, not only that the docstrings and keyword arguments arrived but also the three cases where rosetta deliberately does *less*: a default it refuses to repeat (an unqualified class constant), an enum default that works only because the enum is bound too, and two `scale` overloads that each keep their own comment while the `.d.ts` declares only the first.

18.4 A geometry library

`examples/geom-lib` is the first realistic shape: several classes, one returning a `std::vector` of another, an enum, and a free function.

The two things to look at:

- **Nested user types cross without being mentioned.** A `Surface` method returning `std::vector<Triangle>` works because `Triangle` is in `classes` — not because anything says how to convert it.
- **One module per target, not one per class.** The generated `pygeom` carries `Model`, `Point`, `Surface`, `Triangle` and `Kind` together.

`examples/geom-expanded` takes the same library and builds ten backends with stock toolchains — no fork anywhere in the second pass. If you want to convince yourself (or a colleague) that the reflection-free claim is real, that is the directory to run.

18.5 The dynamic model, end to end

`examples/dynamic` is the most interesting example in the repository and the one that takes longest to appreciate.

A small scene library — `Vec3`, `Mesh`, and a `Relaxer` solver — goes through the `dynamic` backend once. Two entirely separate consumers then drive it:

- a **terminal interpreter** that lists types, gets and sets fields, and calls methods by name;
- a **Qt viewer** with a 3-D view, a property panel and a console.

Neither names a single bound type. The property panel is a walk over `fields()`: `doc` becomes a tooltip, `range` becomes slider bounds, `combobox` becomes a drop-down.

What adding a class costs

The example's own history is the argument. Adding `scene::Relaxer` — tangential Laplacian relaxation, a genuine algorithm with tuning parameters and one method that runs for a while — touched three files, *all of them the library's own*:

1. `scene.h`, to add the class;
2. `Mesh.ann.json`, for one new accessor's annotation;
3. one line of `manifest.json`.

No consumer changed. Not the Qt viewer, not the interpreter, not any of the five scriptable drivers — yet all of them gained it: the solver appeared in the type list, its parameters appeared in the property panel with their declared ranges, and its method appeared as a button.

That is what "UI by query rather than by codegen" buys, stated as a diff.

The fourth consumer: an AI agent

The manifest also lists an `mcp` target, and the viewer hosts an MCP server for its own scene, so Claude can be the next consumer — from Claude Code in a terminal, or from the viewer's own Claude tab:

```
./run.sh viewer
claude mcp add --transport http scene-viewer http://127.0.0.1:8770/mcp
```

It needed no new metadata and no change to `scene.h`. Asked to "make the cube red and subdivide the sphere once", Claude made three tool calls (`set_fields`, `call_method subdivide`, `call_method triangleCount`) and the view changed under them; asked to relax the bunny, it sent one Lua script. Chapter 16 walks through it.

18.6 Rosetta in the wild

Real libraries bound with rosetta, each from a single manifest and with no hand-written wrappers:

Project	Bound library	Targets
`pmp-rosetta`	PMP — polygon mesh processing (remeshing, smoothing, subdivision, decimation)	Python, Node.js, WebAssembly, TypeScript
`geogram-rosetta`	geogram — geometry processing (reconstruction, remeshing, parameterisation, booleans)	Python, Node.js, WebAssembly, TypeScript, Lua
`arch-rosetta`	Arch — 3-D boundary-element geomechanics	Python, Node.js, WebAssembly, TypeScript
`cassini-rosetta`	Cassini — FEM geomechanical restoration, through a single high-level facade	Python, Node.js, WebAssembly, TypeScript
`triax-rosetta`	Triax — discrete-element triaxial compression	Python, Node.js, WebAssembly

The geogram manifest is the best worked reference for a difficult library: it uses `sequences` for `GEO::vector`, `extensions` to reach geometry behind raw-pointer accessors, `signature` to pick one overload of `mesh_union`, `module_init` for `GEO::initialize()`, `generated_headers` for the configured `version.h`, and `link_options` for the wasm NODEFS mount — in other words, nearly every field in Part II, each for a reason you can see.

18.7 Extending a generated binding by hand

Everything under `bindings/` is regenerated output and must never be edited. When the generated binding misses something you need — a helper the walker skips, a typed-array export for a renderer — add it in a **separate hand-written file** compiled *alongside* the generated source, from a small build of your own that lives outside `bindings/`.

The binding frameworks accept several registration blocks per module, so your file contributes a second one. Nothing generated is touched, and regenerating never clobbers your extensions.

```
// viz_helpers.cpp -- compiled next to the generated auto_emscripten.cpp
emscripten::val mesh_positions(const pmp::SurfaceMesh &mesh);  // Float32Array
emscripten::val mesh_triangles(const pmp::SurfaceMesh &mesh);  // Uint32Array
void triangulate_mesh(pmp::SurfaceMesh &m) { pmp::triangulate(m); }

EMSCRIPTEN_BINDINGS(pmp_viz) {   // a second block; the generated one keeps its own
    emscripten::function("mesh_positions", &mesh_positions);
    emscripten::function("mesh_triangles", &mesh_triangles);
    emscripten::function("triangulate_mesh", &triangulate_mesh);
}
```

WATCH OUT

Embind is the friendliest here because it accepts any number of `EMSCRIPTEN_BINDINGS` blocks in one module. Backends with a single module entry point — pybind11's `PYBIND11_MODULE`, N-API's `NODE_API_MODULE` — cannot add a second block to the *same* module. There, ship your extras as a small companion module built the same way (it sees the same user headers and links the same library) and import both.

Before reaching for this, check whether `extensions` (Section 5.5) covers your case: a free function taking `Cls&` needs no build of your own and reaches every backend at once.

Part VII

Reference

Appendix A

Manifest field reference

Every top-level field, alphabetically. • marks a required field.

Field	Req.	Default	Meaning
`build_type`		`Release`	Default `CMAKE_BUILD_TYPE` baked into every compiled backend's `CMakeLists.txt`. §7.3
`classes`	•*	—	The classes, structs and enums to bind. *Optional when a `preset` supplies them. Ch. 5
`compile_definitions`		`[]`	Preprocessor definitions (`"NAME"` or `"NAME=VALUE"`) applied to the generator driver *and* every compiled binding target. Ch. 7
`cpp26_cc`		`${cpp26_root}/bin/clang`	C compiler of the reflection toolchain.
`cpp26_cxx`		`${cpp26_root}/bin/clang++`	C++ compiler of the reflection toolchain.
`cpp26_lib`		`${cpp26_root}/lib`	Directory holding the fork's `libc++` / `libc++abi`.
`cpp26_root`		a built-in path	Root of the C++26 / P2996 toolchain. Moves the other three together. §4.5
`doc_comments`		`true`	Read the Doxygen documentation and default arguments already present in the bound headers, and carry them into every backend. `false` generates from annotations alone. Ch. 11
`functions`		`[]`	Free (non-member) functions to bind. §5.4
`generated_headers`		`[]`	Headers the bound library's own build system would have produced, written into the generated tree and put first on the include path. §7.4
`generator_name`		manifest's directory name	Basename of the emitted reflection driver; last-resort default for `module_name`.

Field	Req.	Default	Meaning
`header_dir`		—	Directory fragment prepended to every entry header. §4.7
`interop`		`[]`	Foreign libraries whose types the target's binding framework marshals itself (`["eigen"]`). §8.3
`matrices`		`[]`	Foreign 2-D containers that marshal as an array of row arrays. §8.2
`module_init`		—	Statements the generated module runs when it *loads*, plus the headers declaring them. §5.7
`module_name`		a preset's, else `generator_name`	Default binding module name, used by any bare-string target.
`namespace`		—	Default C++ namespace for entry names carrying no `::` of their own. §4.7
`optimization`		—	Explicit optimisation flag applied after the build type's own. §7.3
`out_dir`		—	Where every target's *built artifact* is copied after each build. A per-target `out_dir` overrides it. §6.4
`out_params`		`{}`	Which parameters a method returns *through a reference*, keyed `"Class::method"`. Never inferred. §5.6
`plugins`		`[]`	Extra `.cpp` sources compiled into the generator driver — a custom backend. Ch. 17
`preset`		—	A named JSON fragment supplying a fixed binding surface. Ch. 9
`rosetta_include`	•	—	Path to rosetta's `include/` directory.
`sequences`		`[]`	Foreign sequence containers that marshal like `std::vector<T>`. §8.1
`targets`	•	—	The language backends to emit. Ch. 6
`user_include`	•	—	Directory holding your class headers, or an array of directories.
`user_lib`		—	Pre-built external libraries to link the bindings against; one object or an array. Ch. 7
`user_sources`		`[]`	User `.cpp` / `.c` files compiled into every binding target. Globs allowed. Ch. 7
`variables`		`[]`	Manifest-local variables, substituted as `$NAME` / `${NAME}` into every string. §4.4
`version`		`0.1.0`	Distribution version stamped into the packaging artifacts (PEP 440). §7.5

Keys beginning with `//` are comments and are ignored everywhere, including inside groups and entries.

A.1 Class entry fields

Field	Req.	Meaning
`header`	•	Filename emitted into `#include`, resolved against `user_include`. May be a glob.
`name`		C++ type name; defaults to the header's stem. May be qualified.
`expose`		The binding name. Must be a plain identifier.
`annotations`		Path to an out-of-line annotation side-car.
`doc`		Description for the document backends.
`extensions`		Free functions exposed as instance methods.
`final`		No trampoline even with public virtuals.
`exclude`		Glob-entry only: patterns whose matches are dropped.
`entries`		Makes this a *group* rather than an entry. §4.7.1

A.2 Function entry fields

Field	Req.	Meaning
`name`	•	Function name, optionally qualified.
`header`	•	Header declaring it.
`doc`		Description.
`expose`		The binding name.
`signature`		C++ function type of the one overload to bind. Required when `name` is overloaded, meaningless otherwise.

A.3 Target entry fields

Field	Req.	Meaning
`lang`	•	The backend. See Appendix C.
`name`		Module / library name; defaults to `module_name`.
`link_options`		Extra linker flags for this target only.
`out_dir`		Where this target's artifact is copied.
`python`		Interpreter the binding is built for.
`requires_python`		Minimum Python version; feeds both CMake and `pyproject.toml`.
`napi_version`		`NAPI_VERSION=` for a Node addon.
`node_engine`		`engines.node` in `package.json`.
`wheel`		Build a wheel for this target on every `-build`. `python` / `nanobind` only — an error elsewhere. §7.5
`wheel_dir`		Where this target's wheels go instead of its own `dist/`. Implies `wheel`.
`wheel_files`		Files / directories bundled into the wheel next to the module (a pure-Python package, data files); a string, or `{"path", "dest"}`. §7.5.1
`wheel_dependencies`		`[project] dependencies` of the emitted `pyproject.toml`.
`wheel_scripts`		Console entry points, name → `"module:function"`.

A bare string in `targets` is shorthand for `{"lang": "..."}`.

A.4 `user_lib` entry fields

`name`	•	Library base name — the `space` in `libspace.dylib`.
`dir`	•	Directory holding it; used for `-L` and rpath.
`link`		`"shared"` (default), `"static"`, or `"dynamic"`.

Appendix B

Annotation reference

B.1 The kinds

Kind	Applies to	Inline / side-car	Meaning
`doc`	fields, methods, classes*	`= rosetta::doc{"..."}` `"doc": "..."`	Description text.
`range`	numeric fields	`= rosetta::range{lo, hi}` `"range": [lo, hi]`	Value bounds; validating setter, clamping slider, documented constraint.
`readonly`	fields	`= rosetta::readonly` `"readonly": true`	Getter-only property.
`combobox`	string fields	`= rosetta::combobox{{"a","b"}}` `"combobox": ["a","b"]`	Allowed choices, maximum 16.
`label`	fields, methods	`= rosetta::label{"..."}` `"label": "..."`	Display-name override in UI backends only.
`button`	methods	`= rosetta::button{"Run"}` `"button": "Run"`	Call-button caption.
`widget::*`	fields	`= rosetta::widget::slider` `"widget": "slider"`	Editor hint; see below.

* Class-level and free-function `doc` come from the manifest's `doc` key, since free functions carry no in-source annotations.

B.2 Widget hints

Hint	Field type	Renders as
`spin`	number	spin box / typed input
`slider`	number + `range`	slider bounded by the range
`textfield`	number	validated free-text input
`checkbox`	number (0/1), bool	checkbox
`color`	string `"#rrggbb"`	colour picker; writes back hex
`multiline`	string	multi-line text area
`radio`	string + `combobox`	radio-button group
`file`	string path	entry + platform file dialog

B.3 Consumption by backend family

Family	Honours
Scripting — `python`, `nanobind`, `node`, `wasm`, `lua`, `julia`, `csharp`, `java`	`doc` (docstrings where the framework has them), `range` (validating setters), `readonly` (getter-only properties). The UI kinds are ignored.
UI inspectors — `qt`, `qml`, `imgui`	Everything.
Documents — `markdown`, `html`, `openapi`, `typescript`	`doc` and the constraints (`range`, `readonly`, `combobox`) rendered into the reference text or schema.

An annotation a backend has no use for is ignored, never an error.

B.4 Inline versus side-car

	Inline	Side-car
Header stays plain C++	**no**	**yes**
Generated binding builds on an off-the-shelf compiler	**no**	**yes**
Works on third-party headers you cannot edit	**no**	**yes**
Expressiveness	full set	full set
Edit requires	regeneration	regeneration

Wiring a side-car:

```
"classes": [
  { "name": "Algo", "header": "Algo.h", "annotations": "Algo.ann.json" }
]
```

B.5 Side-car schema

Keys are member identifiers — fields or methods. Each value is an object carrying any subset of:

Key	JSON type	Example
`doc`	string	`"Visible items"`
`range`	2-element array	`[0, 100]`, `[1e-10, 1e-6]`
`readonly`	boolean	`true`
`combobox`	string array	`["fast", "slow"]`
`label`	string	`"Max iterations"`
`button`	string	`"Run"`
`widget`	string	`"slider"`

A key naming no member of the type **fails the build**, naming the offending key. That is deliberate: a renamed field must not silently lose its metadata.

B.6 Merge rules

- No side-car: the class is un-annotated. Not an error.
- Inline annotations present: the JSON entries are *concatenated* after them.
- Precedence: a reader that keeps the last match lets a JSON entry override an inline one of the same kind; cumulative readers such as `doc` see both.

B.7 Internal

`rosetta::virtual_spec` is synthesised by the walk and never written by users. It marks a method's reflection as virtual or overriding so backends can emit trampolines.

Appendix C

Target reference

C.1 Every `lang` value

`lang`	Output	Builds with	Overloads
`python`	pybind11 extension module	off-the-shelf compiler	all
`nanobind`	nanobind extension module	off-the-shelf compiler	all
`node`	N-API native addon	off-the-shelf compiler	first only
`wasm`	Emscripten / embind module	off-the-shelf emsdk	first only
`lua`	sol2 module, `require`-able	off-the-shelf compiler	first only
`julia`	CxxWrap.jl / jlcxx module	compiler + CxxWrap.jl	all
`csharp`	C-ABI lib + P/Invoke wrappers	compiler + .NET	first only
`java`	C-ABI + FFM wrappers	compiler + JDK	first only
`qt`	Qt Widgets inspector	compiler + Qt 6	all
`qml`	QML / QtQuick inspector	compiler + Qt 6	first only
`imgui`	Dear ImGui inspector app	off-the-shelf compiler	—
`rest`	cpp-httplib server + client	**C++26 toolchain**	first only
`openapi`	OpenAPI 3.1 specification	text output	—
`json`	nlohmann (de)serialisation	**C++26 toolchain**	—
`typescript`	ambient `.d.ts` declarations	text output	first only
`markdown`	API reference document	text output	—
`html`	styled API reference page	text output	—
`paraview`	Server Manager plugin XML	text output	—
`dynamic`	the IR as runtime data	off-the-shelf compiler	all, at call time
`mcp`	MCP server (stdio) for AI agents	off-the-shelf compiler	all, at call time

Each name also accepts a deprecated `-expanded` spelling resolving to the same backend.

C.2 Feature support

Which manifest features reach which backend. A blank cell means the backend ignores the field rather than failing.

lang	module_init	out_dir	extensions	sequences	out_params
python	yes	yes	yes	yes	yes
nanobind	yes	yes	yes	yes	yes
node	yes	yes	yes	yes	yes
wasm	yes	yes	yes	yes	yes
lua	yes	yes		yes	yes
julia	yes	yes			
csharp					
java					
qt					
qml					
imgui					
rest					
typescript			yes	yes	yes
markdown			yes		
html			yes		

C.3 Documentation output

Where each backend puts the documentation, parameter names and default arguments harvested from your headers (chapter 11). A dash means the target has nowhere to put it, not that the data is missing from the IR.

lang	Documentation lands as	Names & defaults
python nanobind	docstring, with `Args:` / `Returns:` sections; the class comment is the type's docstring	`py::arg` / `nb::arg` keyword arguments; real Python defaults (see the import-time rule)
typescript	TSDoc block: prose, `@param`, `@returns`	names in the signature; a defaulted parameter is `optional?`
markdown html	prose, a list of documented arguments, a *Returns* line	`name: type = default` in the signature
openapi	operation `description`; per-argument `description`	argument `title`, and `default` where the value is expressible in JSON
csharp	`/// <summary>` XML doc	names in the wrapper signature (keyword-guarded)
java	Javadoc block	names in the wrapper signature (keyword-guarded)
qt qml imgui	a field's tooltip	the name labels the argument box
dynamic	text in the emitted `MetaClass` tables	names in the parameter tables
mcp	`describe_class` output the agent reads	keyword arguments by parameter name
node wasm lua julia	— (no docstring slot; pair with `typescript`)	— (the wrappers spell positional locals)

`lang`	**Documentation lands as**	**Names & defaults**
`rest`	— (the spec is `openapi`'s job)	names in the browser client's field placeholders
`json paraview`	—	—

C.4 Packaging

Only two backends emit a `make_wheel.py`, so only those two accept the per-target `wheel` / `wheel_dir` (Section 7.5); the field is an error on any other `lang`.

`python`	One wheel per CPython version — pybind11 has no stable-ABI mode. Covering several means running `make_wheel.py` once per interpreter.
`nanobind`	One `abi3` wheel covering every CPython 3.12+; below 3.12, tagged for the exact interpreter that built it.
every other	No packaging step.

C.5 Interop and foreign types

`lang`	`interop: ["eigen"]`
`python`	native numpy, through `<pybind11/eigen.h>`
`nanobind`	native numpy, through `<nanobind/eigen/dense.h>` (dense only)
every other	skips the member — register the concrete type under `sequences` / `matrices` to give them a flat array

`lang`	`std::filesystem::path`	`std::shared_ptr<T>` **returned**
`python`	native (`os.PathLike` in, `pathlib.Path` out)	`py::class_<T, shared_ptr<T>>`
`nanobind`	native	native
`wasm`	string, via a copy adapter	`.smart_ptr<...>` on the class
`lua`	string, via a copy adapter	native (sol2)
`node`	string, via a copy adapter	the JS object adopts the pointer; untrampolined pointees only
`typescript`	`string`	the pointee type

A `shared_ptr<T>` *parameter* is skipped everywhere — take `const T&`.

C.6 Build shapes for `-build`

Backend	Build
`python`, `nanobind`, `rest`, `json`, `lua`, `imgui`, `dynamic`, `mcp`	plain `cmake` configure + build
`node`	`npm install` + `npm run build`
`wasm`	`emcmake cmake` + `cmake -build`
`julia`	`cmake` (locates JlCxx by running `julia`)
`csharp`	`cmake`, then `dotnet build`
`java`	`cmake`, then `mvn package`
`qt`, `qml`	`cmake` with `-DQT_DIR`
`markdown`, `html`, `typescript`, `openapi`, `paraview`	nothing to compile

A missing toolchain is a *skip with a note*, not a failure. A missing `dotnet` or `mvn` downgrades its backend to "OK (native only)".

C.7 External requirements

`python`	pybind11 (`pip install pybind11`)
`nanobind`	nanobind; wheels are `abi3` on Python 3.12+
`node`	npm, and `cmake-js`
`wasm`	an activated emsdk
`lua`	Lua 5.1–5.4 or LuaJIT — **not** 5.5, which sol2 does not support yet. sol2 itself is fetched at configure time
`julia`	CxxWrap.jl; the build runs `julia` to locate JlCxx
`csharp`	a .NET SDK
`java`	a JDK with the FFM API
`qt`, `qml`	Qt 6 — but no `moc` on the generated code
`imgui`	nothing; GLFW and OpenGL3 are auto-fetched
`rest`	cpp-httplib and nlohmann/json, fetched on first configure, plus the C++26 toolchain
`mcp`	nlohmann/json (fetched if missing); optionally Lua 5.3+ for `run_script`, and Python 3.10+ dev files + pybind11 for `run_python` (`-DROSETTA_MCP_PYTHON=ON`)

Appendix D

Troubleshooting

D.1 A method is missing from the binding

Read `bindings/coverage.json` first. It was written before anything was compiled, and it names every member that did not reach a backend, with a reason. The likely entries:

Reason	What to do
`overload_not_expressible`	The target keys methods by name and this overload lost the tie. Wrap it in an `extensions` entry under a distinct name, or reach it through the dynamic model. §5.5
`unmarshalable_signature`	`detail` names the offending type. If it is a foreign container or an Eigen type, Chapter 8 is the fix. If it is a `std::function` or a raw pointer, write an `extensions` wrapper with a marshallable signature.
`no_identifier`	An operator or conversion function. There is no name to bind to — expose it as a named extension method.
`function_template`	A member template has no fixed parameter pack to splice. Add an explicit non-template wrapper.
`hidden_by_derived`	C++ name hiding, honoured deliberately. Add `using Base::f;` to the derived class if you want the base overloads visible.
`sequence_not_adaptable`	A registered sequence with no `std::vector` boundary adapter on this backend.
`extension_has_no_member_pointer`	This backend dispatches through member pointers and cannot take extensions. Expected on `qt`, `qml`, `csharp`, `java`.

If the member is not in `coverage.json` at all, it was not public, or its class is not in `classes`.

D.2 The generated driver does not compile

```
no member named 'vec3' in namespace 'geom'
```

A glob entry derived a class name from a file stem that names no such class. See the table in Section 5.2: list the class explicitly, or add the header to `exclude`.

An error inside a `signature` you wrote

The signature is spliced verbatim. Spell the types the way the header does, fully qualified. `rosetta_gen` only checks its shape — the types are the compiler's business.

An error inside a `module_init` statement

Same cause: those are spliced verbatim into the module entry point. Spell types qualified, and remember there are no arguments available at that point.

The bound header sees a different class than the binding will

A configuration macro is missing from `compile_definitions`. Those are emitted for the *generator driver* as well as the binding targets precisely so the two cannot disagree — if you passed the macro some other way, move it into the manifest.

D.3 A side-car annotation had no effect

- Did you **re-run `rosetta_gen`**? The JSON is baked into `bindings.h` at generation time. A rebuild of the binding alone will not pick it up.
- A key naming no member *fails the build*, so a silent no-effect usually means the annotation reached a backend that ignores it — `label` and `widget` are UI-only. See Appendix B.
- `widget::slider` without a `range` falls back to the default editor. `widget::radio` without a `combobox` does nothing.

D.4 The binding is slow

- Check the build type. Omitting `build_type` now defaults to `Release`, but a `-DCMAKE_BUILD_TYPE=Debug` on the command line wins. An unoptimised pybind11 call costs roughly eight times an optimised one.
- If you are going through the dynamic model, every access is a linear name lookup plus a scoring pass. Cache the resolved `MetaMethod`, and keep an expanded binding for hot paths. §14
- A foreign container crosses *by copy* through a `std::vector` boundary. For bulk data on the Python family, prefer `interop` — the numpy caster avoids the copy where the layout allows.

D.5 The module will not load

- **Missing shared library.** Every `user_lib dir` lands in `BUILD_RPATH` and `INSTALL_RPATH`, so this usually means a dependency was not listed. Add it — and remember the order is the link order.
- **A wasm target linking a native library.** Every `user_lib` entry is taken as `lib<name>.a` built with the *same* emsdk. A native `.so` cannot enter a wasm module.
- **Something the library needs was never initialised.** That is what `module_init` is for. §5.7

D.6 The pipeline produced stale output

- **A generator binary from a previous emit.** If the emitted sources changed but the old `generator` still sits next to the output directory, running it re-emits the old bindings.

The tool prints a rebuild note — believe it.

- **A `user_sources` glob.** Globs expand when `rosetta_gen` runs. A new `.cpp` does not appear by itself.
- **Stale CMake caches** after a toolchain or layout change. Use `-fresh`.

D.7 `-build` skipped a backend

That is by design: a missing toolchain is a skip with a note, not an error. The summary line says which tool was not found.

```
  wasm        SKIPPED (emcc not found -- activate emsdk)
```

A backend that *failed* is reported differently, and `-build` exits non-zero after trying the rest.

D.8 The wrong Python was used

`-cmake-arg -DPython_EXECUTABLE=...` **does not work** — the generated CMake sets that variable with `CACHE ... FORCE` after probing. Pin the interpreter on the target instead:

```
{ "lang": "nanobind", "python": "3.11", "requires_python": ">=3.10" }
```

This also fixes a wheel tagged for the wrong version, since the wheel is built by whichever interpreter runs `make_wheel.py`.

D.9 `-clean` did not remove something

By design in three cases: the manifest is never removed; a folder whose `CMakeLists.txt` lacks the `Generated by rosetta` stamp is never touched; and only the per-backend folders *the manifest declares* are removed, so anything else parked under `bindings/` survives.

```
not removing gen (no rosetta_gen marker in its CMakeLists)
```

D.10 A variable in the manifest did not expand

Only a name declared in `variables` is substituted. `$ENV{HOME}` and any undeclared `${...}` are CMake's and pass through untouched — and a *misspelt* name is indistinguishable from those, so it passes through silently and surfaces as a path that does not exist.

`${NAME}` rather than `$NAME` is worth preferring: `$Px` is one name, `Px`.

D.11 Reporting something

The two artifacts that make a report actionable are `bindings/coverage.json` and the manifest. Between them they say what rosetta thought your API was and what it decided to do about it, which is almost always where the disagreement is.